光伏电站出力特性与融合气象信息的功率预测

摘　要

光伏出力既有由太阳位置决定的日周期，也受天气变化影响。本文选取 PVOD 数据集中的 station08 电站，利用 15 分钟功率、现场气象和数值天气预报（NWP）数据，分析出力特性，比较历史功率预测、气象信息融合与局地统计订正的效果。样本覆盖 2018 年 7 月至 2019 年 6 月的 345 个完整观测日，未满一年。主实验在四个指定月末周固定起报，预测未来七天，训练标签与历史功率输入均早于起报时刻。

针对问题一，建立晴空辐照、板面转换与逆变器相结合的参考功率模型。月度日积分估计均值在 2019 年 5 月最高，为 99.23 MWh，2 月最低，为 49.56 MWh；白昼实际功率较参考平均低 5.34 MW。相邻 15 分钟功率变化绝对值的 95 分位为 2.798 MW，其变化与现场辐照变化的相关系数为 0.843。

针对问题二，比较持续性、近 14 日同时刻均值、晴空比例和残差梯度提升树，以起报前独立验证周选定主方案。四周 1346 个白昼点的容量归一化均方根误差（nRMSE）为 0.1210；固定晴空比例基线为 0.1184，表明复杂模型与验证选优均不必然优于简单基线。

针对问题三，在目标时刻 NWP 于起报时可用的假设下，加入气象字段后 nRMSE 降至 0.1086，相对第二问主方案下降 10.28%。中、高预报辐照档改善，唯一低辐照测试日退化。按日期分层重抽所得误差差的 95% 区间跨零，尚不足以确认稳定收益。

针对问题四，采用辐照、气温订正与功率映射组成的两阶段模型。非线性局地订正 nRMSE 为 0.1193，较同一功率映射的原始气象对照上升 3.94%，未取得总体改善。由于原始 NWP 空间尺度及站点提取关系缺失，本实验仅验证单站订正，未验证空间分辨率提升。

结果说明，气象信息在固定测试样本上有预测价值，但需结合场景和日期差异评价。七天固定起报的总体结果不能全部解释为 24—48 小时日前精度；实际应用仍需补齐预报发行信息并开展逐日回测。

关键词：光伏功率预测；晴空参考模型；梯度提升树；数值天气预报；局地统计订正

目　录

一、问题重述.....—

二、问题分析.....—

三、模型假设.....—

四、符号说明.....—

五、问题一的模型建立与求解.....—

六、问题二的模型建立与求解.....—

七、问题三的模型建立与求解.....—

八、问题四的模型建立与求解.....—

九、模型检验与不确定性分析.....—

十、模型的评价及推广.....—

十一、结论.....—

参考文献.....—

附录.....—

一、问题重述

1.1 问题背景

光伏发电出力受太阳辐照、云量、气温等天气条件影响，具有明显的随机性与间歇性。电网在安排次日供电计划时，需要提前掌握光伏电站的出力水平。若预测偏高，会因预留备用容量不足而影响供电安全；若预测偏低，则会增加常规机组备用、抬高运行成本。因此，提高光伏功率日前预测精度具有直接的工程价值。

随着数值天气预报（NWP）的发展，将预报的辐照、气温等气象要素纳入预测模型已成为主流思路。与此同时，光伏电站的出力还受到局地云场、地形与设备状态的影响，而 NWP 本身存在空间分辨率不足的问题。本题围绕上述背景，给出某光伏电站的实测功率、现场气象与 NWP 预报数据，要求建立预测模型并检验气象信息的价值。

本次建模采用 PVOD v1.0 公开数据集[1–3]。该数据集包含多个光伏电站的多源功率输出数据，本文选取 station08 电站作为主站，station01 仅用于问题一的质量对照。

1.2 问题重述

根据题目与附件 1（误差分析方法）[4]的要求，需要完成以下四项工作：

问题一：分析光伏电站出力的变化特性。要求对实测数据进行质量检查与必要处理，刻画功率的日周期、月度与季节变化以及短时波动特征，并可与参考水平作对比。

问题二：在不使用现场逐时气象和 NWP 预报字段、以历史功率为主要输入的条件下，建立次日功率预测模型。要求给出可解释的基线方法与回归模型，并按规定的测试集评价。

问题三：在问题二的基础上加入 NWP 气象信息，验证气象信息带来的增益，并分析在不同天气（辐照）条件下模型的适用性。

问题四：处理 NWP 的空间尺度问题，检验将粗分辨率气象信息映射到电站局地尺度的效果。

题目规定：测试集为第 2、5、8、11 月的最后一周，即 2018-08-25—08-31、2018-11-24—11-30、2019-02-22—02-28、2019-05-25—05-31（北京时间），每周 672 点，共 2688 点。误差分析须按附件 1 的定义执行，功率以装机（开机）容量归一化。

二、问题分析

2.1 问题一的分析

问题一的目标是认识数据本身：数据是否可靠、出力的时间规律如何、以及在给定天气条件下实际出力与理论水平的差距有多大。

已有条件是 15 分钟分辨率的实测功率与现场气象。需要达到的目标是：给出可信的清洗结论、量化日与季节的出力形态、并建立一个可解释的参考基准。所选数学方法为"数据质量核查 + 确定性物理参考模型 + 描述性统计分析"。工作流程为：先核查时间网格与数值合法性，再做清洗，随后用晴空辐照与板面转换模型建立参考功率，最后从月度、季节、短时波动与参数敏感性几个角度刻画特性。需要强调，参考功率用于解释差异，不是预测，也不是严格上限。

2.2 问题二的分析

问题二在不使用现场逐时气象和 NWP 预报字段的条件下预测次日功率，其实质是检验"仅凭功率自身的历史规律能做到什么程度"。

由于光伏出力具有强日周期，且晴空条件下的出力近似只由太阳几何决定，因此合适的思路是先建立简单、可解释的基线：持续性（最近一个完整日同时刻）、气候态（近 14 日同时刻均值）、晴空比例（以晴空参考为骨架、用历史功率标定比例系数）。在此基础上，再用梯度提升树学习相对物理基线的残差，以捕捉历史状态与季节的剩余规律。评价采用相对容量的归一化误差，并严格遵守时间因果：预测第 D 日时，训练与历史输入均早于起报时刻。

2.3 问题三的分析

问题三在问题二的基础上加入 NWP 预报的辐照、气温、湿度、风速、压力等要素。核心问题是：气象信息是否真的带来增益，以及在什么条件下有效。

合适的思路是在相同历史特征与残差目标上增加 NWP 字段，得到第三问模型；再通过"辐照消融"（只保留辐照类字段）判断非辐照信息是否有额外贡献；进一步按预报晴空指数把样本分为低、中、高三档辐照场景，检验分场景表现，并允许场景融合在必要时不启用 NWP。评价同样只在白昼点上进行，并另做按日期的重抽检验以判断改善是否稳定。

2.4 问题四的分析

问题四关注 NWP 的空间尺度问题：粗分辨率网格气象如何映射到电站所在的局地尺度。

当前数据只提供单站 NWP 与现场测量，未说明原始网格尺度及站点提取关系，无法验证

分辨率提升。本文采用"模式尺度预报量→电站测点气象"的单点统计降尺度，即对辐照和气温做回归订正。因此本文的分析策略是：在单点意义上检验"气象订正是否改善功率回测"，并与原始气象对照、均值偏差订正及问题三强基线比较，从而给出有边界的结论。

三、模型假设

1. 在数据质量核查与异常标记之后，保留观测用于刻画样本期运行状态；未记录的停机、限电等因素不作独立识别。

2. 功率采样按 15 分钟计，时间戳视为采样时刻进行点值对齐；采样为瞬时值或区间均值尚未确认（见第十节与结论的边界说明）。

3. 题目未提供随时间变化的开机容量记录，故以装机容量 20 MW 作为开机容量，用作误差归一化的容量基准 C。

4. 电站直流组件额定容量按元数据计算，即 78042 块 × 265 Wp = 20.68113 MWp；交流限制暂设为 20 MW。两者分别定义，均未经独立核实。

5. 参考模型的物理参数作如下假设：海拔 0 m（另设 200 m 作敏感性）、地面反照率 0.25、组件温度固定 25 ℃、逆变器额定效率 0.96。组件数量、规格及朝南 33° 倾角来自元数据，其余为参考模型的工作假设。

6. NWP 字段在每次起报时刻均可获得（"可用性假设"）。原始数据未提供发行时刻、预报版本与提前量，因此该假设未获证实，第三、四问的结论均限于此假设下的条件回测。

7. 测试周为四个自然月最后 7 日；白昼定义为几何太阳高度角大于 0°，不依据实际功率判定。

8. 向前实验中，训练标签与历史输入均严格早于起报时刻；七天目标日共用起报时的历史信息，不进行真值递推。

四、符号说明

表 1 主要符号与单位

符号	含义	单位
P_i	第 i 个白昼点的实测功率	MW
$\hat{P}_i$	对应的预测功率	MW

符号	含义	单位
C	误差归一化容量（取 20 MW，装机容量代替开机容量）	MW
e_i	归一化误差，$e_i=(\hat{P}_i-P_i)/C$（正表示预测偏高）	无量纲
E_{rmse}	归一化均方根误差	无量纲
E_{mae}	归一化平均绝对误差	无量纲
E_{me}	归一化平均误差（偏差）	无量纲
r	实测与预测的 Pearson 相关系数	无量纲
C_R	准确率，$C_R=(1-E_{rmse})\times 100\%$	%
Q_R	合格率，$\lvert e_i\rvert<0.25$ 的样本比例	%
G_{poa}	板面（倾斜面）总辐照	W/m²
G_{cs}	晴空辐照	W/m²
P_{cs}	晴空参考功率	MW
a	晴空比例系数（过原点最小二乘标定）	无量纲
$x_{o,t}$	起报时刻 o 下目标时刻 t 的特征向量	—
f_2，f_3	问题二、问题三的残差梯度提升树	—
k_d	预报晴空指数，即目标日白昼 G_{NWP}/G_{cs} 的均值	无量纲
w_s	场景 s 下第三问模型的融合权重	无量纲
D_G，D_T	辐照、气温的局地统计订正映射	—
F	由真实局地气象到功率的映射	—

误差指标定义如下（n 为参与评价的白昼样本数）：

$$E_{rmse}=\sqrt{\,},\quad E_{mae}=\frac{1}{n}\sum_{i=1}^{n}|e_i|,\quad E_{me}=\frac{1}{n}\sum_{i=1}^{n}e_i, \tag{1}$$

$$C_R=(1-E_{rmse})\times 100\%,\quad Q_R=\frac{1}{n}\sum_{i=1}^{n}1(|e_i|<0.25)\times 100\%. \tag{2}$$

其中 E_{rmse} 与 E_{mae} 为归一化（无量纲）误差，C_R、Q_R 以百分数报告。E_{rmse} 值 0.1086 对

应约 10.86% 的容量归一化均方根误差。

五、问题一的模型建立与求解

5.1 数据来源与预处理

主站数据取自 data/PVOD/station08.csv 与 metadata.csv。时间按 UTC 读取并转换为北京时间，覆盖 2018-07-01 00:00 至 2019-06-13 23:45，共 33120 条 15 分钟记录。按题目要求，参赛数据集的关键信息以表格形式呈现如下。

表 2 光伏电站及数据规格

数据规格	参数值
数据来源（公开数据集链接）	PVOD v1.0（Multi-source photovoltaic power output dataset with Python toolkit），Science Data Bank，2021，DOI:10.11922/sciencedb.01094；本项目使用其解压文件，主站文件为 data/PVOD/station08.csv，附带 metadata.csv
光伏电站装机容量	元数据容量 20000 kW，即 20 MW（作为开机/评分归一化容量的假设）。直流组件额定容量按 78042 块 × 265 Wp = 20.68113 MWp 计算；交流限制亦暂设 20 MW。二者分别表述，均未经独立核实
发电功率和 NWP 数据时间分辨率	1 采样点 / 15 min
发电功率和 NWP 数据起始—截至时间	2018.07.01—2019.06.13（北京时间 2018-07-01 00:00 至 2019-06-13 23:45；UTC 2018-06-30 16:00 至 2019-06-13 15:45），不足完整一年
NWP 属性	全球辐照、直射辐照、气温、湿度、风速、风向、气压（对应 nwp_* 列）；现场气象另有 lmd_* 列（总辐照、散射辐照、气温、气压、风向、风速）。原始数据未提供 NWP 发行时刻、预报版本与提前量
站点信息	主站 station08：东经 113.89999°、北纬 36.70761°，组件朝南倾角 33°；海拔未提供（暂设 0 m，另设 200 m 作敏感性）。对照站 station01：东经 117.45722°、北纬 38.18306°，朝南 33°，74000 块 × 270 Wp = 19.98 MWp

核查结果：无重复时间戳、无字段缺失与基础非法值；时间网格缺失 288 点，全部位于 2018-11-11 至 2018-11-13 三个整日，缺失日期不补零、不插值。共有 345 个完整观测日（每日 96 点功率完整），观测跨度为 348 个日历日，未达到题目要求的完整一年；缺测与未覆盖时

段均未用其他站点数据拼接。

清洗沿用 station01 的阈值与规则：仅当功率与六项现场气象连续至少 4 点完全相同，且段内至少一个点同时满足太阳高度角低于 −6°、功率大于 0.01 MW、现场总辐照大于 0 时，才将后续重复值置为缺失。本次 station08 未发现可疑持值段，修改字段值 0 个。白昼强辐照下的零/低功率 28 点、近地平线夜间正功率 9 点予以保留并标记。需要说明：未发现异常不等于所有物理口径都已核实。

5.2 晴空参考功率模型

为刻画晴空条件下的参考出力形态，采用 pvlib 实现物理计算链[5–6]。计算链为：太阳位置 → Ineichen 晴空辐照 → 各向同性板面辐照转换 → 固定 25 ℃ 组件直流功率 → PVWatts 逆变器交流输出。直流参考功率为

$$P_{dc,ref}(t)=20.68113\times\frac{G_{poa}(t)}{1000}\times(1-L)(MW),\quad(3)$$

其中 L 为额外损耗比例，逆变器直流输入限制取 $20/0.96$ MW，额外损耗与逆变器损耗分别计算。夜间参考设为 0，实测值不作修改。模型参数取自元数据与工作假设（见第三节），未通过实测功率拟合。

必须强调：该参考曲线是简化物理工况下的估计，不是任何天气下都成立的严格发电上限，也不是日前功率预测，因此其后与实测的偏差不能称为预测误差。

5.3 月度与日内特性分析

日积分估计为每个完整日的 96 个采样功率之和乘以 0.25 h。由于功率是瞬时值还是区间均值尚未确认，故仅称"积分估计"，不宣称精确实测电量。

表 3 月度日积分估计

月份	观测日	完整功率日	日积分估计均值 (MWh)	同日参考均值 (MWh)
2018-07	31	31	69.562	140.088
2018-08	31	31	65.618	139.457
2018-09	30	30	63.617	135.540
2018-10	31	31	75.777	127.330
2018-11	27	27	54.216	116.723
2018-12	31	31	52.465	116.570

月份	观测日	完整功率日	日积分估计均值 (MWh)	同日参考均值 (MWh)
2019-01	31	31	51.943	116.292
2019-02	28	28	49.560	132.864
2019-03	31	31	93.896	141.805
2019-04	30	30	72.725	145.501
2019-05	31	31	99.234	144.081
2019-06	13	13	88.282	142.532

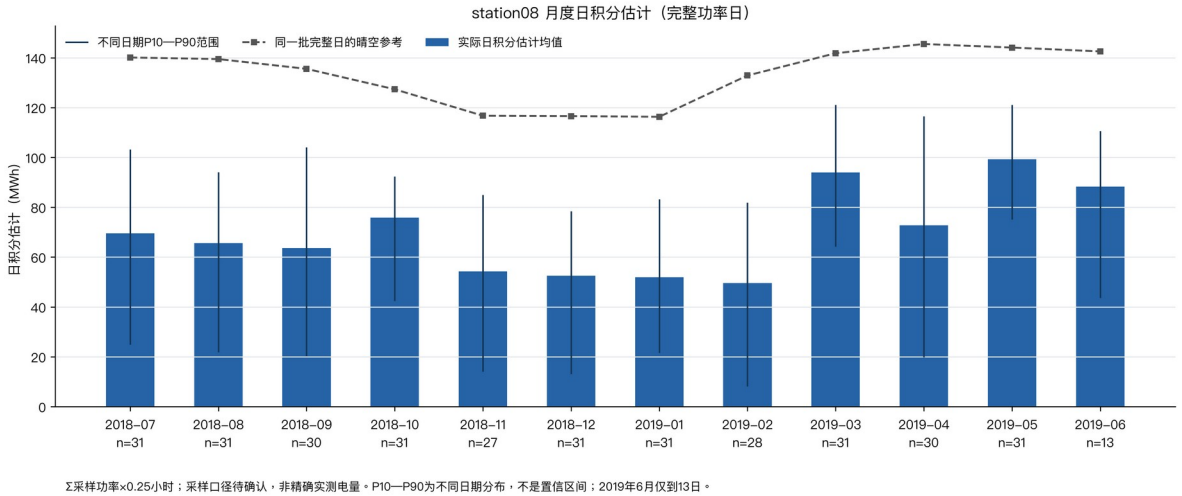


图 1 station08 月度日积分估计（完整功率日）

日积分估计最高为 2019 年 5 月的 99.23 MWh，最低为 2019 年 2 月的 49.56 MWh。2018 年 11 月缺失 3 日，2019 年 6 月仅到 13 日，故不能把样本均值推广为完整月结果，也不能把月度差异唯一归因于季节。

按 3—5 月春、6—8 月夏、9—11 月秋、12—2 月冬划分季节，得到各季节的平均日内曲线峰值：

表 4 季节平均日内曲线特征

季节	完整功率日	平均曲线峰值 (MW)	峰值时刻	包含月份
春	92	12.181	11:45	2019-03、04、05
夏	75	9.594	13:15	2018-07 、 08 、 2019-06
秋	88	10.028	12:15	2018-09、10、11

季节	完整功率日	平均曲线峰值 (MW)	峰值时刻	包含月份
冬	90	9.266	12:30	2018-12 、 2019-01、02

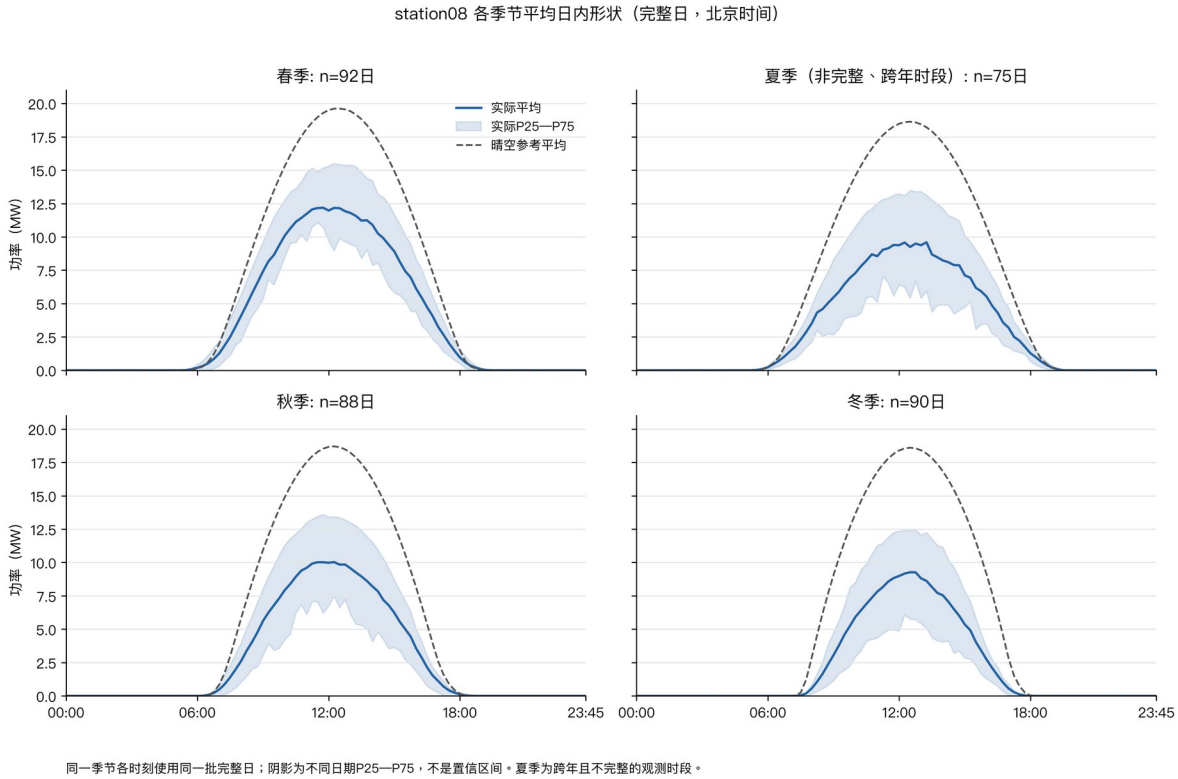


图 2 station08 各季节平均日内曲线（完整功率日）

春季峰值最高且出现最早，冬季峰值最低；夏季样本跨 2018 年 7、8 月与 2019 年 6 月 1—13 日，并非单一完整夏季。平均峰值时刻是样本统计结果，不代表逐日峰值时刻的范围。

5.4 参考偏差与 15 分钟波动

在共同有效白昼点 16476 个上，实际功率减参考的平均值为 −5.340 MW，平均绝对偏差为 5.355 MW，实际超过参考的点有 816 个。该偏差反映简化物理工况与实际运行的差距，不能写成预测误差。

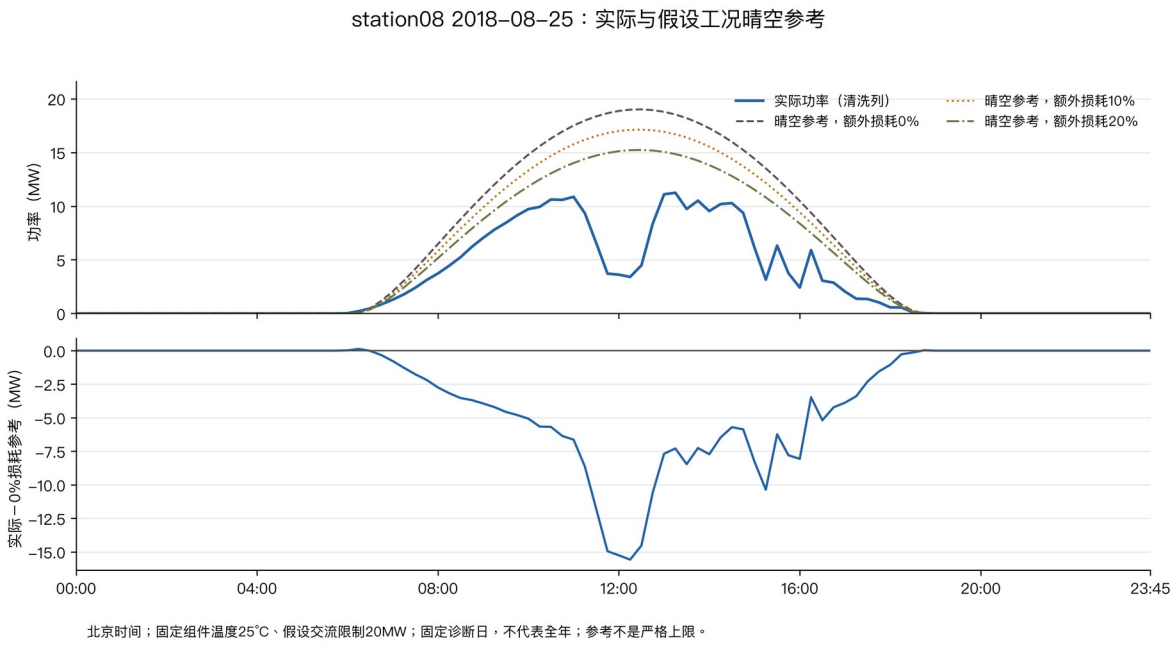


图 3 station08 固定诊断日（2018-08-25）实际功率与晴空参考对照

对连续 15 分钟且两端均为白昼且功率有效的点对计算变化量，共 16131 个有效点对：绝对变化中位数 0.578 MW，95 分位 2.798 MW，最大值 12.229 MW；功率变化与现场辐照变化的 Pearson 相关系数为 0.843。相关性支持"辐照变化是功率波动的重要来源"这一解释，但不足以认定唯一因果。

最大变化事件出现在 2019-05-11 15:00，功率在 15 分钟内变化 12.229 MW，同期现场总辐照变化 648.0 W/m²。该事件取自 station08 结果表。

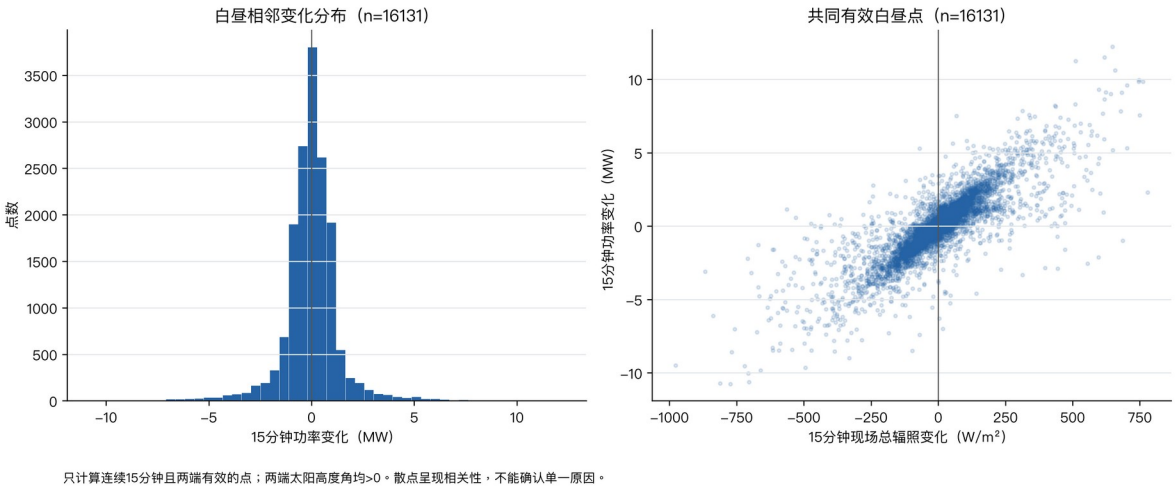


图 4 station08 白昼相邻 15 分钟功率变化分布与辐照变化关系

5.5 参数敏感性与站点对比

考虑额外损耗 0%、10%、20% 以及海拔 200 m 四种情景，结果如下：

表 5 晴空参考参数敏感性

情景	白昼样本数	参考峰值(MW)	平均实际减参考(MW)	平均绝对偏差(MW)	实际超参考点数
额外损耗 0%	16476	20.000	−5.340	5.355	816
额外损耗 10%	16476	18.130	−4.228	4.253	1138
额外损耗 20%	16476	16.129	−3.113	3.241	2653
海拔 200 m，损耗 0%	16476	20.000	−5.380	5.398	891

损耗越大，参考越接近实测，但这不能证明真实损耗就是该值；海拔 0 m 与 200 m 均为假设。尚缺实际组件温度、海拔、逆变器数量与运维信息。

进一步与 station01 对比。两站各有 345 个完整功率日，但日期集合不同：station01 因清洗排除 3 个日期，station08 原始缺 3 个日期。质量核查对比为：

表 6 两站数据质量比较

站点	原始记录	缺失时间点	可疑持值段	清洗缺失功率点	完整功率日	强辐照零低功率点	清洗后夜间正功率点
station01	33408	0	2	138	345	65	6
station08	33120	288	0	0	345	28	9

为控制日历覆盖，仅比较两站共同完整的 342 天（各 32832 条记录）：

表 7 两站共同完整日对比

站点	共同完整日	日积分估计均值(MWh)	强辐照零低功率点	变化绝对值 P95(MW)	功率-辐照变化相关系数
station01	342	87.318	65	3.587	0.784
station08	342	69.095	28	2.800	0.842

station08 在当前规则下没有 station01 的深夜多字段持值问题，强辐照零/低功率标记也更少，适合作为主分析站。但它有 3 天原始缺测，清洗后完整日数量并未增加，近地平线夜间正功率也没有更少。因此只能说当前已检查到的异常较少，不能说全面优于 station01，更不能据此推断后续预测精度更高。

5.6 问题一小结

问题一完成了 station08 的质量核查、晴空参考建模与出力特性分析。主要结论为：数据时间网格完整（除 3 个缺日）、无字段级非法值；出力具有清晰的日周期与季节性；实际出力较简化参考平均偏低约 5.34 MW；15 分钟级波动中位数约 0.58 MW，且与辐照变化高度相关。这些结论为后续预测建模提供了数据口径与物理直觉。

六、问题二的模型建立与求解

6.1 数据与统一评价口径

数据：沿用 data/processed/station08_q1_clean.csv 与问题一的参考假设。原始 UTC 转北京时间，2018-07-01 至 2019-06-13，共 33120 点、345 个完整日，缺 2018-11-11 至 11-13 整 3 日，不补零。全部日期网格用于太阳几何计算，缺测标签始终不参与训练与评分。

测试集：按本地自然月末 7 日，即 2018-08-25—08-31、2018-11-24—11-30、2019-02-22—02-28、2019-05-25—05-31，各 672 点，共 2688 点；白昼采用几何太阳高度角大于 0°，不依据实际功率判定。

结果提交格式：按题目表 2 填写每个测试周的逐点结果（起报时间、预报时间、实际功率、各方法预测功率），四周共 2688 条，作为附件提交。本文采用"每周开始前一日 00:00 固定起报、预测七天"的口径生成该表，对应提前量 24 至 191.75 小时，七天内不更新真实功率输入。该表与证据包中的逐点预测一一对应。这里将“预测时间范围 7 天”解释为固定起报多步预测；仅第一个目标日处于题目所述 24—48 小时日前窗口，后六日是延伸预测，不能把总体指标全部称为日前精度。

误差指标：按第四节定义，$e_i=(\hat{P}_i-P_i)/C$，$C=20$ MW，即预测减实测，正值表示预测偏高。附件 1 中"实际/预测"符号存在歧义，本文统一采用上述约定。C_R、Q_R 以百分数报告，Q_R 中"合格"的条件是归一化绝对误差严格小于 0.25。

两种评价协议必须区分：

- 向前协议：在每个起报时刻只用此前的非测试标签训练，用于核对时间因果关系。

- 补充离线协议：使用整个数据范围的非测试日期训练，符合题目的静态划分，但训练集中可能包含晚于早期测试周的日期，因此不能作为真实向前预测能力的证据。

两种协议测试范围相同，均永久排除四周测试功率作为模型标签或历史输入。

容量口径说明：20 MW 仅为评分归一化容量假设，当前没有随时间变化的开机容量记录；直流组件容量 20.68113 MWp 与假设的 20 MW 交流限制分别表述。

6.2 模型与输入

问题二不读取现场逐时气象或 NWP 字段，但采用太阳几何和预先计算的晴空参考；后者含 pvlib 的气候浑浊度资料，不等同于完全没有气象先验。考虑三类基线与一个回归模型：

(1) 持续性基线：重复起报前最后一个完整非测试日的 96 点功率。

(2) 气候态基线：取起报前 14 天非测试日同时刻功率的均值（缺日不计入分母）。

(3) 晴空比例基线：以晴空参考为骨架，用起报前 28 天有效白昼功率与参考功率作过原点最小二乘，标定比例系数。其中 H(o) 为起报前 28 天内的非测试、白昼、实测功率有效且参考功率大于 0.5 MW 的时刻集合。

$$a(o)=\frac{\sum_{t\in H(o)} P_t P_{cs,t}}{\sum_{t\in H(o)} P_{cs,t}^2},\ \hat{P}_{phys,t}=a(o)P_{cs,t}. \quad (4)$$

(4) 残差梯度提升树：以物理基线（晴空比例）为骨架，对其残差建模：

$$\hat{P}_{2,t}=\Pi_{[0,C]}\left[a(o)P_{cs,t}+Cf_2(x_{o,t})\right], \quad (5)$$

其中 f_2 为直方图梯度提升树[7]，$\Pi_{[0,C]}$ 为投影裁剪。特征向量 $x_{o,t}$ 包含：太阳高度角与方位角、时刻与年内周期编码、参考功率、目标日序号、最后完整日同时刻功率、14 日同时刻均值/中位数/标准差、最近 7 日平均功率以及历史晴空系数。所有历史功率均早于起报时刻 o，七天目标日共用起报时的历史信息，不进行真值递推。所有模型的夜间输出置 0，白昼输出裁剪到 $[0,20]$ MW；此处只裁剪预测，不改变评分用的真实功率。

6.3 训练与选参

候选历史起报从首日后第 14 天开始、每 3 天一次，每次生成七天目标。历史不足的日期不作为残差树训练目标，但仍可用作历史输入。训练标签排除缺失、夜间与四个测试周；同一目标时刻可对应 1—3 个起报提前量，训练按起报—目标对等权，因此不同真实时刻的总训练权重并不相同。验证起报为测试起报前 8 天，验证目标为其后第 1—7 天，验证模型只用验证起报之前的标签训练。

树的关键参数为：学习率 0.05、180 轮、叶节点候选 7 或 15、叶子最少 30 样本、L2 正则 2、随机种子 20261014、不启用随机内部早停。各方法按独立验证周的白昼 nRMSE 选择叶节点数，第二问主方案再从四个历史候选模型中按验证 nRMSE 选优。测试真值完全不参与选参。最终向前拟合纳入测试起报前的验证标签，但仍不纳入任何测试周标签。

6.4 求解结果与分析

向前协议下，各模型在 1346 个白昼点上的结果如下（同时列出第三、四问模型以便统一复核）：

表 8 向前协议下各方法总体指标

模型	白昼样本	nRMSE	归一 MAE	归一 ME	r	CR(%)	QR(%)
Q2 持续性	1346	0.1729	0.1182	+0.0911	0.8126	82.71	83.95
Q2 近 14 日气候态	1346	0.1199	0.0849	−0.0238	0.8499	88.01	94.43
Q2 晴空比例	1346	0.1184	0.0870	−0.0152	0.8543	88.16	94.87
Q2 历史树模型	1346	0.1403	0.0993	+0.0670	0.8384	85.97	90.34
Q2 验证选优	1346	0.1210	0.0845	−0.0004	0.8408	87.90	94.28
Q3 历史+NWP	1346	0.1086	0.0747	+0.0245	0.8817	89.14	95.62
Q3 仅加辐照	1346	0.1247	0.0897	+0.0374	0.8474	87.53	92.94
Q3 场景融合	1346	0.1141	0.0787	+0.0132	0.8616	88.59	94.50
Q4 原始气象对照	1346	0.1148	0.0835	+0.0164	0.8605	88.52	96.36
Q4 均值偏差订正	1346	0.1329	0.1071	+0.0660	0.8565	86.71	94.73
Q4 非线性局地订正	1346	0.1193	0.0808	+0.0422	0.8717	88.07	92.87

分析：

1. 简单基线并不弱。晴空比例（0.1184）与近 14 日气候态（0.1199）均优于历史树模型（0.1403）。这说明在有效观测不足一年的条件下，仅靠历史信息时增加非线性复杂度并不必然提高精度。

2. 验证选优固定了主方案的选择规则。第二问主方案在独立验证周选优得到 0.1210，其组成模型在各测试周并不相同（分别为气候态、晴空比例、持续性等），说明各验证周偏好的历

史模型不同。该规则避免由测试误差倒推方案，但本次并未优于固定晴空比例基线。

3. 持续性最差（0.1729），符合其忽略日间天气变化的预期；但其 ME 为正（+0.0911），反映系统性偏高。

4. 需要强调，测试表内某候选表现较好不能反过来替换验证选优规则，否则构成对测试集的信息泄漏。

6.5 问题二小结

问题二建立了持续性、气候态、晴空比例三种基线与残差梯度提升树，并按独立验证周选优。在向前协议下得到 0.1210 的 nRMSE。结果表明：仅用历史功率时，物理骨架型简单基线的表现与复杂模型相当甚至更好，为后续引入气象信息提供了合理的比较起点。

七、问题三的模型建立与求解

7.1 融合方法与消融设计

在问题二相同的历史特征与残差目标上，增加 NWP 的全球辐照、直射辐照、气温、湿度、风速、压力、风向正余弦以及预报晴空指数，得到模型 f_3。需要说明，NWP 原始列是预报量而非未来现场测量。

辐照消融实验只保留全球/直射辐照与预报晴空指数，其余建模流程一致，两套特征方案分别选择叶节点数。因此，这属于各自调参后的特征方案比较，不能把全部误差差异单独归因于新增字段。

7.2 场景划分与验证融合

对目标自然日太阳高度角大于 10° 的预报点，计算预报晴空指数 k_d（分母至少 20 W/m²）：

- 低辐照：$k_d \le 0.35$

- 中辐照：$0.35 < k_d \le 0.65$

- 高辐照：$k_d > 0.65$

该划分只依赖当日预报与太阳几何，不读取当日功率或现场天气。这些名称是"预报辐照档"，不应直接写成已确认的阴雨/云/雾类别。

场景融合为 $\hat{P}_s = (1 - w_s)\hat{P}_2 + w_s \hat{P}_3$，权重 w_s 从 $\{0, 0.25, 0.5, 0.75, 1\}$ 中按验证误差选择；

若某类验证白昼点不足 20，则沿用总体最优权重。该规则没有最低日期数约束：2018 年 8 月验证周中辐照档虽有 53 点，却只有 1 天；2018 年 11 月无低辐照日，2019 年 2 月无中辐照日，2019 年 5 月只有高辐照日，缺失场景均回退总体权重。该设计允许在无收益时不启用 NWP。

向前协议下，分场景 nRMSE 如下：

表 9 预报辐照分档结果

预报辐照档	白昼样本	Q2 验证选优	Q3 历史+NWP	Q3 场景融合	Q4 非线性局地订正
低（$k_d \le 0.35$）	52	0.1052	0.1304	0.1126	0.1515
中（$0.35 < k_d \le 0.65$）	301	0.1464	0.1189	0.1186	0.1317
高（$k_d > 0.65$）	993	0.1130	0.1040	0.1128	0.1132

相对变化（负值表示误差降低）：低辐照档 0.1052 → 0.1304，变化 +23.88%；中辐照档 0.1464 → 0.1189，变化 −18.81%；高辐照档 0.1130 → 0.1040，变化 −8.03%。

可以看出：加入 NWP 在中、高辐照场景带来改善，但在低辐照场景反而退化；场景融合的总体 nRMSE 为 0.1141，并未超过直接全字段 NWP 模型（0.1086），故不能宣称场景门控必然更优。低、中、高辐照档分别覆盖 1、6、21 个日期。低辐照档的 52 点全部来自 2018-08-25，因此该退化只能说明这一测试日的表现，尚不足以推断低辐照天气的一般规律。

7.3 分月结果

按测试月汇总 nRMSE：

表 10 测试月份的功率预测误差

测试月	Q2 验证选优	Q3 历史+NWP	Q3 场景融合	Q4 原始气象对照	Q4 非线性局地订正
2018-08	0.1206	0.1373	0.1350	0.1262	0.1617
2018-11	0.1248	0.1021	0.1021	0.1114	0.1050
2019-02	0.1230	0.0883	0.1180	0.0964	0.1008
2019-05	0.1172	0.0969	0.0969	0.1190	0.0934

Q3 相对 Q2 的分月变化为：2018-08 变化 +13.86%、2018-11 变化 −18.21%、2019-02 变化

−28.28%、2019-05 变化 −17.32%。可见四个月中只有 2018 年 8 月变差，其余三个月明显改善。该现象说明总体改善不能写成"各季节均改善"。

7.4 结果分析与小结

第三问的主要发现是：

1. 加入 NWP 在固定样本上降低了总体误差。向前 nRMSE 由 0.1210 降至 0.1086，相对下降 10.28%。相对固定晴空比例和气候态基线，降幅分别为 8.33% 和 9.45%。

2. 收益不均匀。低辐照场景退化、2018 年 8 月变差，说明气象信息的作用依赖具体条件。

3. 稳定性待进一步检验。按日期分块的 95% 重抽区间跨 0（见第九节），因此只能报告固定测试样本上的点估计改善。

八、问题四的模型建立与求解

8.1 可验证范围与问题界定

空间降尺度的目标是描述粗尺度预报未能直接表达的局地气象差异。站点统计回归可作为候选方法，但其空间解释仍需要原始 NWP 分辨率及站点提取关系。当前文件只有单站预报和现场测量，未记录上述信息，也没有可用于插值的邻域格点，故本文仅检验局地统计订正的预测效果。

本文实现的仅是"模式尺度预报量→电站测点气象"的单点统计降尺度的有限形式，即对辐照与气温做回归订正，并检验其对功率回测的作用。本实验未验证空间尺度提升。

8.2 双阶段统计映射方法与对照设计

只用训练时段的现场辐照与气温作为标签，学习两个订正映射：D_G（NWP 与太阳几何 → 现场辐照，目标按 1000 W/m² 缩放）与 D_T（NWP 与太阳几何 → 现场气温减 NWP 气温的残差）。令辐照树的无量纲输出为 gθ，则 D_G=max(1000gθ,0)，单位为 W/m²；D_T 输出温度残差，单位为 ℃。两者均采用 15 叶、180 轮的固定提升树。随后只用训练时段的真实局地辐照、气温与太阳几何，学习相同的功率映射 F。树拟合的标签为 P/C；下式中 F 已包含乘以 C 的反缩放、白昼 [0,C] 裁剪及夜间置零：

$$\hat{P}_{raw}=F\left(G_{NWP},T_{NWP},\text{solar}\right), \quad (6)$$

$$\hat{P}_{bias}=F\left(max\left(0,G_{NWP}+G_{obs}-\overline{G}_{NWP}\right),\ T_{NWP}+T_{obs}-\overline{T}_{NWP},\ \text{solar}\right), \quad (7)$$

$$\hat{P}_{local} = F\left(D_G(\mathrm{NWP}, \mathrm{solar}),\ T_{NWP} + D_T(\mathrm{NWP}, \mathrm{solar}),\ \mathrm{solar}\right). \quad (8)$$

偏差均值仅由对应训练样本计算。同一 F 用于三组对照：原始气象对照隔离"映射输入订正"带来的增益；均值偏差订正隔离简单偏差修正与非线性效应；非线性局地订正为本文方法。测试时没有真实局地辐照或气温作为预测输入，现场测量只在生成预测后用于独立的气象误差评价。为避免用弱对照掩盖整体效果，同时与第三问强基线比较。

8.3 求解结果与分析

气象订正本身的误差（以现场测量为标签，向前协议）为：

表 11 局地气象订正误差

现场标签	气象输入	白昼点	RMSE（原单位）	MAE（原单位）
GHI (W/m²)	原始 NWP	1346	152.187	106.864
GHI (W/m²)	非线性局地订正	1346	160.316	104.848
气温 (℃)	原始 NWP	1346	1.613	1.219
气温 (℃)	非线性局地订正	1346	1.672	1.148

可以看到：辐照的 MAE 略降（106.86 → 104.85 W/m²）但 RMSE 反而上升（152.19 → 160.32 W/m²）；气温的 MAE 也略降（1.219 → 1.148 ℃）。这说明平均绝对偏差的小幅改善并不能保证大误差尾部同步改善。

功率回测结果为：向前协议下，非线性局地订正 nRMSE 为 0.1193，原始气象对照为 0.1148，总体未改善，反而增加 3.94%；均值偏差订正为 0.1329，同样没有改善。此外，气象订正后的总体功率也没有超过第三问的 0.1086（相对第三问高出 9.85%）。因此，本次局地订正未取得功率预测收益，也未验证空间尺度提升。

需要说明：GHI 的 MAE 变小而 RMSE 变大，与早期样本少、非线性外推及两阶段输入分布变化相容，但尚未证明唯一原因。统计映射可用于拟合给定样本中的条件偏差，但不产生未观测的云场空间结构；即使功率误差有所下降，也不能归因为恢复了次网格云影位置或识别了具体故障。

8.4 问题四小结

问题四在单点意义上实现了气象统计订正的有限实证，并给出了严格的边界说明：受数据限制，空间尺度提升未获验证。在向前协议下，单站非线性订正未带来功率回测改善。进一步检验需补充原始 NWP 尺度、站点提取关系及发行版本；若开展空间插值，还需同期邻近格点及其坐标。

九、模型检验与不确定性分析

本章对问题 2—4 的预测模型做统一检验与不确定性分析，图 5 给出三个模型在四个测试周的表现总览；以下各节分别给出配对日期块检验、提前量与口径敏感性、离线题目划分对照与数据泄漏核对。

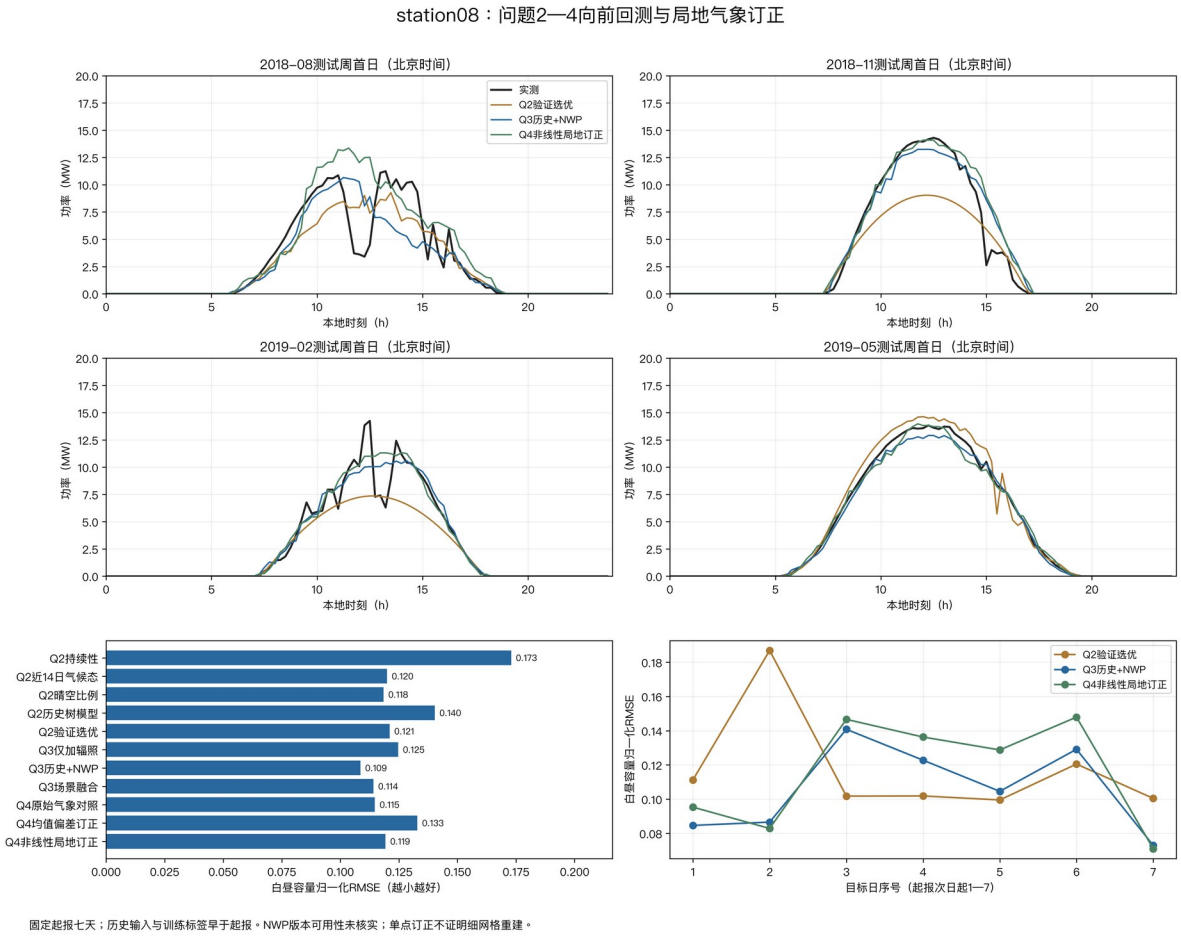


图 5 问题 2—4 前瞻回测总览（四个测试周首日曲线、方法指标对比与按目标日序号汇总的 nRMSE）

9.1 配对日期块检验

为判断改善是否稳定，采用四个测试周内按日期分层的重抽（共 2000 次），统计"第一模型 nRMSE 减第二模型 nRMSE"的 95% 分位区间。正值表示第二模型较好，区间跨 0 表示证据

不足。以 28 个日期作为重抽单位，避免把相邻 15 分钟点视为独立重复；周内相邻日期仍可能相关，区间仅条件于这四个固定测试周。

表 12 按日期分层重抽的误差差区间

第一 → 第二	日期数	nRMSE 差	区间下界	区间上界
Q2 验证选优 → Q3 历史+NWP	28	0.0124	−0.0157	0.0377
Q2 验证选优 → Q3 场景融合	28	0.0069	−0.0187	0.0310
Q4 原始气象对照 → Q4 非线性局地订正	28	−0.0045	−0.0238	0.0145
Q3 历史+NWP → Q4 非线性局地订正	28	−0.0107	−0.0181	−0.0030

NWP 相对第二问的 nRMSE 差为 0.0124，区间 [−0.0157, 0.0377] 跨 0。因此本文只报告固定测试点上的改善，不声称已在日期层面证明稳定显著收益。该区间条件于四个固定测试周，不能外推为全年显著性。

9.2 提前量与白昼口径敏感性

按目标日序号（提前量）汇总：

表 13 目标日序号与预测误差

目标日序号	白昼样本	Q2 验证选优	Q3 历史+NWP	Q4 非线性局地订正
1	191	0.1113	0.0847	0.0953
2	192	0.1869	0.0866	0.0829
3	192	0.1018	0.1408	0.1466
4	192	0.1019	0.1228	0.1363
5	193	0.0995	0.1046	0.1288
6	193	0.1205	0.1291	0.1479
7	193	0.1005	0.0730	0.0711

由于每个序号在四个测试周只有 4 个目标日，且 NWP 行未按提前量存档，故不能据此证

明误差必须随提前量单调增长；该表也是固定目标周的样本比较。

按太阳高度角阈值筛选白昼的敏感性：

表 14 白昼判定阈值敏感性

模型	阈值 0°（正式口径）	阈值 5°（敏感性）
Q2 验证选优	0.1210 (n=1346)	0.1255 (n=1250)
Q3 历史+NWP	0.1086 (n=1346)	0.1125 (n=1250)
Q4 非线性局地订正	0.1193 (n=1346)	0.1235 (n=1250)

0° 为正式评分口径，5° 仅作敏感性。筛除低太阳高度角样本后三者 nRMSE 均略升。样本组成发生变化，不能据此认定模型能力改变。

9.3 离线题目划分对照

作为补充，采用整段非测试日期训练的离线协议。树模型统一采用最后一周（2019 年 5 月）的验证选参结果，第二问方案与场景权重仍沿用各周前置验证选择。其训练范围与部分选参设置均不同于向前协议，结果如下：

表 15 离线协议补充结果

模型	白昼样本	nRMSE	归一 MAE	CR(%)	QR(%)
Q2 持续性	1346	0.1729	0.1182	82.71	83.95
Q2 近 14 日气候态	1346	0.1199	0.0849	88.01	94.43
Q2 晴空比例	1346	0.1184	0.0870	88.16	94.87
Q2 历史树模型	1346	0.1240	0.0906	87.60	93.16
Q2 验证选优	1346	0.1210	0.0845	87.90	94.28
Q3 历史+NWP	1346	0.1047	0.0729	89.53	95.77
Q3 仅加辐照	1346	0.1118	0.0792	88.82	95.17

模型	白昼样本	nRMSE	归一 MAE	CR(%)	QR(%)
Q3 场景融合	1346	0.1086	0.0755	89.14	95.39
Q4 原始气象对照	1346	0.1136	0.0819	88.64	96.36
Q4 均值偏差订正	1346	0.1178	0.0909	88.22	96.14
Q4 非线性局地订正	1346	0.1035	0.0714	89.65	96.66

离线协议下，Q4 原始气象对照为 0.1136、非线性订正为 0.1035（改善 8.91%），与向前协议结论相反。离线训练纳入了部分早期测试周之后的日期，训练规模、季节覆盖及部分选参设置均有变化；现有对照未单独分离各因素的影响。离线结果只能作为不同协议的补充，不能证明真实起报时已经可以获得这些数据。

9.4 样本与数据泄漏核对

表 16 向前训练与历史输入时序核对

测试月	起报时刻	最晚训练标签	唯一训练点	最晚历史输入	第二问验证选择
2018-08	2018-08-24 00:00	2018-08-23 19:00	2149	2018-08-23 23:45	climatology
2018-11	2018-11-23 00:00	2018-11-22 17:00	5828	2018-11-22 23:45	physical
2019-02	2019-02-21 00:00	2019-02-20 18:00	9120	2019-02-20 23:45	physical
2019-05	2019-05-24 00:00	2019-05-23 19:15	13462	2019-05-23 23:45	persistence

自动核对结果确认：测试点 2688 个、每周 672 点；实测无缺失；所有向前训练均早于起报；测试训练标签为 0；预测值有限且落在容量范围内；夜间预测均为 0。同时明确标记两项未证实：NWP 起报可用性未核实、多格点空间降尺度未核实。

十、模型的评价及推广

10.1 模型的优点

1. 物理与统计结合。以晴空参考与晴空比例为确定性骨架，再以梯度提升树学习残差，既保留可解释的日周期结构，又能利用历史状态。

2. 评价协议严格。坚持向前评价并区分离线协议，核对了每一步训练标签与历史输入的时

序，排除了测试标签泄漏。

3. 多模型对照充分。从持续性、气候态到残差树，从辐照消融到分场景融合，形成了完整的对照链条，结论不依赖单一模型。

4. 结论有边界。对 NWP 起报可用性与空间降尺度两项未证实要求如实设限，对改善给出了区间与场景限制，不夸大适用范围。

5. 结果可溯源。逐点预测、指标、配置、输入哈希与源代码集中保存，可一键复核。

10.2 模型的缺点

1. 观测时长不足完整一年，且主站缺 3 个整日，季节结论不能外推多年。

2. NWP 起报可用性未证实，第三、四问结论限于"可用性假设条件下的回测"。

3. 空间尺度提升未获验证，第四问只有单点统计订正。

4. 容量与采样口径为假设：20 MW 交流限制与开机容量口径未核实，功率为瞬时值或区间均值未确认，日积分只是估计。

5. 改善的统计稳定性有限：按日期分块的 95% 区间跨 0，不能用"显著"表述。

6. 低辐照场景样本少（52 点），分场景结论的置信度有限。

10.3 模型的推广

1. 按天气状态切换策略：场景融合相对第二问的点估计误差较低，但弱于全字段第三问模型且日期重抽区间跨零，可进一步引入随预报晴空指数连续变化的权重，或按季节自适应。

2. 与日周期匹配的历史特征：可尝试以太阳几何对齐的历史同刻序列、以及多尺度滑动统计，替代简单的固定窗口。

3. 容量与夜间约束的精细化：若能获得逐时开机容量与逆变器限功率记录，可提高归一化口径的准确性。

4. 局地校正与多源融合：在获得邻近格点 NWP 后，可比较原始格点、插值/降尺度及其功率预测，检验空间分辨率敏感性；这一方向在补齐网格坐标后即可开展。

十一、结论

本文围绕光伏电站发电功率日前预测，基于 station08 电站数据完成出力特性分析、功率预测对照和局地气象订正的有限验证，主要结论如下：

1. 出力特性（问题一）：数据质量总体良好（除 3 个缺日外时间网格完整）；日积分估计 5 月最高 99.23 MWh、2 月最低 49.56 MWh；实际功率较简化晴空参考平均偏低 5.34 MW；15 分钟波动中位数 0.578 MW、P95 为 2.798 MW，与辐照变化相关系数 0.843。

2. 仅用历史功率预测（问题二）：在四个测试周 1346 个白昼点上，验证选优模型向前 nRMSE 为 0.1210；简单基线（晴空比例 0.1184、气候态 0.1199）与复杂模型相当，说明增加复杂度并不必然提升精度。

3. 引入 NWP（问题三）：历史+NWP 模型向前 nRMSE 降至 0.1086，相对问题二下降 10.28%；但收益不均匀——中高辐照场景改善、低辐照场景退化，且分月看 2018 年 8 月变差。

4. 气象空间处理（问题四）：单点非线性局地订正的向前 nRMSE 为 0.1193，较原始气象对照 0.1148 反而增加 3.94%；受数据限制，空间尺度提升未获验证。

加入 NWP 的收益主要体现在中、高预报辐照档，且依赖目标时刻预报在起报时已经可用的假设。将这一结果用于调度前，仍需补齐发行档案、完整年度观测及逐日 24—48 小时回测；单站订正的负结果也不能推广为所有空间降尺度方法均无效。

参考文献

[1] Yao T, Wang J, Wu H, et al. A photovoltaic power output dataset: Multi-source photovoltaic power output dataset with Python toolkit. Solar Energy, 230: 122–130, 2021. DOI:10.1016/j.solener.2021.09.050.

[2] Yao T, Wang J, Wu H, et al. PVOD v1.0: A photovoltaic power output dataset. Science Data Bank, 2021. DOI:10.11922/sciencedb.01094. 数据引用按作者仓库记录；本地文件修订版本未单独核实。

[3] yaotc. PVODataset. https://github.com/yaotc/PVODataset，访问时间：2026-10-16。

[4] B 题 光伏电站发电功率日前预测问题；附件 1 误差分析方法. 赛题材料，见支撑材料 docs 目录。

[5] Holmgren W F, Hansen C W, Mikofski M A. pvlib python: a python package for modeling solar energy systems. Journal of Open Source Software, 3(29): 884, 2018. DOI:10.21105/joss.00884.

[6] pvlib developers. Location.get_clearsky. https://pvlib-python.readthedocs.io/en/stable/reference/generated/pvlib.location.Location.get_clearsky.html，访问时间：2026-10-16。

[7] scikit-learn developers. HistGradientBoostingRegressor. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html，访问时间：2026-10-16。

附录

附录 A 软件与复现命令

计算环境为 Python 3.13.15、numpy 2.5.4、pandas 3.0.6、scikit-learn 1.9.1、pvlib 0.16.1，具体依赖见 requirements-lock.txt。随机种子为 20261014。以下命令在项目根目录运行。--inspect 仅查看已有证据；完整重训会覆盖证据包、汇总图及自动生成的交接稿。论文排版采用 python-docx，PDF 由 LibreOffice 导出。

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-lock.txt
.venv/bin/python src/q1_check_data.py --station station08 --source data/PVOD
.venv/bin/python src/q1_clean_data.py --rules docs/q1_cleaning_rules_station08.json
.venv/bin/python src/q1_reference_power.py --assumptions
docs/q1_reference_assumptions_station08.json --source data/PVOD
.venv/bin/python src/q1_characteristics.py --station station08
.venv/bin/python src/build_q1_station08_report.py
OMP_NUM_THREADS=2 .venv/bin/python src/q234_forecast.py
.venv/bin/python src/q234_forecast.py --inspect
.venv/bin/python src/q234_forecast.py --export-table2 paper/表2.csv
.venv/bin/python -m unittest discover -s tests -v
```

附录 B 数据及结果清单

支撑材料保留主站 station08、对照站 station01 原始 CSV、完整 metadata.csv 及两站清洗表；同时保留两站的清洗规则、参考参数、第一问结果、第二至四问证据包和图件。station01 仅用于第一问共同完整日比较，不参与第二至四问训练。

逐点预测文件“表 2_七天固定起报逐点预测.csv”包含 2688 条唯一目标时间记录，均使用北京时间；功率单位为 MW。四周各 672 条，白昼评价 1346 条。每周固定一个起报时刻，提前量为 24—191.75 小时，不能以修改时间列的方式将其转成逐日滚动预测。证据包同时保存主实验与补充离线预测、指标、训练审计和输入文件 SHA-256。

附录 C 计算源程序

以下完整列出六个计算与报告脚本及四个测试文件，内容与支撑材料中的源文件一致。长行在排版中自动换行，不改变附件中的程序内容。

src/build_q1_station08_report.py

```
"""汇总 station08 第一问重跑，按共同完整日比较 station01，生成当前对比稿。"""
import hashlib
import json

import pandas as pd
from q1_check_data import ROOT, save_csv


def load(station, name):
    return json.loads((ROOT / 'results/q1' / station / name).read_text())


def markdown_table(frame):
    frame = frame.rename(columns={
        'year_month': '月份', 'observed_days': '观测日', 'complete_power_days': '完整功率日',
        'mean_daily_integral_estimate_mwh': '日积分均值(MWh)', 'reference_mean_daily_integral_
same_days_mwh': '同日参考均值(MWh)',
        'season': '季节', 'average_profile_peak_mw': '平均曲线峰值(MW)', 'average_profile_peak
_clock': '峰值时刻',
        'included_year_months': '包含月份', 'scenario': '情景', 'common_daylight_points': '白
昼样本数',
        'reference_peak_mw': '参考峰值(MW)', 'mean_actual_minus_reference_mw': '平均实际减参考
(MW)',
        'mae_to_reference_mw': '平均绝对偏差(MW)', 'station': '站点', 'common_complete_days':
'共同完整日',
        'mean_daily_integral_mwh': '日积分均值(MWh)', 'strong_irradiance_zero_low_points': '强
辐照零低功率点',
        'p95_abs_ramp_mw': '变化绝对值 P95(MW)', 'ramp_irradiance_pearson': '功率辐照变化相关系
数'})
    if '情景' in frame:
        frame['情景'] = frame['情景'].replace({'reference_ac_loss0_mw': '额外损耗 0%', 'referen
ce_ac_loss10_mw': '额外损耗 10%',
             'reference_ac_loss20_mw': '额外损耗 20%', 'reference_ac_altitude200_loss0_mw': '海
拔 200m、损耗 0%'})
    if '季节' in frame:
        frame['季节'] = frame['季节'].replace({'spring': '春', 'summer': '夏', 'autumn': '秋',
 'winter': '冬'})
    def format_value(x):
        return f'{x:.3f}' if isinstance(x, float) else str(x)
    return '\n'.join(['| ' + ' | '.join(frame.columns) + ' |',
                      '| ' + ' | '.join(['---'] * len(frame.columns)) + ' |'] +
                     ['| ' + ' | '.join(map(format_value, row)) + ' |' for row in frame.itertu
ples(index=False, name=None)])


def main():
    stations = ('station01', 'station08')
    folder = ROOT / 'results/q1/station08'
    # 昨日 ZIP 已不在项目中，按昨日清洗表保留的全部原始列核对 station01。
    original = pd.read_csv(ROOT / 'data/PVOD/station01.csv')
    previous = pd.read_csv(ROOT / 'data/processed/station01_q1_clean.csv')
    pd.testing.assert_frame_equal(original, previous[original.columns], check_exact=False, rto
l=1e-12, atol=1e-12)
    audits = {s: load(s, 'summary.json') for s in stations}
    cleans = {s: load(s, 'cleaning_summary.json') for s in stations}
    chars = {s: load(s, 'characteristics_summary.json') for s in stations}
    refs = {s: load(s, 'reference_summary.json') for s in stations}
    daily = {s: pd.read_csv(ROOT / 'results/q1' / s / 'daily_characteristics.csv') for s in st
ations}
    common_dates = sorted(set.intersection(*[set(daily[s].loc[daily[s].complete_power_day, 'da
te_local']) for s in stations]))
    save_csv(pd.DataFrame({'date_local': common_dates}), folder / 'comparison_common_complete_
dates.csv')
    quality, matched = [], []
    for station in stations:
        a, c = audits[station], cleans[station]
```

```
        quality.append({'站点': station, '原始记录': a['rows'], '缺失时间点': a['missing_times
tamps'],
                        '可疑持值段': c['candidate_frozen_runs'], '清洗缺失功率点': c['invalid
_power_points'],
                        '完整功率日': c['complete_power_days'], '强辐照零低功率点': c['day_zer
o_flags_retained'],
                        '清洗后夜间正功率点': c['remaining_night_positive_power_points']})
        data = pd.read_csv(ROOT / 'data/processed' / f'{station}_q1_clean.csv')
        path = ROOT / 'data/processed' / f'{station}_q1_clean.csv'
        assert hashlib.sha256(path.read_bytes()).hexdigest() == chars[station]['cleaned_data_s
ha256'] == refs[station]['cleaned_data_sha256']
        assert hashlib.sha256((ROOT / 'results/q1' / station / 'reference_power.csv').read_byt
es()).hexdigest() == chars[station]['reference_data_sha256']
        times = pd.to_datetime(data.time_utc, utc=True).dt.tz_convert('Asia/Shanghai')
        data['time_local_parsed'] = times
        data = data.loc[times.dt.strftime('%Y-%m-%d').isin(common_dates)].copy()
        selected_daily = daily[station].loc[daily[station].date_local.isin(common_dates)]
        assert len(data) == len(common_dates) * 96 and data.power_clean.notna().all()
        pairs = data.time_local_parsed.diff().eq(pd.Timedelta(minutes=15)) & data.is_daylight
& data.is_daylight.shift(fill_value=False)
        ramps = data.power_clean.diff().loc[pairs]
        iramps = data.lmd_totalirrad_clean.diff().loc[pairs]
        daylight = data.loc[data.is_daylight]
        zero = (daylight.lmd_totalirrad_clean > 200) & (daylight.power_clean <= 0.01)
        matched.append({'station': station, 'common_complete_days': len(common_dates), 'common
_records': len(data),
                        'mean_daily_integral_mwh': selected_daily.actual_daily_integral_estima
te_mwh.mean(),
                        'daylight_points': len(daylight), 'strong_irradiance_zero_low_points':
 int(zero.sum()),
                        'zero_low_fraction_of_daylight': zero.mean(), 'daylight_ramp_pairs': l
en(ramps),
                        'p95_abs_ramp_mw': ramps.abs().quantile(.95), 'max_abs_ramp_mw': ramps
.abs().max(),
                        'ramp_irradiance_pearson': ramps.corr(iramps)})
    save_csv(pd.DataFrame(quality), folder / 'station_quality_comparison.csv')
    save_csv(pd.DataFrame(matched), folder / 'common_days_comparison.csv')
    missing = pd.read_csv(folder / 'missing_timestamps.csv')
    missing_days = sorted(pd.to_datetime(missing.missing_time_local).dt.strftime('%Y-%m-%d').u
nique())
    a, c, ch, r = audits['station08'], cleans['station08'], chars['station08'], refs['station0
8']
    rules = ROOT / 'docs/q1_cleaning_rules_station08.json'
    assumptions = ROOT / 'docs/q1_reference_assumptions_station08.json'
    assert hashlib.sha256(rules.read_bytes()).hexdigest() == c['rules_sha256']
    assert hashlib.sha256(assumptions.read_bytes()).hexdigest() == r['assumptions_sha256']
    assert a['source_sha256'] == c['source_sha256'] == r['source_zip_sha256']
    monthly = pd.read_csv(folder / 'monthly_characteristics.csv')
    seasonal = pd.read_csv(folder / 'seasonal_characteristics.csv')
    sensitivity = pd.read_csv(folder / 'reference_sensitivity.csv')
    extreme = pd.read_csv(folder / 'largest_daylight_changes.csv').iloc[0]
    base = r['scenarios'][0]
    h, low = ch['monthly_highest'], ch['monthly_lowest']
    text = rf'''# 第一问重跑与站点对比：station08

整理日期：2026-10-13。沿用第一问的模型和统计口径；station01 仅保留对比复现必需的数据和结果，当
前正文以 station08 为主。

## 1 数据与清洗

从`data/PVOD/station08.csv`和`metadata.csv`读取，UTC 转换为北京时间。范围为{a['start_local']}至
{a['end_local']}，共{a['rows']:,}条 15 分钟记录。无重复时间戳、字段缺失或基础非法值。时间网格缺
失{a['missing_timestamps']}点，全部位于{', '.join(missing_days)}，对应 3 个整日；缺失日期没有补
零或插值。现有{c['complete_power_days']}个观测日均有完整 96 点功率，但不代表时间范围内没有缺日。
四个规定月末测试周各 672 点完整，清洗后功率也完整。

清洗沿用 station01 的阈值和规则，仅当功率与六项现场气象连续至少 4 点完全相同，且段内存在深夜正功率
、正辐照时，才将后续重复值置缺失。本次候选持值段{c['candidate_frozen_runs']}个，修改字段值{c['
modified_cells']}个。白昼强辐照下零/低功率{c['day_zero_flags_retained']}点、近地平线夜间正功率
```

{c['remaining_night_positive_power_points']}点继续保留并标记。规则未发现异常不等于所有物理口径已核实。

2 晴空参考模型

使用 station08 元数据：经度 113.89999°E、纬度 36.70761°N，朝南 33°，78042 块 265 Wp 组件，直流额定容量 20.68113 MWp。装机容量字段 20000 kW，仍暂设交流限制 20 MW；组件容量和交流限制分别定义。采用 Ineichen 晴空辐照、isotropic 板面转换、固定组件温度 25°C 和 PVWatts 逆变器（假设额定效率 96%）。海拔基准 0m，反射率 0.25。计算公式为：

$$P_{{dc,ref}}(t)=20.68113\,\frac{{G_{{poa}}(t)}}{{1000}}(1-L)\quad\mathrm{{MW}}.$$

逆变器 DC 输入限制为 20/0.96 MW，额外损耗与逆变器损耗分别计算。坐标和组件配置来自元数据，其余假设沿用昨日方案；未通过实测功率拟合。采用 pvlib {r['pvlib_version']}。夜间参考设为 0，实测值不改动。参考不是严格上限，也不是日前预测。

3 月度和日内特性

日积分估计为 96 个采样功率求和×0.25 小时；功率是瞬时或区间平均尚未确认，因此不宣称精确实测电量。实际和参考仅使用同一批完整日。缺日不进入分母。

{markdown_table(monthly[['year_month', 'observed_days', 'complete_power_days', 'mean_daily_integral_estimate_mwh', 'reference_mean_daily_integral_same_days_mwh']])}

![月度日积分估计](../figures/q1/station08_monthly_integral.png)

日积分估计均值最高为{h['year_month']}的{h['mean_daily_integral_estimate_mwh']:.2f} MWh，最低为{low['year_month']}的{low['mean_daily_integral_estimate_mwh']:.2f} MWh。2018 年 11 月缺失 3 日，2019 年 6 月仅到 13 日；不能将样本均值推广为完整月结果，也不能把月度差异唯一归因于季节。

{markdown_table(seasonal[['season', 'complete_power_days', 'average_profile_peak_mw', 'average_profile_peak_clock', 'included_year_months']])}

![季节平均日内曲线](../figures/q1/station08_seasonal_diurnal.png)

季节定义与昨日一致：3–5 月春、6–8 月夏、9–11 月秋、12–2 月冬。每组各时刻使用同一批完整日，阴影为不同日期 P25–P75 分布，不是置信区间。夏季跨 2018 年 7/8 月和 2019 年 6 月 1–13 日，不是单一完整夏季。平均峰值时刻是样本统计结果。

4 参考偏差与 15 分钟波动

共同有效白昼点{base['common_daylight_points']:,}个，实际减基准参考平均为{base['mean_actual_minus_reference_mw']:.3f} MW，平均绝对偏差{base['mae_to_reference_mw']:.3f} MW，实际超过参考{base['actual_exceeds_reference_points']}点。该 MAE 是物理参考偏差，不能写成预测误差。

![固定诊断日参考对照](../figures/q1/station08_reference_day.png)

只对连续 15 分钟且两端白昼、功率有效的点计算变化。有效点对{ch['daylight_power_pairs']:,}个，绝对变化中位数{ch['median_abs_15min_change_mw']:.3f} MW、P95 为{ch['p95_abs_15min_change_mw']:.3f} MW、最大{ch['max_abs_15min_change_mw']:.3f} MW。功率变化与现场辐照变化的 Pearson 相关系数为{ch['power_irradiance_change_correlation']:.3f}，不能据此认定唯一因果原因。

最大变化时刻{extreme['time_local']}，功率变化{extreme['power_change_15min_mw']:.3f} MW，同期现场总辐照变化{extreme['irradiance_change_15min_wm2']:.1f} W/m²。事件取自 station08 结果表，没有沿用 station01 事件。

![白昼 15 分钟波动](../figures/q1/station08_daylight_fluctuation.png)

5 参数敏感性

{markdown_table(sensitivity[['scenario', 'common_daylight_points', 'reference_peak_mw', 'mean_actual_minus_reference_mw', 'mae_to_reference_mw']])}

0%、10%、20%损耗和 200m 海拔均为人为设定的敏感性情景。损耗较大时更接近实测并不能证明真实损耗就是该值；尚缺实际组件温度、海拔、逆变器数量和运维信息。

6 station08 是否更适合

{markdown_table(pd.DataFrame(quality))}

两站各有 345 个完整功率日，但日期集合不同：station01 因清洗排除 3 个日期，station08 原始缺 3 个日期。因此进一步只比较两站共同完整的{len(common_dates)}天、各{len(common_dates)*96:,}条记录。站点坐标不同，白昼点数可以不同；共同日期控制日历覆盖，仍不能控制各站天气与设备差异。

{markdown_table(pd.DataFrame(matched)[['station', 'common_complete_days', 'mean_daily_integral_mwh', 'strong_irradiance_zero_low_points', 'p95_abs_ramp_mw', 'ramp_irradiance_pearson']])}

station08 在当前规则下没有 station01 的深夜多字段持值问题，强辐照零/低功率标记也更少，适合用于第一问的替代站点分析。但它有 3 天原始缺测，清洗后完整日数量没有增加，近地平线夜间正功率也没有更少。因此可称当前已检查的异常较少，不能称全面优于 station01，更不能据此认定后续预测准确率更高。后续预测需采用相同切分、基线和评估口径实测。

7 复现与证据

```bash
.venv/bin/python src/q1_check_data.py --station station08 --source data/PVOD
.venv/bin/python src/q1_clean_data.py --rules docs/q1_cleaning_rules_station08.json
.venv/bin/python src/q1_reference_power.py --assumptions docs/q1_reference_assumptions_station08.json --source data/PVOD
.venv/bin/python src/q1_characteristics.py --station station08
.venv/bin/python src/build_q1_station08_report.py
```

昨日 ZIP 已不在项目内，本次核对目录 station01 与昨日清洗表保留的全部原始列一致（浮点容差 1e-12）；两站结果文件也分别校验数据哈希。逐点结果、月度/季节表、敏感性表、日期对照及哈希清单位于`results/q1/station08/`。第一问仍受不足完整一年、功率 AC/DC 和采样区间口径未确认、参考设备参数简化、异常原因未确认等限制。模型公式和方法依据沿用`docs/第一问_参考功率方案.md`中的 pvlib 接口说明；该文件的 station01 设备参数不用于本次 station08 计算。
'''
    draft = ROOT / 'paper/第一问_station08 重跑与对比.md'
    draft.write_text(text, encoding='utf-8')
    inputs = [ROOT / 'data/PVOD/metadata.csv', ROOT / 'data/PVOD/station08.csv', rules, assumptions]
    for station in stations:
        for name in ('summary.json', 'cleaning_summary.json', 'reference_summary.json', 'characteristics_summary.json', 'daily_characteristics.csv'):
            inputs.append(ROOT / 'results/q1' / station / name)
        inputs.extend([ROOT / 'data/processed' / f'{station}_q1_clean.csv', ROOT / 'results/q1' / station / 'reference_power.csv'])
    inputs.extend(ROOT / 'src' / name for name in ('q1_check_data.py', 'q1_clean_data.py', 'q1_reference_power.py', 'q1_characteristics.py', 'build_q1_station08_report.py'))
    manifest = {'version': 'q1-station08-rerun-v1', 'date': '2026-10-13', 'station01_raw_columns_match_previous_processed': True, 'original_zip_available': False,
                'common_complete_days': len(common_dates), 'missing_station08_dates': missing_days,
                'inputs_sha256': {str(p.relative_to(ROOT)): hashlib.sha256(p.read_bytes()).hexdigest() for p in inputs},
                'draft_sha256': hashlib.sha256(draft.read_bytes()).hexdigest(), 'status': '练习审阅稿，设备及采样口径仍待核实'}
    (folder / 'rerun_manifest.json').write_text(json.dumps(manifest, ensure_ascii=False, indent=2)+'\n')
    print(json.dumps({'draft': str(draft), 'common_days': len(common_dates), 'comparison': matched}, ensure_ascii=False, indent=2))

if __name__ == '__main__':
    main()

src/q1_characteristics.py

```
"""第一问步骤 4：月度日积分估计、季节日内曲线、白昼偏差与连续 15 分钟变化。"""

import calendar
import argparse
import hashlib
import json

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import font_manager

from q1_check_data import ROOT, STEP, save_csv


SEASONS = {12: "winter", 1: "winter", 2: "winter", 3: "spring", 4: "spring", 5: "spring",
           6: "summer", 7: "summer", 8: "summer", 9: "autumn", 10: "autumn", 11: "autumn"}
SEASON_LABELS = {"spring": "春季", "summer": "夏季（非完整、跨年时段）", "autumn": "秋季", "wi
nter": "冬季"}


def complete_daily_integrals(data):
    """只对本地日历日的完整 96 点求积分近似；缺失绝不隐式当 0。"""
    records = []
    for day, group in data.groupby(data["time_local"].dt.date):
        start = pd.Timestamp(day, tz="Asia/Shanghai")
        expected = pd.date_range(start, periods=96, freq="15min")
        complete_grid = pd.DatetimeIndex(group["time_local"]).equals(expected)
        valid_points = int(group["power_clean"].notna().sum())
        complete = complete_grid and valid_points == 96
        records.append({
            "date_local": str(day), "year_month": str(day)[:7], "season": SEASONS[day.month],
            "observed_points": len(group), "valid_power_points": valid_points,
            "complete_grid": complete_grid, "complete_power_day": complete,
            "review_points": int(group["needs_review"].sum()),
            "actual_daily_integral_estimate_mwh": float(group["power_clean"].sum() * 0.25) if
complete else np.nan,
            "reference_daily_integral_same_day_mwh": float(group["reference_ac_loss0_mw"].sum(
) * 0.25) if complete else np.nan,
            "raw_daily_integral_estimate_mwh": float(group["power"].sum() * 0.25) if complete_
grid and group["power"].notna().all() else np.nan,
        })
    return pd.DataFrame(records)


def continuous_changes(data):
    """只有时间连续且两个样本均非缺失时计算变化，天气变化也单独检查有效性。"""
    contiguous = data["time_local"].diff().eq(STEP)
    power_pair = contiguous & data["power_clean"].notna() & data["power_clean"].shift().notna(
)
    weather_pair = contiguous & data["lmd_totalirrad_clean"].notna() & data["lmd_totalirrad_cl
ean"].shift().notna()
    daylight_pair = data["is_daylight"] & data["is_daylight"].shift(fill_value=False)
    output = data[["source_csv_row", "time_local", "power_clean", "lmd_totalirrad_clean"]].cop
y()
    output["power_change_15min_mw"] = data["power_clean"].diff().where(power_pair)
    output["irradiance_change_15min_wm2"] = data["lmd_totalirrad_clean"].diff().where(weather_
pair)
    output["valid_power_pair"] = power_pair
    output["both_points_daylight"] = daylight_pair
    output["valid_daylight_weather_power_pair"] = power_pair & weather_pair & daylight_pair
    return output


def aggregate_months(daily):
    rows = []
    for month, group in daily.groupby("year_month", sort=True):
```

```
        eligible = group.loc[group["complete_power_day"]]
        year, number = map(int, month.split("-"))
        energy = eligible["actual_daily_integral_estimate_mwh"]
        rows.append({"year_month": month, "calendar_days": calendar.monthrange(year, number)[1
],
                     "observed_days": len(group), "complete_power_days": len(eligible),
                     "incomplete_power_days": len(group) - len(eligible),
                     "mean_daily_integral_estimate_mwh": energy.mean(), "median_daily_integral
_estimate_mwh": energy.median(),
                     "p10_daily_integral_estimate_mwh": energy.quantile(0.1), "p90_daily_integ
ral_estimate_mwh": energy.quantile(0.9),
                     "reference_mean_daily_integral_same_days_mwh": eligible["reference_daily_
integral_same_day_mwh"].mean(),
                     "raw_mean_daily_integral_all_observed_days_mwh": group["raw_daily_integra
l_estimate_mwh"].mean(),
                     "partial_calendar_month": len(group) != calendar.monthrange(year, number)
[1]})
    return pd.DataFrame(rows)


def seasonal_profiles(data, daily):
    eligible_dates = set(daily.loc[daily["complete_power_day"], "date_local"])
    eligible = data.loc[data["time_local"].dt.strftime("%Y-%m-%d").isin(eligible_dates)].copy(
)
    eligible["season"] = eligible["time_local"].dt.month.map(SEASONS)
    eligible["quarter_hour"] = eligible["time_local"].dt.hour * 4 + eligible["time_local"].dt.
minute // 15
    profiles = eligible.groupby(["season", "quarter_hour"]).agg(
        samples=("power_clean", "count"), actual_mean_mw=("power_clean", "mean"),
        actual_p25_mw=("power_clean", lambda x: x.quantile(0.25)),
        actual_p75_mw=("power_clean", lambda x: x.quantile(0.75)),
        reference_mean_mw=("reference_ac_loss0_mw", "mean")).reset_index()
    profiles["clock_local"] = profiles["quarter_hour"].map(lambda q: f"{q // 4:02d}:{q % 4 * 1
5:02d}")
    season_rows = []
    for season, group in daily.groupby("season"):
        valid = group.loc[group["complete_power_day"]]
        profile = profiles.loc[profiles["season"].eq(season)]
        peak = profile.loc[profile["actual_mean_mw"].idxmax()] if len(profile) else None
        season_rows.append({"season": season, "included_year_months": ", ".join(group["year_mo
nth"].unique()),
                            "observed_days": len(group), "complete_power_days": len(valid),
                            "mean_daily_integral_estimate_mwh": valid["actual_daily_integral_e
stimate_mwh"].mean(),
                            "average_profile_peak_mw": peak["actual_mean_mw"] if peak is not N
one else np.nan,
                            "average_profile_peak_clock": peak["clock_local"] if peak is not N
one else None,
                            "season_coverage_note": "夏季合并2018年7/8月与2019年6月1–13日；不
是单一完整夏季" if season == "summer"
                            else "仅描述本数据观测时段；不据此推广多年季节规律"})
    return profiles, pd.DataFrame(season_rows)


def plot_results(monthly, profiles, ramps, season_summary, figures, station="station01"):
    fonts = {font.name for font in font_manager.fontManager.ttflist}
    chinese = next((name for name in ("PingFang HK", "Songti SC", "Heiti TC", "SimHei", "Noto
Sans CJK SC")
                    if name in fonts), None)
    plt.rcParams["font.family"] = chinese or "DejaVu Sans"
    plt.rcParams["axes.unicode_minus"] = False
    fig, ax = plt.subplots(figsize=(13, 5.5))
    x = np.arange(len(monthly))
    ax.bar(x, monthly["mean_daily_integral_estimate_mwh"], width=0.55, color="#2563a6",
           label="实际日积分估计均值" if chinese else "Mean actual daily integral estimate")
    ax.vlines(x, monthly["p10_daily_integral_estimate_mwh"], monthly["p90_daily_integral_estim
ate_mwh"],
              color="#153759", linewidth=1.2, label="不同日期P10–P90范围" if chinese else "Ac
ross-day P10–P90")
```

```
    ax.plot(x, monthly["reference_mean_daily_integral_same_days_mwh"], color="#555555", linest
yle="--", marker="s",
            markersize=4, label="同一批完整日的晴空参考" if chinese else "Clear-sky reference
on the same complete days")
    ax.set_xticks(x, [f"{r.year_month}\nn={r.complete_power_days}" for r in monthly.itertuples
()])
    ax.set_ylim(bottom=0)
    ax.set_ylabel("日积分估计（MWh）" if chinese else "Daily integral estimate (MWh)")
    ax.set_title(f"{station} 月度日积分估计（完整功率日）" if chinese else f"{station}: monthl
y daily integral estimates (complete days)")
    ax.legend(frameon=False, fontsize=9, ncol=3)
    ax.grid(axis="y", color="#e4e7eb")
    ax.spines[["top", "right"]].set_visible(False)
    fig.text(0.06, 0.015, "Σ 采样功率×0.25 小时；采样口径待确认，非精确实测电量。P10–P90 为不同
日期分布，不是置信区间；2019 年 6 月仅到 13 日。" if chinese else
             "Sum(samples) × 0.25 h; sampling convention unconfirmed. P10–P90 is spread, not c
onfidence; June ends on the 13th.", fontsize=9)
    fig.tight_layout(rect=(0, 0.065, 1, 1))
    fig.savefig(figures / f"{station}_monthly_integral.png", dpi=300)
    plt.close(fig)

    fig, axes = plt.subplots(2, 2, figsize=(12, 8), sharex=True, sharey=True)
    for ax, season in zip(axes.flat, ("spring", "summer", "autumn", "winter")):
        frame = profiles.loc[profiles["season"].eq(season)]
        q = frame["quarter_hour"].to_numpy() / 4
        ax.plot(q, frame["actual_mean_mw"], color="#2563a6", linewidth=1.7, label="实际平均" i
f chinese else "Actual mean")
        ax.fill_between(q, frame["actual_p25_mw"], frame["actual_p75_mw"], color="#2563a6", al
pha=0.15,
                        label="实际 P25–P75" if chinese else "Actual P25–P75")
        ax.plot(q, frame["reference_mean_mw"], color="#555555", linestyle="--", linewidth=1.3,
                label="晴空参考平均" if chinese else "Mean clear-sky reference")
        days = int(season_summary.loc[season_summary["season"].eq(season), "complete_power_day
s"].iloc[0])
        ax.set_title(f"{SEASON_LABELS[season] if chinese else season}: n={days}日" if chinese
else f"{season}: n={days} days")
        ax.set_xlim(0, 23.75)
        ax.set_ylim(0, 21)
        ax.set_xticks([0, 6, 12, 18, 23.75], ["00:00", "06:00", "12:00", "18:00", "23:45"])
        ax.grid(axis="y", color="#e4e7eb")
        ax.spines[["top", "right"]].set_visible(False)
    axes[0, 0].legend(frameon=False, fontsize=9)
    for ax in axes[:, 0]: ax.set_ylabel("功率（MW）" if chinese else "Power (MW)")
    fig.suptitle(f"{station} 各季节平均日内形状（完整日，北京时间）" if chinese else f"{statio
n}: seasonal diurnal profiles (complete days, Asia/Shanghai)")
    fig.text(0.06, 0.017, "同一季节各时刻使用同一批完整日；阴影为不同日期 P25–P75，不是置信区
间。夏季为跨年且不完整的观测时段。" if chinese else
             "Each season uses the same complete days at all times; shading is P25–P75, not co
nfidence. Summer is partial and spans years.", fontsize=9)
    fig.tight_layout(rect=(0, 0.065, 1, 0.96))
    fig.savefig(figures / f"{station}_seasonal_diurnal.png", dpi=300)
    plt.close(fig)

    daylight = ramps.loc[ramps["valid_power_pair"] & ramps["both_points_daylight"]]
    paired = ramps.loc[ramps["valid_daylight_weather_power_pair"]]
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    axes[0].hist(daylight["power_change_15min_mw"], bins=50, color="#2563a6", rwidth=0.98)
    axes[0].set_xlabel("15 分钟功率变化（MW）" if chinese else "15-minute power change (MW)")
    axes[0].set_ylabel("点数" if chinese else "Count")
    axes[0].set_title(f"白昼相邻变化分布（n={len(daylight)}）" if chinese else f"Daylight ramp
 distribution (n={len(daylight)})")
    axes[1].scatter(paired["irradiance_change_15min_wm2"], paired["power_change_15min_mw"],
                    color="#2563a6", s=6, alpha=0.15, rasterized=True)
    axes[1].set_xlabel("15 分钟现场总辐照变化（W/m²）" if chinese else "15-minute local irradia
nce change (W/m²)")
    axes[1].set_ylabel("15 分钟功率变化（MW）" if chinese else "15-minute power change (MW)")
    axes[1].set_title(f"共同有效白昼点（n={len(paired)}）" if chinese else f"Common valid dayl
ight pairs (n={len(paired)})")
    for ax in axes:
        ax.axvline(0, color="#555555", linewidth=0.8)
```

```
        ax.grid(axis="y", color="#e4e7eb")
        ax.spines[["top", "right"]].set_visible(False)
    fig.text(0.06, 0.018, "只计算连续 15 分钟且两端有效的点；两端太阳高度角均>0。散点呈现相关性
，不能确认单一原因。" if chinese else
             "Only contiguous 15-min pairs with valid endpoints above the horizon; association
 does not establish a unique cause.", fontsize=9)
    fig.tight_layout(rect=(0, 0.065, 1, 1))
    fig.savefig(figures / f"{station}_daylight_fluctuation.png", dpi=300)
    plt.close(fig)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--station", default="station01")
    args = parser.parse_args()
    station = args.station
    folder = ROOT / "results/q1" / station
    clean_path = ROOT / "data/processed" / f"{station}_q1_clean.csv"
    reference_path = folder / "reference_power.csv"
    provenance = json.loads((folder / "reference_summary.json").read_text())
    if hashlib.sha256(clean_path.read_bytes()).hexdigest() != provenance["cleaned_data_sha256"
]:
        raise ValueError("清洗数据已变化，先重跑 q1_reference_power.py，避免混用版本")
    cleaned = pd.read_csv(clean_path)
    reference = pd.read_csv(reference_path)
    cols = ["source_csv_row", "time_utc", "is_daylight", "reference_ac_loss0_mw", "deviation_a
ctual_minus_reference_mw"]
    data = cleaned.merge(reference[cols], on=["source_csv_row", "time_utc"], how="left", valid
ate="one_to_one", suffixes=("", "_reference"))
    if len(data) != len(reference) or data["reference_ac_loss0_mw"].isna().any():
        raise ValueError("清洗数据与参考曲线不完整对应")
    data["time_local"] = pd.to_datetime(data["time_utc"], utc=True).dt.tz_convert("Asia/Shangh
ai")
    data = data.sort_values("time_local").reset_index(drop=True)
    if data["time_local"].duplicated().any(): raise ValueError("重复时间戳不能静默汇总")
    if not data["is_daylight"].eq(data["is_daylight_reference"]).all(): raise ValueError("两步
骤白昼规则不一致")
    daily = complete_daily_integrals(data)
    monthly = aggregate_months(daily)
    profiles, seasonal = seasonal_profiles(data, daily)
    ramps = continuous_changes(data)
    daylight_ramps = ramps.loc[ramps["valid_power_pair"] & ramps["both_points_daylight"]]
    paired = ramps.loc[ramps["valid_daylight_weather_power_pair"]]
    abs_ramp = daylight_ramps["power_change_15min_mw"].abs()
    common = data.loc[data["is_daylight"] & data["power_clean"].notna()].copy()
    common["year_month"] = common["time_local"].dt.strftime("%Y-%m")
    deviation = common.groupby("year_month").agg(points=("power_clean", "count"),
        mean_actual_minus_reference_mw=("deviation_actual_minus_reference_mw", "mean"),
        mean_abs_deviation_mw=("deviation_actual_minus_reference_mw", lambda x: x.abs().mean()
),
        actual_above_reference_fraction=("deviation_actual_minus_reference_mw", lambda x: x.gt
(0).mean())).reset_index()
    extreme = daylight_ramps.assign(abs_change_mw=abs_ramp).sort_values("abs_change_mw", ascen
ding=False).head(10)
    zero_events = daylight_ramps.loc[(daylight_ramps["power_clean"] <= 0.01)
        & (daylight_ramps["lmd_totalirrad_clean"] > 200)].copy()
    zero_events["abs_change_mw"] = zero_events["power_change_15min_mw"].abs()
    zero_events = zero_events.sort_values("abs_change_mw", ascending=False).head(5)
    corr = paired["power_change_15min_mw"].corr(paired["irradiance_change_15min_wm2"])
    for frame, name in [(daily, "daily_characteristics.csv"), (monthly, "monthly_characteristi
cs.csv"),
                        (profiles, "seasonal_diurnal_profiles.csv"), (seasonal, "seasonal_char
acteristics.csv"),
                        (ramps, "continuous_15min_changes.csv"), (extreme, "largest_daylight_c
hanges.csv"),
                        (deviation, "monthly_daylight_deviation.csv"),
                        (zero_events, "zero_power_daylight_events.csv")]: save_csv(frame, fold
er / name)
    summary = {"station": station, "analysis_version": "q1-characteristics-v1", "rows": len(da
ta),
```

```
                "cleaned_data_sha256": hashlib.sha256(clean_path.read_bytes()).hexdigest(),
                "reference_data_sha256": hashlib.sha256(reference_path.read_bytes()).hexdigest(
),
                "reference_model_version": provenance["model_version"], "observed_days": len(da
ily),
                "complete_power_days": int(daily["complete_power_day"].sum()),
                "daylight_power_pairs": len(daylight_ramps), "daylight_weather_power_pairs": le
n(paired),
                "median_abs_15min_change_mw": float(abs_ramp.median()),
                "p95_abs_15min_change_mw": float(abs_ramp.quantile(0.95)),
                "max_abs_15min_change_mw": float(abs_ramp.max()),
                "power_irradiance_change_correlation": float(corr) if np.isfinite(corr) else No
ne,
                "energy_method": "每个完整本地日的96个采样功率求和×0.25h；矩形积分近似，不宣称
精确实测电量",
                "monthly_highest": monthly.loc[monthly["mean_daily_integral_estimate_mwh"].idxm
ax()].to_dict(),
                "monthly_lowest": monthly.loc[monthly["mean_daily_integral_estimate_mwh"].idxmi
n()].to_dict(),
                "largest_change": extreme.iloc[0].astype(str).to_dict(),
                "limits": ["功率采样与时间戳区间口径仍未确认，电量为积分估计",
                           "首尾月份覆盖及夏季跨年情况须结合逐日覆盖表复核",
                           "参考功率参数含明确假设；实际/参考差异不是预测误差或唯一故障证据",
                           "保留的可疑零功率、近地平线正功率和持值首点仍可能影响统计",
                           "月度与季节均为描述性比较，没有对天气、设备状态和采样日期作因果控制"
]}
    # pandas标量转为标准JSON类型，保持输入行与所有计算表可追溯。
    (folder / "characteristics_summary.json").write_text(json.dumps(summary, ensure_ascii=Fals
e, indent=2, default=str), encoding="utf-8")
    plot_results(monthly, profiles, ramps, seasonal, ROOT / "figures/q1", station)
    highest, lowest = summary["monthly_highest"], summary["monthly_lowest"]
    lines = ["# 第一问月度、日内与波动分析（v1）", "", "本文件由src/q1_characteristics.py生成
；观察与原因解释分开记录。", "",
             "## 计算口径", "", f"- {summary['observed_days']}个观测日中，月度与日内曲线使用{s
ummary['complete_power_days']}个完整96点功率日。",
             "- 日积分估计=Σ采样功率×0.25小时，单位MWh；功率采样口径未确认，这不是精确实测电
量。",
             "- 月度参考和实际使用同一批完整日。偏差与波动另使用共同有效白昼点，数量单独报告。
",
             "- 日内阴影是不同日期的P25–P75分布，月度范围是P10–P90，均不是置信区间。", "",
             "## 月度结果", "", "| 月份 | 观测天数 | 完整功率日 | 日积分估计均值（MWh） | 同日
参考均值（MWh） |", "|---|---:|---:|---:|---:|"]
    lines += [f"| {r.year_month} | {r.observed_days} | {r.complete_power_days} | {r.mean_daily
_integral_estimate_mwh:.2f} | {r.reference_mean_daily_integral_same_days_mwh:.2f} |" for r in
monthly.itertuples()]
    lines += ["", f"在本次有效日和积分近似下，最高月份为{highest['year_month']}，均值{highest[
'mean_daily_integral_estimate_mwh']:.2f} MWh，完整日{highest['complete_power_days']}天；",
              f"最低为{lowest['year_month']}，均值{lowest['mean_daily_integral_estimate_mwh']:
.2f} MWh，完整日{lowest['complete_power_days']}天。",
              "不能据此认定单一气象原因或长期季节规律。2019年6月不完整，不能把其均值直接推广到
整个6月。",
              "原始全观测日均值另存在monthly_characteristics.csv，供检查清洗敏感性；它与清洗均
值的日集合不同，不是同样本对照。", "",
              "## 平均日内形状", "", "| 季节分组 | 完整日 | 平均曲线峰值（MW） | 峰值时刻（北
京时间） | 包含月份 |", "|---|---:|---:|---|---|"]
    lines += [f"| {SEASON_LABELS[r.season]} | {r.complete_power_days} | {r.average_profile_pea
k_mw:.2f} | {r.average_profile_peak_clock} | {r.included_year_months} |" for r in seasonal.ite
rtuples()]
    lines += ["", "夏季含2018年7/8月和2019年6月1–13日，不是一个完整连续夏季。平均曲线的峰值时
刻是观测统计结果，不等于每天的太阳正午或最大发电时刻。", "",
              "## 连续15分钟变化", "", f"- 两端均在白昼且功率有效的连续点对：{len(daylight_ram
ps)}；同时有有效辐照变化的点对：{len(paired)}。",
              f"- 绝对功率变化中位数{summary['median_abs_15min_change_mw']:.3f} MW，P95为{summ
ary['p95_abs_15min_change_mw']:.3f} MW，最大{summary['max_abs_15min_change_mw']:.3f} MW。",
              f"- 最大变化位于{summary['largest_change']['time_local']}，带符号变化{float(summ
ary['largest_change']['power_change_15min_mw']):.3f} MW，见largest_daylight_changes.csv。",
              f"- 同期功率变化与现场总辐照变化的Pearson相关系数为{corr:.3f}；相关性不能确认单
一原因。",
              "- 有辐照但功率为零的真实记录仍在样本中；最大突降应结合原始记录、辐照和运维信息
解释，不直接认定云遮挡或故障。", "",
```

```
                "## 保留的零功率事件", "",
                "zero_power_daylight_events.csv 列出白昼总辐照>200 W/m²、功率≤0.01 MW 时绝对变化
最大的 5 个记录；没有根据事后解释删除。", "",
                "| 目标时刻（北京时间） | 15 分钟功率变化（MW） | 同期辐照变化（W/m²） |", "|---|
---:|---:|"]
    lines += [f"| {r.time_local} | {r.power_change_15min_mw:.3f} | {r.irradiance_change_15min_
wm2:.1f} |" for r in zero_events.itertuples()]
    lines += ["", "功率降到零而辐照仍为正或上升，说明仅用辐照变化无法解释所有事件；尚缺故障、
限电、测量状态等记录，原因不能确认。", "",
                "## 实际与参考偏差", "", "monthly_daylight_deviation.csv 按月保留白昼有效点数、平
均实际减参考、平均绝对差及超过参考的比例。",
                "这些量是对简化物理参考的描述，不是日前预测评测；缺测实际值不填补。", "",
                "## 图与结果位置", "", f"- figures/q1/{station}_monthly_integral.png：月度均值、
日间范围与同日参考。",
                f"- figures/q1/{station}_seasonal_diurnal.png：各季节平均日内形状。",
                f"- figures/q1/{station}_daylight_fluctuation.png：白昼变化分布与辐照变化散点。"
,
                f"- 计算表保存在 results/q1/{station}，摘要数字可逐项追溯。", "",
                "## 局限与下一步", "", *[f"- {item}" for item in summary["limits"]], "",
                "下一步由 A、B、C 结合结果写第一问解释草稿，并用损耗、清洗与积分口径敏感性讨论限制
；获得测量或设备说明后再修订。"]
    (folder / "特性分析摘要.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
    print(json.dumps({key: summary[key] for key in ["complete_power_days", "monthly_highest",
"monthly_lowest",
                      "daylight_power_pairs", "p95_abs_15min_change_mw", "power_irradiance_chan
ge_correlation"]}, ensure_ascii=False, indent=2, default=str))


if __name__ == "__main__":
    main()
```

src/q1_check_data.py

"""第一问第一个步骤：读取原始 ZIP，检查 station01 并保存诊断结果。

不填补、不删行、不裁剪功率；可疑值仅作标记。运行方法见 README.md。
"""

```
import argparse
import calendar
import hashlib
import importlib.metadata
import json
from pathlib import Path
import platform
import zipfile

import matplotlib

matplotlib.use("Agg")
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
from matplotlib import font_manager
import numpy as np
import pandas as pd
import pvlib


ROOT = Path(__file__).resolve().parents[1]
LOCAL_TZ = "Asia/Shanghai"
STEP = pd.Timedelta(minutes=15)
UNITS = {
    "power": "MW",
    "nwp_globalirrad": "W/m²", "nwp_directirrad": "W/m²",
    "nwp_temperature": "°C", "nwp_humidity": "%",
    "nwp_windspeed": "m/s", "nwp_winddirection": "degree", "nwp_pressure": "hPa",
    "lmd_totalirrad": "W/m²", "lmd_diffuseirrad": "W/m²",
    "lmd_temperature": "°C", "lmd_pressure": "hPa",
    "lmd_winddirection": "degree", "lmd_windspeed": "m/s",
}
FLAG_LABELS = {
    "duplicate_timestamp": "重复时间戳",
    "missing_or_invalid_numeric": "数值缺失、解析失败或无穷值",
    "power_negative": "负功率",
    "power_above_installed_capacity": "功率超过装机容量",
    "night_positive_power": "夜间功率超过筛查阈值",
    "deep_night_positive_power": "太阳高度角低于-6°时功率超过筛查阈值",
    "irradiance_negative": "负辐照",
    "humidity_outside_0_100": "NWP 湿度不在 0–100%",
    "windspeed_negative": "负风速",
    "day_low_power_with_irradiance": "白昼较强辐照下功率接近零",
}


def save_csv(frame, path):
    frame.to_csv(path, index=False, encoding="utf-8-sig")


def read_station(archive, station):
    archive = Path(archive)
    if archive.is_dir():
        metadata = pd.read_csv(archive / "metadata.csv")
        raw = pd.read_csv(archive / f"{station}.csv")
    else:
        with zipfile.ZipFile(archive) as bundle:
            with bundle.open("metadata.csv") as stream:
                metadata = pd.read_csv(stream)
            with bundle.open(f"{station}.csv") as stream:
                raw = pd.read_csv(stream)
    selected = metadata.loc[metadata["Station_ID"].eq(station)]
```

```
    if len(selected) != 1:
        raise ValueError(f"元数据中应恰好有一条{station}记录")
    missing = {"date_time", *UNITS} - set(raw.columns)
    if missing:
        raise ValueError(f"缺少必需字段：{sorted(missing)}")
    if raw.empty:
        raise ValueError("站点数据为空")
    return raw, selected.iloc[0]


def source_sha256(source, station):
    """ZIP 按原口径哈希；目录按元数据和站点 CSV 的命名及字节哈希。"""
    source = Path(source)
    if source.is_file():
        return hashlib.sha256(source.read_bytes()).hexdigest()
    digest = hashlib.sha256()
    for name in ("metadata.csv", f"{station}.csv"):
        digest.update(name.encode())
        digest.update(hashlib.sha256((source / name).read_bytes()).digest())
    return digest.hexdigest()


def check_fields(raw):
    rows = []
    numeric = pd.DataFrame(index=raw.index)
    for column in raw.columns:
        record = {"field": column, "raw_dtype": str(raw[column].dtype),
                  "missing_count": int(raw[column].isna().sum()),
                  "missing_fraction": float(raw[column].isna().mean()),
                  "unit": UNITS.get(column, "UTC timestamp")}
        if column != "date_time":
            values = pd.to_numeric(raw[column], errors="coerce")
            record["parse_failure_count"] = int((raw[column].notna() & values.isna()).sum())
            record["infinite_count"] = int(np.isinf(values).sum())
            finite = values.where(np.isfinite(values))
            record.update({"min": finite.min(), "median": finite.median(),
                           "p99": finite.quantile(0.99), "max": finite.max()})
            numeric[column] = finite
        rows.append(record)
    return pd.DataFrame(rows), numeric


def test_week_coverage(times):
    unique = pd.DatetimeIndex(times).unique().sort_values()
    rows = []
    for year in range(unique.min().year, unique.max().year + 1):
        for month in (2, 5, 8, 11):
            last = calendar.monthrange(year, month)[1]
            start = pd.Timestamp(year=year, month=month, day=last - 6, tz=LOCAL_TZ)
            stop = pd.Timestamp(year=year, month=month, day=last, tz=LOCAL_TZ) + pd.Timedelta(
days=1)
            if start > unique.max() or stop <= unique.min():
                continue
            expected = pd.date_range(start, stop, freq="15min", inclusive="left")
            observed = unique[(unique >= start) & (unique < stop)]
            rows.append({"start_local": start.isoformat(),
                         "end_local": (stop - STEP).isoformat(),
                         "expected_points": len(expected), "observed_unique_points": len(obser
ved),
                         "missing_points": len(expected.difference(observed)),
                         "complete_time_grid": expected.equals(observed)})
    return pd.DataFrame(rows, columns=["start_local", "end_local", "expected_points",
                                       "observed_unique_points", "missing_points", "complete_ti
me_grid"])


def make_plot(data, start, days, capacity, station, path):
    stop = start + pd.Timedelta(days=days)
    frame = data.loc[data["time_local"].ge(start) & data["time_local"].lt(stop)].copy()
    if frame.empty:
```

```
        raise ValueError("指定出图窗口没有数据，请调整--plot-start")
    # 重复时间戳不能在诊断图中被静默平均；时间缺口使用NaN断开折线。
    if frame["time_local"].duplicated().any():
        raise ValueError("出图窗口有重复时间戳，请先依据检查结果确定处理规则")
    index = pd.date_range(start, stop, freq="15min", inclusive="left")
    frame = frame.set_index("time_local").reindex(index)
    fonts = {font.name for font in font_manager.fontManager.ttflist}
    chinese = next((name for name in ("PingFang HK", "Songti SC", "Heiti TC", "SimHei", "Noto
Sans CJK SC")
                    if name in fonts), None)
    plt.rcParams["font.family"] = chinese or "DejaVu Sans"
    plt.rcParams["axes.unicode_minus"] = False
    fig, axes = plt.subplots(2, 1, figsize=(13, 6.5), sharex=True)
    if chinese:
        power_label, ghi_label = "实测功率（MW）", "辐照度（W/m²）"
        title = f"{station} 连续{days}天数据检查（北京时间）"
        labels = ["实测功率", "装机容量（口径待确认）", "现场总辐照", "NWP总辐照"]
        note = "原始值未清洗；NWP发布时间未确认，仅作数据检查；固定窗口不代表全年。"
    else:
        power_label, ghi_label = "Measured power (MW)", "Irradiance (W/m²)"
        title = f"{station}: {days}-day data check (Asia/Shanghai)"
        labels = ["Measured power", "Installed capacity (basis unconfirmed)", "Local irradianc
e", "NWP irradiance"]
        note = "Uncleaned values; NWP release time unconfirmed; fixed window, not representati
ve of the year."
    axes[0].plot(index, frame["power"], color="#2563a6", linewidth=1.1, label=labels[0])
    axes[0].axhline(capacity, color="#555555", linestyle="--", linewidth=1, label=labels[1])
    axes[0].set_ylim(min(0, frame["power"].min()), max(capacity, frame["power"].max()) * 1.08)
    axes[0].set_ylabel(power_label)
    axes[1].plot(index, frame["lmd_totalirrad"], color="#2563a6", linewidth=1.1, label=labels[
2])
    axes[1].plot(index, frame["nwp_globalirrad"], color="#be751a", linestyle="--", linewidth=1
, label=labels[3])
    axes[1].set_ylabel(ghi_label)
    axes[1].set_ylim(bottom=min(0, frame[["lmd_totalirrad", "nwp_globalirrad"]].min().min()))
    ticks = pd.date_range(start, stop - pd.Timedelta(days=1), freq="1D").union(pd.DatetimeInde
x([stop - STEP]))
    axes[1].set_xticks(ticks)
    axes[1].xaxis.set_major_formatter(mdates.DateFormatter("%m-%d\n%H:%M", tz=start.tzinfo))
    axes[1].set_xlim(start, stop - STEP)
    for ax in axes:
        ax.legend(loc="upper right", frameon=False, ncol=2)
        ax.grid(axis="y", color="#e4e7eb", linewidth=0.7)
        ax.spines[["top", "right"]].set_visible(False)
    fig.suptitle(title, fontsize=15)
    fig.text(0.08, 0.015, note, fontsize=9, color="#555555")
    fig.tight_layout(rect=(0, 0.055, 1, 0.96))
    fig.savefig(path, dpi=180)
    plt.close(fig)
    return {"start_local": start.isoformat(), "end_local": (stop - STEP).isoformat(),
            "observed_rows": int((data["time_local"].ge(start) & data["time_local"].lt(stop)).
sum()),
            "selection_reason": "固定诊断窗口；默认选第一个测试周，不声称具有全年代表性"}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--station", default="station01")
    parser.add_argument("--source", "--zip", dest="zip", type=Path, default=ROOT / "docs/PVODd
atasets_v1.0.zip")
    parser.add_argument("--plot-start", default="2018-08-25", help="北京时间日期")
    parser.add_argument("--plot-days", type=int, default=7)
    parser.add_argument("--power-tolerance", type=float, default=0.01, help="筛查阈值MW，不是
删除规则")
    args = parser.parse_args()
    if args.plot_days <= 0 or args.power_tolerance < 0:
        parser.error("出图天数应为正数，功率筛查阈值不得小于0")
    if not args.station.startswith("station") or not args.station[7:].isdigit():
        parser.error("站点编号应形如station01")
    raw, metadata = read_station(args.zip, args.station)
```

```
    profile, data = check_fields(raw)
    times = pd.to_datetime(raw["date_time"], utc=True, errors="coerce")
    if times.isna().any():
        raise ValueError(f"存在{times.isna().sum()}条不能解析的时间；未静默丢弃，请先检查原文
件")
    data["source_csv_row"] = np.arange(len(raw)) + 2
    data["time_utc"] = times
    data["time_local"] = times.dt.tz_convert(LOCAL_TZ)
    data = data.sort_values("time_local", kind="stable").reset_index(drop=True)
    capacity = float(metadata["Capacity"]) / 1000
    if not np.isfinite(capacity) or capacity <= 0:
        raise ValueError("装机容量必须为正有限数")
    unique = pd.DatetimeIndex(data["time_local"]).unique().sort_values()
    expected = pd.date_range(unique.min(), unique.max(), freq="15min")
    # 地理信息用于昼夜筛查。海拔缺失，暂设0m；采用几何太阳高度角，不计算参考功率。
    solar = pvlib.solarposition.get_solarposition(
        pd.DatetimeIndex(data["time_utc"]), latitude=float(metadata["Latitude"]),
        longitude=float(metadata["Longitude"]), altitude=0, method="nrel_numpy")
    data["solar_elevation_deg"] = solar["elevation"].to_numpy()
    data["is_daylight"] = data["solar_elevation_deg"] > 0
    flags = {
        "duplicate_timestamp": data["time_local"].duplicated(keep=False),
        "missing_or_invalid_numeric": data[list(UNITS)].isna().any(axis=1),
        "power_negative": data["power"] < 0,
        "power_above_installed_capacity": data["power"] > capacity,
        "night_positive_power": ~data["is_daylight"] & (data["power"] > args.power_tolerance),
        "deep_night_positive_power": (data["solar_elevation_deg"] < -6) & (data["power"] > arg
s.power_tolerance),
        "irradiance_negative": (data[["lmd_totalirrad", "lmd_diffuseirrad", "nwp_globalirrad",
 "nwp_directirrad"]] < 0).any(axis=1),
        "humidity_outside_0_100": (data["nwp_humidity"] < 0) | (data["nwp_humidity"] > 100),
        "windspeed_negative": (data[["nwp_windspeed", "lmd_windspeed"]] < 0).any(axis=1),
        "day_low_power_with_irradiance": data["is_daylight"] & (data["lmd_totalirrad"] > 200)
& (data["power"] <= args.power_tolerance),
    }
    flag_frame = pd.DataFrame(flags)
    flagged = pd.concat([data, flag_frame], axis=1).loc[flag_frame.any(axis=1)]
    weeks = test_week_coverage(data["time_local"])
    daily = data.groupby(data["time_local"].dt.date).agg(rows=("time_local", "size"), unique_t
imes=("time_local", "nunique"))
    daily.index.name = "date_local"
    gaps = pd.DataFrame({"missing_time_local": expected.difference(unique)})
    intervals = pd.Series(unique[1:] - unique[:-1]).value_counts().sort_index()
    interval_frame = pd.DataFrame({"interval_minutes": intervals.index.total_seconds() / 60, "
count": intervals.to_numpy()})
    folder = ROOT / "results/q1" / args.station
    figures = ROOT / "figures/q1"
    folder.mkdir(parents=True, exist_ok=True)
    figures.mkdir(parents=True, exist_ok=True)
    start = pd.Timestamp(args.plot_start)
    start = start.tz_localize(LOCAL_TZ) if start.tzinfo is None else start.tz_convert(LOCAL_TZ
)
    figure = figures / f"{args.station}_week.png"
    plot_info = make_plot(data, start, args.plot_days, capacity, args.station, figure)
    for frame, name in [(profile, "field_profile.csv"), (flagged, "flagged_rows.csv"),
                        (weeks, "test_week_coverage.csv"), (gaps, "missing_timestamps.csv"),
                        (interval_frame, "interval_counts.csv"), (daily.reset_index(), "daily_
coverage.csv"),
                        (data.head(8), "preview.csv")]:
        save_csv(frame, folder / name)
    summary = {
        "station": args.station, "source": str(args.zip.resolve()),
        "source_sha256": source_sha256(args.zip, args.station),
        "source_columns": list(raw.columns), "rows": len(data), "unique_timestamps": len(uniqu
e),
        "source_sorted": bool(times.is_monotonic_increasing),
        "duplicate_extra_rows": int(data["time_local"].duplicated().sum()),
        "off_grid_unique_timestamps": len(unique.difference(expected)),
        "missing_timestamps": len(gaps), "start_utc": times.min().isoformat(), "end_utc": time
s.max().isoformat(),
```

```
        "start_local": unique.min().isoformat(), "end_local": unique.max().isoformat(),
        "observed_days": len(daily), "complete_96_point_days": int(daily["unique_times"].eq(96
).sum()),
        "capacity_mw": capacity, "power_min_mw": float(data["power"].min()), "power_max_mw": f
loat(data["power"].max()),
        "daylight_rows": int(data["is_daylight"].sum()), "night_rows": int((~data["is_daylight
"]).sum()),
        "flag_counts": {name: int(mask.sum()) for name, mask in flags.items()},
        "flag_fractions": {name: float(mask.mean()) for name, mask in flags.items()},
        "flagged_unique_rows": len(flagged), "power_tolerance_mw": args.power_tolerance,
        "daylight_rule": "几何太阳高度角>0；pvlib nrel_numpy；海拔暂设 0m，仅作昼夜筛查",
        "plot": plot_info, "figure": str(figure.relative_to(ROOT)),
        "versions": {"python": platform.python_version(), **{name: importlib.metadata.version(
name)
                     for name in ("pandas", "numpy", "matplotlib", "pvlib")}},
        "limitations": ["未覆盖完整一年", "仅标记可疑值，未清洗或确认异常原因", "海拔、AC/DC 和
功率采样口径待确认",
                        "NWP 发布时间待确认，不能以同时间行对齐证明日前可用", "未计算理论参考功
率或预测指标"],
    }
    (folder / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), en
coding="utf-8")
    lines = [f"# 第一问数据检查：{args.station}", "",
             "本报告由 src/q1_check_data.py 生成。原始数据不修改；本次只检查和标记，不填补、不删
行。", "",
             f"- 输入：{args.zip}中的{args.station}.csv 及 metadata.csv。",
             f"- 时间：{summary['start_local']}至{summary['end_local']}，北京时间。",
             f"- {len(data)}条记录，{len(unique)}个唯一时间戳；重复多余行{summary['duplicate_e
xtra_rows']}条，缺失时间点{len(gaps)}个。",
             f"- 覆盖{len(daily)}日，其中{summary['complete_96_point_days']}日有 96 个唯一时间点
。",
             f"- 功率范围{summary['power_min_mw']:.6f}至{summary['power_max_mw']:.6f} MW；装机
容量{capacity:g} MW（AC/DC 口径待确认）。",
             "- 字段缺失、解析失败、无穷值和分位数见 field_profile.csv。", "",
             "## 筛查标记", "", "各标记可能重叠，不能直接相加；标记不等于已经确认的异常。", ""
,
             "| 标记 | 点数 | 占全部记录比例 |", "|---|---:|---:|"]
    lines += [f"| {FLAG_LABELS[name]}（{name}） | {count} | {count / len(data):.2%} |"
              for name, count in summary["flag_counts"].items()]
    lines += ["", "## 优先复查的日期", "",
              "下表列出两类主要标记的日期及点数。记录存在的置信度高，具体原因未确认；严重程度
应结合原始曲线判断。", "",
              "| 日期（北京时间） | 夜间正功率点数 | 白昼较强辐照下低功率点数 |", "|---|---:|-
--:|"]
    dates = data["time_local"].dt.date
    review_days = flag_frame[["night_positive_power", "day_low_power_with_irradiance"]].groupb
y(dates).sum()
    review_days = review_days.loc[review_days.sum(axis=1) > 0]
    lines += [f"| {day} | {int(row.iloc[0])} | {int(row.iloc[1])} |" for day, row in review_da
ys.iterrows()]
    deep = data.loc[flags["deep_night_positive_power"]]
    if not deep.empty:
        lines += ["", f"太阳高度角低于-6°的可疑点功率范围为{deep['power'].min():.6f}至{deep['p
ower'].max():.6f} MW。",
                  "高夜间功率可能影响月度统计和实际/参考偏差，应优先排查重复持值、采集异常或时
间口径；不依据曲线直接认定原因。"]
    lines += ["", "## 规则与解释边界", "",
              f"- 夜间：几何太阳高度角≤0；深夜筛查：高度角<-6°；正功率筛查阈值{args.power_tol
erance:g} MW。海拔暂设 0m。",
              "- 夜间非零可能涉及测量偏置、数据处理、时间口径等，现有数据不能唯一确定原因；不
要直接归零。",
              "- day_low_power_with_irradiance：白昼现场总辐照>200 W/m² 且功率≤筛查阈值；只定
位待查时段，不断言故障或限电。",
              "- power_above_installed_capacity：超过元数据装机容量，需核实容量口径后再决定是
否处理。",
              "- flagged_rows.csv 保留原始 CSV 行号（含表头，从 2 开始），便于追溯。",
              "- 四个测试月按本地日历月末 7 天检查，结果见 test_week_coverage.csv；跨年练习不满足
完整一年要求。", "",
              "## 连续一周图", "",
```

```
                f"窗口：{plot_info['start_local']}至{plot_info['end_local']}；{plot_info['select
ion_reason']}。",
                f"图：{summary['figure']}。上图功率，下图现场与NWP总辐照；NWP只作数据诊断，不作
预测有效性证明。", "",
                "## 下一步", "",
                "先核查flagged_rows.csv中夜间非零等可疑值，再与A确定清洗规则；补齐参考功率所需参
数与明确假设。",
                "每一步的处理原因和修改点应另存记录，不能覆盖原始数据。", "",
                "## 尚未确认", "", *[f"- {item}" for item in summary["limitations"]]]
    (folder / "检查摘要.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
    print(json.dumps({"station": args.station, "rows": len(data), "flags": summary["flag_count
s"],
                      "summary": str(folder / "检查摘要.md"), "figure": str(figure)}, ensure_a
scii=False, indent=2))


if __name__ == "__main__":
    main()
```

src/q1_clean_data.py

```
"""第一问步骤2：复查持值，保留原始列，另存清洗列和逐值处理日志。"""

import argparse
import hashlib
import json
from pathlib import Path

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
from matplotlib import font_manager
import numpy as np
import pandas as pd
import pvlib

from q1_check_data import ROOT, STEP, UNITS, source_sha256, read_station, save_csv, test_week_
coverage


def find_frozen_runs(data, rules):
    """只把连续、多字段持值且跨越深夜的段作为候选；首点与重复点分开。"""
    fields = rules["frozen_fields"]
    all_finite = np.isfinite(data[fields]).all(axis=1)
    same = (data[fields].eq(data[fields].shift()).all(axis=1)
            & all_finite & all_finite.shift(fill_value=False)
            & data["time_local"].diff().eq(STEP))
    run_ids = (~same).cumsum()
    segment = pd.Series(False, index=data.index)
    repeats = pd.Series(False, index=data.index)
    rows = []
    for run_id, group in data.groupby(run_ids):
        impossible_night = ((group["solar_elevation_deg"] < rules["deep_night_elevation_deg"])
                            & (group["power"] > rules["power_tolerance_mw"])
                            & (group["lmd_totalirrad"] > 0))
        if len(group) < rules["frozen_min_points"] or not impossible_night.any():
            continue
        segment.loc[group.index] = True
        repeated_indices = group.index[1:] if rules["keep_first_frozen_point"] else group.inde
x
        repeats.loc[repeated_indices] = True
        rows.append({
            "run_id": int(run_id), "start_local": group["time_local"].iloc[0].isoformat(),
            "end_local": group["time_local"].iloc[-1].isoformat(), "points": len(group),
            "masked_repeated_points": len(repeated_indices),
            "anchor_source_csv_row": int(group["source_csv_row"].iloc[0]),
            "last_source_csv_row": int(group["source_csv_row"].iloc[-1]),
            "constant_power_mw": float(group["power"].iloc[0]),
            "constant_local_ghi_wm2": float(group["lmd_totalirrad"].iloc[0]),
            "deep_night_points": int(impossible_night.sum()),
            "nwp_ghi_distinct_values": int(group["nwp_globalirrad"].nunique()),
        })
    columns = ["run_id", "start_local", "end_local", "points", "masked_repeated_points",
               "anchor_source_csv_row", "last_source_csv_row", "constant_power_mw",
               "constant_local_ghi_wm2", "deep_night_points", "nwp_ghi_distinct_values"]
    return segment, repeats, pd.DataFrame(rows, columns=columns)


def clean_local_values(data, rules):
    """原始列原样保留，仅创建七项现场量的清洗副本。"""
    result = data.copy()
    fields = rules["frozen_fields"]
    frozen, repeats, runs = find_frozen_runs(data, rules)
    changes = []
    for field in fields:
        numeric = pd.to_numeric(data[field], errors="coerce")
        invalid = ~np.isfinite(numeric)
        if field in ("power", "lmd_totalirrad", "lmd_diffuseirrad", "lmd_windspeed"):
            invalid |= numeric < 0
```

```
        elif field == "lmd_pressure":
            invalid |= numeric <= 0
        elif field == "lmd_winddirection":
            invalid |= (numeric < 0) | (numeric > 360)
        result[field + "_clean"] = numeric.mask(invalid | repeats)
        # 已经是缺失的值无需记录“修改”；非有限值和非法值保留原值到日志。
        affected = (invalid | repeats) & data[field].notna()
        event = data.loc[affected, ["source_csv_row", "time_local", field]].copy()
        event = event.rename(columns={field: "raw_value"})
        event["field"] = field
        event["clean_value"] = np.nan
        event["reason"] = np.where(repeats.loc[affected], "R1_multifield_frozen_repeat", "R0_i
nvalid_local_value")
        changes.append(event)
    result["flag_frozen_run"] = frozen
    result["flag_frozen_repeat"] = repeats
    result["flag_frozen_anchor_review"] = frozen & ~repeats
    result["flag_night_positive_raw"] = (~data["is_daylight"]
        & (data["power"] > rules["power_tolerance_mw"]))
    result["flag_day_low_power_raw"] = (data["is_daylight"]
        & (data["lmd_totalirrad"] > rules["strong_irradiance_threshold_wm2"])
        & (data["power"] <= rules["power_tolerance_mw"]))
    result["flag_above_capacity_raw"] = data["power"] > data["capacity_mw"]
    result["needs_review"] = (frozen | result["flag_night_positive_raw"]
        | result["flag_day_low_power_raw"] | result["flag_above_capacity_raw"])
    result["valid_power_for_q1"] = result["power_clean"].notna()
    local_clean = [field + "_clean" for field in fields if field != "power"]
    result["valid_local_weather_for_q1"] = result[local_clean].notna().all(axis=1)
    log = pd.concat(changes, ignore_index=True)
    log = log[["source_csv_row", "time_local", "field", "raw_value", "clean_value", "reason"]]
    return result, log, runs


def plot_review(data, runs, path):
    if runs.empty:
        return False
    # 选最长的两段，不额外挑选效果最好的图；段数不足时按实际数量绘图。
    selected = runs.sort_values("points", ascending=False).head(2)
    fonts = {font.name for font in font_manager.fontManager.ttflist}
    chinese = next((name for name in ("PingFang HK", "Songti SC", "Heiti TC", "SimHei", "Noto
Sans CJK SC")
                    if name in fonts), None)
    plt.rcParams["font.family"] = chinese or "DejaVu Sans"
    plt.rcParams["axes.unicode_minus"] = False
    fig, axes = plt.subplots(2, len(selected), figsize=(13, 6.5), squeeze=False)
    for column, row in enumerate(selected.itertuples()):
        start = pd.Timestamp(row.start_local).normalize()
        stop = pd.Timestamp(row.end_local).normalize() + pd.Timedelta(days=1)
        frame = data.loc[data["time_local"].ge(start) & data["time_local"].lt(stop)]
        axes[0, column].plot(frame["time_local"], frame["power"], color="#be751a", linestyle="
--",
                             linewidth=1.2, label="原始功率" if chinese else "Raw power")
        axes[0, column].plot(frame["time_local"], frame["power_clean"], color="#2563a6", linew
idth=1.3,
                             label="清洗列（缺失处断线）" if chinese else "Clean column (gaps
preserved)")
        axes[1, column].plot(frame["time_local"], frame["lmd_totalirrad"], color="#be751a", li
nestyle="--",
                             linewidth=1.2, label="原始现场辐照" if chinese else "Raw local ir
radiance")
        axes[1, column].plot(frame["time_local"], frame["lmd_totalirrad_clean"], color="#2563a
6", linewidth=1.3,
                             label="清洗列（缺失处断线）" if chinese else "Clean column (gaps
preserved)")
        for i in range(2):
            ax = axes[i, column]
            ax.axvspan(pd.Timestamp(row.start_local), pd.Timestamp(row.end_local), color="#808
080", alpha=0.12)
            ticks = pd.date_range(start, stop, freq="12h", inclusive="left").union(pd.Datetime
Index([stop - STEP]))
```

```
            ax.set_xticks(ticks)
            ax.xaxis.set_major_formatter(mdates.DateFormatter("%m-%d\n%H:%M", tz=start.tzinfo)
)
            ax.set_xlim(start, stop - STEP)
            ax.grid(axis="y", color="#e4e7eb", linewidth=0.7)
            ax.spines[["top", "right"]].set_visible(False)
            ax.legend(loc="upper right", frameon=False, fontsize=9)
        axes[0, column].set_ylim(0, float(data["capacity_mw"].iloc[0]) * 1.1)
        axes[1, column].set_ylim(0, max(1000, float(data["lmd_totalirrad"].max()) * 1.05))
        axes[0, column].set_title(f"{start:%Y-%m-%d} – {(stop - STEP):%m-%d}")
    axes[0, 0].set_ylabel("功率（MW）" if chinese else "Power (MW)")
    axes[1, 0].set_ylabel("辐照度（W/m²）" if chinese else "Irradiance (W/m²)")
    fig.suptitle(f"{path.stem.removesuffix('_frozen_review')} 多字段持值复查与第一版清洗（北京
时间）" if chinese else
                 f"{path.stem.removesuffix('_frozen_review')}: frozen local data review and fi
rst cleaning (Asia/Shanghai)", fontsize=15)
    fig.text(0.06, 0.018, "灰色为整段持值候选；首点保留待复查，后续重复值设为缺失，没有插值或
补零。" if chinese else
             "Grey marks candidate frozen runs; first sample retained for review; repeated val
ues masked, not filled.", fontsize=9)
    fig.tight_layout(rect=(0, 0.065, 1, 0.95))
    fig.savefig(path, dpi=300)
    plt.close(fig)
    return True


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--rules", type=Path, default=ROOT / "docs/q1_cleaning_rules.json")
    args = parser.parse_args()
    rules = json.loads(args.rules.read_text(encoding="utf-8"))
    if rules["frozen_min_points"] < 2 or rules["power_tolerance_mw"] < 0:
        raise ValueError("持值最少点数应>=2，功率筛查阈值应非负")
    archive = ROOT / rules.get("source", rules.get("source_zip"))
    raw, metadata = read_station(archive, rules["station"])
    data = raw.copy()
    data["source_csv_row"] = np.arange(len(data)) + 2
    data["time_utc"] = pd.to_datetime(data["date_time"], utc=True, errors="raise")
    if data["time_utc"].isna().any() or data["time_utc"].duplicated().any():
        raise ValueError("有缺失或重复时间戳，请单独确定处理方式；本脚本不静默删行或平均")
    data["time_local"] = data["time_utc"].dt.tz_convert(rules["timezone"])
    data = data.sort_values("time_local", kind="stable").reset_index(drop=True)
    solar = pvlib.solarposition.get_solarposition(pd.DatetimeIndex(data["time_utc"]),
        latitude=float(metadata["Latitude"]), longitude=float(metadata["Longitude"]),
        altitude=rules["solar_altitude_assumption_m"], method="nrel_numpy")
    data["solar_elevation_deg"] = solar["elevation"].to_numpy()
    data["is_daylight"] = data["solar_elevation_deg"] > 0
    data["capacity_mw"] = float(metadata["Capacity"]) / 1000
    cleaned, log, runs = clean_local_values(data, rules)
    # 不把物理合理性筛查当成原始列修改，也不删除时间点。
    assert len(cleaned) == len(data)
    pd.testing.assert_frame_equal(cleaned[raw.columns], data[raw.columns])
    pd.testing.assert_frame_equal(cleaned.filter(regex="^nwp_"), data.filter(regex="^nwp_"))
    daily = cleaned.groupby(cleaned["time_local"].dt.date).agg(
        rows=("time_local", "size"), valid_power_points=("valid_power_for_q1", "sum"),
        daylight_points=("is_daylight", "sum"), review_points=("needs_review", "sum"))
    daily.index.name = "date_local"
    daily["complete_power_day"] = daily["rows"].eq(96) & daily["valid_power_points"].eq(96)
    daily["invalid_power_points"] = daily["rows"] - daily["valid_power_points"]
    weeks = test_week_coverage(cleaned["time_local"])
    invalid, reviews = [], []
    for row in weeks.itertuples():
        mask = cleaned["time_local"].between(pd.Timestamp(row.start_local), pd.Timestamp(row.e
nd_local))
        invalid.append(int((~cleaned.loc[mask, "valid_power_for_q1"]).sum()))
        reviews.append(int(cleaned.loc[mask, "needs_review"].sum()))
    weeks["invalid_power_points_after_cleaning"] = invalid
    weeks["review_points"] = reviews
    output = ROOT / "data/processed" / f"{rules['station']}_q1_clean.csv"
    folder = ROOT / "results/q1" / rules["station"]
```

```
    figure = ROOT / "figures/q1" / f"{rules['station']}_frozen_review.png"
    for path in (output.parent, folder, figure.parent):
        path.mkdir(parents=True, exist_ok=True)
    save_csv(cleaned, output)
    save_csv(log, folder / "cleaning_log.csv")
    save_csv(runs, folder / "frozen_runs.csv")
    save_csv(daily.reset_index(), folder / "cleaned_daily_coverage.csv")
    save_csv(weeks, folder / "cleaned_test_week_coverage.csv")
    made_plot = plot_review(cleaned, runs, figure)
    summary = {
        "rule_version": rules["version"], "source_sha256": source_sha256(archive, rules["stati
on"]),
        "rules_sha256": hashlib.sha256(args.rules.read_bytes()).hexdigest(), "rows": len(clean
ed),
        "source_columns_unchanged": True, "nwp_columns_unchanged": True,
        "candidate_frozen_runs": len(runs), "candidate_frozen_points": int(cleaned["flag_froze
n_run"].sum()),
        "masked_frozen_repeat_points": int(cleaned["flag_frozen_repeat"].sum()),
        "anchor_points_retained_for_review": int(cleaned["flag_frozen_anchor_review"].sum()),
        "modified_cells": len(log), "invalid_power_points": int((~cleaned["valid_power_for_q1"
]).sum()),
        "valid_power_points": int(cleaned["valid_power_for_q1"].sum()),
        "complete_power_days": int(daily["complete_power_day"].sum()),
        "incomplete_power_days": int((~daily["complete_power_day"]).sum()),
        "day_zero_flags_retained": int(cleaned["flag_day_low_power_raw"].sum()),
        "remaining_night_positive_power_points": int((~cleaned["is_daylight"]
            & (cleaned["power_clean"] > rules["power_tolerance_mw"])).sum()),
        "remaining_deep_night_positive_power_points": int(((cleaned["solar_elevation_deg"] < r
ules["deep_night_elevation_deg"])
            & (cleaned["power_clean"] > rules["power_tolerance_mw"])).sum()),
        "processed_file": str(output.relative_to(ROOT)), "figure": str(figure.relative_to(ROOT
)) if made_plot else None,
        "limits": rules["limitations"],
    }
    (folder / "cleaning_summary.json").write_text(json.dumps(summary, ensure_ascii=False, inde
nt=2), encoding="utf-8")
    lines = [f"# 第一问第一版清洗摘要：{rules['station']}", "", f"规则版本：{rules['version']}
；规则文件：{args.rules}。", "",
             "原始数据和原始列保持不变；全部时间点保留，七项现场量的清洗结果另存为*_clean 列。"
, "",
             "## 已处理的持值段", "", "| 开始（北京时间） | 结束 | 全段点数 | 后续重复点设缺失
 | 保留首点 |", "|---|---|---:|---:|---:|"]
    lines += [f"| {r.start_local} | {r.end_local} | {r.points} | {r.masked_repeated_points} |
{r.points - r.masked_repeated_points} |"
              for r in runs.itertuples()]
    lines += ["", "候选段要求功率和六个现场气象量完全相同，且包含太阳高度角<-6°的正功率和正辐
照点；无候选段时不进行持值清洗。",
              "满足规则只能标为可疑持值，不能确认采集故障、通信问题或既有填补。存在候选段时首
点保留并标注边界待复查。", "",
              "## 处理结果", "", f"- {summary['rows']}条记录仍全部保留；{summary['invalid_powe
r_points']}个功率清洗值为缺失，未插值或补零。",
              f"- 共{summary['modified_cells']}个现场字段值被设为缺失，逐值原值、行号、时间和
原因见 cleaning_log.csv。",
              f"- {summary['complete_power_days']}日保留完整 96 点功率；{summary['incomplete_pow
er_days']}日功率不完整，见 cleaned_daily_coverage.csv。",
              f"- {summary['day_zero_flags_retained']}个白昼较强辐照下零/低功率点保留，原因未
确认；不能据此断言故障或限电。",
              f"- 清洗后仍有{summary['remaining_night_positive_power_points']}个夜间正功率点，
但深夜正功率点为{summary['remaining_deep_night_positive_power_points']}个；近地平线样本继续保
留待查。",
              "- 四个测试周的时间覆盖和功率清洗缺失数见 cleaned_test_week_coverage.csv；不更换
评分真值。", "",
              "## 使用清洗表", "", f"文件：{summary['processed_file']}。", "",
              "- 原始列 power 仍是原始实测 MW；第一问统计使用 power_clean 并检查 valid_power_for_q1
。",
              "- 现场气象使用对应*_clean 列；valid_local_weather_for_q1 表示六项现场量均通过当前
基础规则。",
              "- needs_review 为复查提示，与 valid_power_for_q1 不是互补关系；保留的可疑零功率和
首点仍需在分析中说明。",
```

```
                "- 不完整日的日发电量不能按缺失补零。月度比较应使用完整有效日并报告天数，也应保
留未清洗结果作敏感性对照。",
                "- 只有相邻两个功率清洗值有效且时间相差 15 分钟时，才计算 15 分钟功率变化。",
                "- 采样口径未确认前，不能把功率求和×0.25 小时无条件宣称为精确日发电量。", "",
                "## 图与复现", "", f"图：{summary['figure']}。", f"运行：.venv/bin/python src/q1
_clean_data.py --rules {args.rules}。", "",
                "## 假设与边界", "", *[f"- {item}" for item in summary["limits"]], "",
                "## 下一步", "", "明确第一问参考功率的物理口径与参数假设，再实现晴空参考曲线；所
有结果必须记录清洗规则版本。"]
    (folder / "清洗摘要.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
    print(json.dumps(summary, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

src/q1_reference_power.py

```
"""第一问步骤 3：计算明确假设下的晴空参考功率，保留 DC 和 AC 口径。"""

import argparse
import hashlib
import json
from pathlib import Path
import re

import numpy as np
import pandas as pd
import pvlib
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
from matplotlib import font_manager

from q1_check_data import ROOT, STEP, source_sha256, read_station, save_csv


def reference_from_poa(poa, dc_capacity_w, ac_limit_w, efficiency, extra_loss=0):
    """固定组件温度 25°C，直流输出线性随板面辐照变化；逆变器独立定义 DC 输入限制。"""
    if dc_capacity_w <= 0 or ac_limit_w <= 0 or not 0 < efficiency <= 1 or not 0 <= extra_loss
 < 1:
        raise ValueError("容量、额定效率及额外损耗参数不合法")
    pdc_w = np.maximum(np.asarray(poa, dtype=float), 0) / 1000 * dc_capacity_w * (1 - extra_lo
ss)
    # inverter.pdc0 是逆变器 DC 输入限制，不能直接拿组件额定 DC 容量代入。
    pac_w = pvlib.inverter.pvwatts(pdc_w, pdc0=ac_limit_w / efficiency, eta_inv_nom=efficiency
)
    return pdc_w / 1e6, np.asarray(pac_w) / 1e6


def clear_sky_poa(times, metadata, assumptions, altitude):
    location = pvlib.location.Location(float(metadata["Latitude"]), float(metadata["Longitude"
]),
                                       tz="Asia/Shanghai", altitude=altitude)
    solar = location.get_solarposition(times)
    radiation = location.get_clearsky(times, model=assumptions["clear_sky_model"], solar_posit
ion=solar)
    poa = pvlib.irradiance.get_total_irradiance(
        assumptions["tilt_deg"], assumptions["azimuth_deg"], solar["apparent_zenith"], solar["
azimuth"],
        radiation["dni"], radiation["ghi"], radiation["dhi"], albedo=assumptions["albedo_assum
ed"],
        model=assumptions["transposition_model"])
    # 所有步骤统一几何太阳高度角>0 为白昼；夜间参考置 0，不改实测功率。
    daylight = solar["elevation"] > 0
    return solar, radiation, poa["poa_global"].where(daylight, 0).clip(lower=0)


def plot_day(table, assumptions, path):
    fonts = {font.name for font in font_manager.fontManager.ttflist}
    chinese = next((name for name in ("PingFang HK", "Songti SC", "Heiti TC", "SimHei", "Noto
Sans CJK SC")
                    if name in fonts), None)
    plt.rcParams["font.family"] = chinese or "DejaVu Sans"
    plt.rcParams["axes.unicode_minus"] = False
    start = pd.Timestamp(assumptions["plot_day_local"], tz="Asia/Shanghai")
    end = start + pd.Timedelta(days=1) - STEP
    frame = table.loc[table["time_local"].between(start, end)]
    if len(frame) != 96:
        raise ValueError("固定诊断日应有 96 条时间记录")
    fig, axes = plt.subplots(2, 1, figsize=(11, 6), sharex=True)
    axes[0].plot(frame["time_local"], frame["power_clean"], color="#2563a6", linewidth=1.8,
                 label="实际功率（清洗列）" if chinese else "Measured power (clean column)")
    for loss, color, style in [(0, "#555555", "--"), (10, "#be751a", ":"), (20, "#6d7947", "-.
")]:
```

```
        axes[0].plot(frame["time_local"], frame[f"reference_ac_loss{loss}_mw"], color=color, l
inestyle=style,
                     linewidth=1.3, label=f"晴空参考，额外损耗{loss}%" if chinese else f"Clear
-sky reference, extra loss {loss}%")
    axes[0].set_ylim(0, max(assumptions["ac_limit_mw_assumed"], frame["power_clean"].max()) *
1.08)
    axes[0].set_ylabel("功率（MW）" if chinese else "Power (MW)")
    axes[0].legend(frameon=False, ncol=2, fontsize=9)
    axes[1].plot(frame["time_local"], frame["deviation_actual_minus_reference_mw"], color="#25
63a6", linewidth=1.5)
    axes[1].axhline(0, color="#555555", linewidth=1)
    axes[1].set_ylabel("实际− 0%损耗参考（MW）" if chinese else "Actual minus 0%-loss referenc
e (MW)")
    ticks = pd.date_range(start, end, freq="4h").union(pd.DatetimeIndex([end]))
    axes[1].set_xticks(ticks)
    axes[1].xaxis.set_major_formatter(mdates.DateFormatter("%H:%M", tz=start.tzinfo))
    axes[1].set_xlim(start, end)
    for ax in axes:
        ax.grid(axis="y", color="#e4e7eb", linewidth=0.7)
        ax.spines[["top", "right"]].set_visible(False)
    fig.suptitle(f"{assumptions['station']} {start:%Y-%m-%d}：实际与假设工况晴空参考" if chine
se else
                 f"{assumptions['station']} {start:%Y-%m-%d}: actual vs assumed clear-sky refe
rence", fontsize=14)
    fig.text(0.08, 0.018, "北京时间；固定组件温度 25°C、假设交流限制 20MW；固定诊断日，不代表全
年；参考不是严格上限。" if chinese else
             "Asia/Shanghai; fixed cell temperature 25°C; assumed AC limit 20 MW; diagnostic d
ay, not a strict upper bound.", fontsize=9)
    fig.tight_layout(rect=(0, 0.055, 1, 0.95))
    fig.savefig(path, dpi=300)
    plt.close(fig)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--assumptions", type=Path, default=ROOT / "docs/q1_reference_assumpti
ons.json")
    parser.add_argument("--source", type=Path, default=ROOT / "docs/PVODdatasets_v1.0.zip")
    args = parser.parse_args()
    config_path = args.assumptions
    assumptions = json.loads(config_path.read_text())
    archive = args.source
    _, metadata = read_station(archive, assumptions["station"])
    match = re.search(r"Pmax:\s*([\d.]+)\s*Wp", metadata["Module"])
    if not match:
        raise ValueError("无法从组件信息读取 Pmax，不猜测额定功率")
    panel_w = float(match.group(1))
    dc_w = int(metadata["Panel_Number"]) * panel_w
    source = ROOT / "data/processed" / f"{assumptions['station']}_q1_clean.csv"
    cleaned = pd.read_csv(source)
    times = pd.DatetimeIndex(pd.to_datetime(cleaned["time_utc"], utc=True))
    if times.hasnans or not times.is_unique:
        raise ValueError("清洗数据时间必须有效且唯一")
    solar, radiation, poa = clear_sky_poa(times, metadata, assumptions, assumptions["altitude_
m_assumed"])
    table = cleaned[["source_csv_row", "power_clean", "valid_power_for_q1", "needs_review"]].c
opy()
    table["time_utc"] = times
    table["time_local"] = times.tz_convert("Asia/Shanghai")
    table["solar_elevation_deg"] = solar["elevation"].to_numpy()
    table["is_daylight"] = table["solar_elevation_deg"] > 0
    for column in ("ghi", "dni", "dhi"):
        table[f"clear_sky_{column}_wm2"] = radiation[column].to_numpy()
    table["clear_sky_poa_wm2"] = poa.to_numpy()
    for loss in assumptions["extra_loss_fraction_scenarios"]:
        pdc, pac = reference_from_poa(poa, dc_w, assumptions["ac_limit_mw_assumed"] * 1e6,
                                      assumptions["inverter_nominal_efficiency_assumed"], loss)
        suffix = f"loss{round(loss * 100)}_mw"
        table["reference_dc_" + suffix] = pdc
        table["reference_ac_" + suffix] = pac
```

```
    _, _, high_poa = clear_sky_poa(times, metadata, assumptions, assumptions["altitude_sensiti
vity_m_assumed"])
    _, high_ac = reference_from_poa(high_poa, dc_w, assumptions["ac_limit_mw_assumed"] * 1e6,
                                    assumptions["inverter_nominal_efficiency_assumed"])
    table["reference_ac_altitude200_loss0_mw"] = high_ac
    table["deviation_actual_minus_reference_mw"] = table["power_clean"] - table["reference_ac_
loss0_mw"]
    reference_columns = list(table.filter(regex="^reference_ac_").columns)
    assert np.isfinite(table[reference_columns].to_numpy()).all()
    assert (table[reference_columns] >= 0).all().all()
    assert (table[reference_columns] <= assumptions["ac_limit_mw_assumed"] + 1e-10).all().all(
)
    assert table.loc[~table["is_daylight"], "reference_ac_loss0_mw"].eq(0).all()
    assert (table["reference_ac_loss0_mw"] >= table["reference_ac_loss10_mw"] - 1e-10).all()
    assert (table["reference_ac_loss10_mw"] >= table["reference_ac_loss20_mw"] - 1e-10).all()
    folder = ROOT / "results/q1" / assumptions["station"]
    save_csv(table, folder / "reference_power.csv")
    common = table.loc[table["is_daylight"] & table["valid_power_for_q1"]]
    rows = []
    for column in reference_columns:
        delta = common["power_clean"] - common[column]
        rows.append({"scenario": column, "common_daylight_points": len(common),
                     "reference_peak_mw": float(table[column].max()),
                     "mean_actual_minus_reference_mw": float(delta.mean()),
                     "mae_to_reference_mw": float(delta.abs().mean()),
                     "actual_exceeds_reference_points": int(delta.gt(0).sum())})
    save_csv(pd.DataFrame(rows), folder / "reference_sensitivity.csv")
    summary = {"model_version": assumptions["version"], "assumptions": assumptions,
               "cleaned_data_sha256": hashlib.sha256(source.read_bytes()).hexdigest(),
               "source_zip_sha256": source_sha256(archive, assumptions["station"]),
               "assumptions_sha256": hashlib.sha256(config_path.read_bytes()).hexdigest(),
               "source": str(archive.resolve()), "metadata": metadata.to_dict(), "pvlib_versio
n": pvlib.__version__, "rows": len(table),
               "panel_number": int(metadata["Panel_Number"]), "panel_pmax_wp": panel_w,
               "derived_dc_capacity_mwp": dc_w / 1e6, "assumed_ac_limit_mw": assumptions["ac_l
imit_mw_assumed"],
               "common_daylight_points": len(common), "scenarios": rows,
               "comparison_is_forecast_evaluation": False,
               "no_parameters_fitted_from_power": True}
    (folder / "reference_summary.json").write_text(json.dumps(summary, ensure_ascii=False, ind
ent=2), encoding="utf-8")
    plot_day(table, assumptions, ROOT / "figures/q1" / f"{assumptions['station']}_reference_da
y.png")
    print(json.dumps({key: summary[key] for key in ("model_version", "rows", "derived_dc_capac
ity_mwp", "scenarios")},
                     ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

src/q234_forecast.py

```
"""station08 问题 2–4：冻结七天预测、NWP 消融、单站统计降尺度及 C 交接稿。

AI 辅助实现；所有结论由本脚本实际运行生成。默认仅生成一个证据包、一张图、一份稿件。
运行：OMP_NUM_THREADS=2 .venv/bin/python src/q234_forecast.py
导出题目表 2：.venv/bin/python src/q234_forecast.py --export-table2 /指定目录/表 2.csv
"""
import argparse
import hashlib
import json
import platform
from pathlib import Path
import time

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
import numpy as np
import pandas as pd
import pvlib
import sklearn
from sklearn.ensemble import HistGradientBoostingRegressor
from threadpoolctl import threadpool_limits

from q1_check_data import ROOT, source_sha256
from q1_reference_power import clear_sky_poa, reference_from_poa

SEED = 20261014
CAPACITY = 20.0
TEST_MONTHS = (2, 5, 8, 11)
MODEL_NAMES = {
    'persistence': 'Q2 持续性', 'climatology': 'Q2 近 14 日气候态', 'physical': 'Q2 晴空比例',
    'hist_tree': 'Q2 历史树模型', 'q2_selected': 'Q2 验证选优',
    'nwp_tree': 'Q3 历史+NWP', 'nwp_radiation': 'Q3 仅加辐照', 'q3_gate': 'Q3 场景融合',
    'q4_raw_control': 'Q4 原始气象对照', 'q4_bias': 'Q4 均值偏差订正', 'q4_local': 'Q4 非线性局地
订正'}
CONFIGS = ({'max_leaf_nodes': 7, 'l2_regularization': 2.0},
           {'max_leaf_nodes': 15, 'l2_regularization': 2.0})


def test_mask(index):
    """最后一周定义为本地自然月最后 7 日，而非最后一个周一至周日。"""
    return np.asarray(index.month.isin(TEST_MONTHS) & (index.day > index.days_in_month - 7))


def metrics(actual, predicted, daylight, capacity=CAPACITY):
    a, p = np.asarray(actual, float), np.asarray(predicted, float)
    mask = np.asarray(daylight, bool) & np.isfinite(a) & np.isfinite(p)
    a, p = a[mask], p[mask]
    if not len(a):
        return {'n': 0, **{k: None for k in ('nRMSE', 'nMAE', 'nME', 'r', 'CR_pct', 'QR_pct')}
}
    error = (p-a)/capacity
    rmse = float(np.sqrt(np.mean(error**2)))
    corr = float(np.corrcoef(a, p)[0, 1]) if np.std(a)>0 and np.std(p)>0 else None
    return {'n': len(a), 'nRMSE': rmse, 'nMAE': float(np.mean(np.abs(error))),
            'nME': float(np.mean(error)), 'r': corr, 'CR_pct': (1-rmse)*100,
            'QR_pct': float(np.mean(np.abs(error)<.25)*100)}


def project(prediction, daylight):
    return np.where(daylight, np.clip(prediction, 0, CAPACITY), 0.0)


def load_data():
    path = ROOT/'data/processed/station08_q1_clean.csv'
    clean = pd.read_csv(path)
```

```
    reference = json.loads((ROOT/'results/q1/station08/reference_summary.json').read_text())
    assert hashlib.sha256(path.read_bytes()).hexdigest() == reference['cleaned_data_sha256'],
'清洗表与第一问记录版本不一致'
    cleaning = json.loads((ROOT/'results/q1/station08/cleaning_summary.json').read_text())
    assert source_sha256(ROOT/'data/PVOD', 'station08') == cleaning['source_sha256'], '原始输
入已变化，需先复核清洗版本'
    times = pd.DatetimeIndex(pd.to_datetime(clean.time_utc, utc=True)).tz_convert('Asia/Shangh
ai')
    assert times.is_unique and times.is_monotonic_increasing
    clean.index = times
    grid = pd.date_range(times.min().normalize(), times.max().normalize()+pd.Timedelta(hours=2
3, minutes=45), freq='15min')
    data = clean.reindex(grid)
    meta = pd.read_csv(ROOT/'data/PVOD/metadata.csv').set_index('Station_ID').loc['station08']
    assert float(meta.Capacity)/1000 == CAPACITY
    assumptions = json.loads((ROOT/'docs/q1_reference_assumptions_station08.json').read_text()
)
    solar, radiation, poa = clear_sky_poa(grid.tz_convert('UTC'), meta, assumptions, 0)
    _, pac = reference_from_poa(poa, 78042*265, CAPACITY*1e6, .96)
    data['reference_mw'] = pac
    data['clear_ghi'] = radiation.ghi.to_numpy()
    data['elevation'] = solar.elevation.to_numpy()
    data['azimuth'] = solar.azimuth.to_numpy()
    data['daylight'] = data.elevation > 0
    data['test'] = test_mask(grid)
    return data


def geometry(data):
    i = data.index
    clock = (i.hour*60+i.minute).to_numpy()/1440
    year = i.dayofyear.to_numpy()/365.25
    return pd.DataFrame({'clear_power': data.reference_mw/CAPACITY,
        'clear_ghi': data.clear_ghi/1000, 'elevation': data.elevation/90,
        'az_sin': np.sin(np.deg2rad(data.azimuth)), 'az_cos': np.cos(np.deg2rad(data.azimuth))
,
        'clock_sin': np.sin(2*np.pi*clock), 'clock_cos': np.cos(2*np.pi*clock),
        'year_sin': np.sin(2*np.pi*year), 'year_cos': np.cos(2*np.pi*year)}, index=i)


def weather_features(data):
    x = geometry(data)
    for name in ('globalirrad', 'directirrad', 'temperature', 'humidity', 'windspeed', 'pressu
re'):
        x['nwp_'+name] = data['nwp_'+name]
    angle = np.deg2rad(data.nwp_winddirection)
    x['nwp_wind_sin'], x['nwp_wind_cos'] = np.sin(angle), np.cos(angle)
    x['nwp_clear_index'] = data.nwp_globalirrad/np.maximum(data.clear_ghi, 20)
    return x


def history_features(data, issue, targets):
    """全部功率输入严格早于 issue；四个测试周的功率始终不参与历史输入。"""
    hist = data.loc[(data.index < issue) & (data.index >= issue-pd.Timedelta(days=28)) & ~data
.test].copy()
    hist['slot'] = hist.index.hour*4+hist.index.minute//15
    hist['date'] = hist.index.strftime('%Y-%m-%d')
    days = hist.groupby('date').power_clean.agg(['count', 'sum'])
    complete = days.index[days['count'].eq(96)]
    if not len(complete):
        raise ValueError(f'{issue}之前没有完整历史功率日')
    last = hist.loc[hist.date.eq(complete[-1])].set_index('slot').power_clean
    slots = targets.hour*4+targets.minute//15
    recent = hist.loc[hist.index >= issue-pd.Timedelta(days=14)]
    prof = recent.groupby('slot').power_clean.agg(['mean', 'median', 'std'])
    profile = prof.reindex(slots)
    recent7 = hist.loc[hist.index >= issue-pd.Timedelta(days=7)]
    valid = hist.loc[hist.daylight & hist.power_clean.notna() & (hist.reference_mw > .5)]
    scale = np.dot(valid.power_clean, valid.reference_mw)/np.dot(valid.reference_mw, valid.ref
erence_mw)
```

```
    x = geometry(data.loc[targets]).copy()
    x['horizon_day'] = (targets.normalize()-issue).days
    x['last_profile'] = last.reindex(slots).to_numpy()/CAPACITY
    x['recent_mean'] = profile['mean'].to_numpy()/CAPACITY
    x['recent_median'] = profile['median'].to_numpy()/CAPACITY
    x['recent_std'] = profile['std'].to_numpy()/CAPACITY
    x['recent_daily_cf'] = recent7.power_clean.mean()/CAPACITY
    x['historical_scale'] = scale
    physical = data.loc[targets].reference_mw.to_numpy()*scale
    baselines = {'persistence': last.reindex(slots).to_numpy(),
                 'climatology': profile['mean'].to_numpy(), 'physical': physical}
    # 可复核输入上界；返回值只用于审计，不作为模型特征。
    return x, baselines, hist.index.max()


def episode(data, issue):
    targets = pd.date_range(issue+pd.Timedelta(days=1), periods=672, freq='15min')
    x, base, last_history = history_features(data, issue, targets)
    weather = weather_features(data.loc[targets])
    full = x.join(weather.loc[:, weather.columns.str.startswith('nwp_')])
    radiation = x.join(weather[['nwp_globalirrad', 'nwp_directirrad', 'nwp_clear_index']])
    info = data.loc[targets, ['power_clean', 'daylight', 'test', 'elevation']].copy()
    info['issue'], info['last_history'] = issue, last_history
    info['horizon_day'] = (targets.normalize()-issue).days
    info['physical'] = base['physical']
    return x, full, radiation, info, base


def train_pool(data):
    blocks = []
    last = data.index.max().normalize()-pd.Timedelta(days=7)
    for issue in pd.date_range(data.index.min().normalize()+pd.Timedelta(days=14), last, freq=
'3D'):
        x, nwp, rad, info, _ = episode(data, issue)
        blocks.append((x, nwp, rad, info))
    return tuple(pd.concat([b[k] for b in blocks]) for k in range(4))


def fit_tree(x, y, config, seed=SEED):
    model = HistGradientBoostingRegressor(max_iter=180, learning_rate=.05, min_samples_leaf=30
,
        early_stopping=False, random_state=seed, **config)
    model.fit(x, y)
    return model


def fit_power_models(pool, cutoff, config_by_name, exclude_dates=(), seed=SEED):
    info = pool[3]
    mask = ~info.test & info.power_clean.notna() & info.daylight
    if cutoff is not None:
        mask &= (info.index < cutoff) & (info.issue < cutoff)
    if exclude_dates:
        mask &= ~info.index.normalize().isin(exclude_dates)
    # 重叠起报使同一个目标可能出现多次：给每个目标相等总权重不在HGB接口中实现，报告重复数。
    residual = (info.power_clean-info.physical)/CAPACITY
    models = {name: fit_tree(pool[k].loc[mask], residual.loc[mask], config_by_name[name], seed
)
              for k, name in enumerate(('hist_tree', 'nwp_tree', 'nwp_radiation'))}
    return models, {'training_episode_rows': int(mask.sum()), 'unique_training_times': int(inf
o.index[mask].nunique()),
                    'max_train_target': info.index[mask].max().isoformat(), 'test_labels_in_tr
aining': int(info.loc[mask].test.sum())}


def predict_power(models, ep):
    output = {name: project(value, ep[3].daylight) for name, value in ep[4].items()}
    for k, name in enumerate(('hist_tree', 'nwp_tree', 'nwp_radiation')):
        pred = models[name].predict(ep[k])*CAPACITY + ep[3].physical.to_numpy()
        output[name] = project(pred, ep[3].daylight)
    return output
```

```
def scene_codes(data):
    """按预报日的白昼晴空指数划分；预先固定阈值，不使用测试实测值。"""
    ratio = data.nwp_globalirrad/np.maximum(data.clear_ghi, 20)
    ratio = ratio.where(data.elevation>10)
    daily = ratio.groupby(data.index.normalize()).mean()
    value = daily.reindex(data.index.normalize()).to_numpy()
    return np.select([value<=.35, value<=.65], ['low', 'medium'], default='high')


def choose_gate(actual, predictions, daylight, codes):
    q2_names = ('persistence', 'climatology', 'physical', 'hist_tree')
    selected = min(q2_names, key=lambda n: metrics(actual, predictions[n], daylight)['nRMSE'])
    weights = {}
    mask = np.asarray(daylight, bool)
    for code in ('all', 'low', 'medium', 'high'):
        valid = mask if code=='all' else mask & (codes==code)
        if valid.sum()<20:
            weights[code] = weights['all']
        else:
            weights[code] = min((0, .25, .5, .75, 1), key=lambda w:
                metrics(actual, (1-w)*predictions[selected]+w*predictions['nwp_tree'], valid)[
'nRMSE'])
    return selected, weights


def apply_gate(predictions, selected, weights, codes):
    output = dict(predictions)
    output['q2_selected'] = output[selected]
    w = np.array([weights[c] for c in codes])
    output['q3_gate'] = (1-w)*output[selected]+w*output['nwp_tree']
    return output


def local_power_features(data, ghi, temperature):
    x = geometry(data)
    x['local_ghi'], x['local_temperature'] = np.asarray(ghi), np.asarray(temperature)
    return x


def fit_downscaling(data, cutoff, exclude_dates=(), seed=SEED):
    mask = ~data.test & data.power_clean.notna() & data.daylight
    if cutoff is not None:
        mask &= data.index < cutoff
    if exclude_dates:
        mask &= ~data.index.normalize().isin(exclude_dates)
    mask &= data.lmd_totalirrad_clean.notna() & data.lmd_temperature_clean.notna()
    d = data.loc[mask]
    wx = weather_features(d)
    config = {'max_leaf_nodes': 15, 'l2_regularization': 2.0}
    ghi = fit_tree(wx, d.lmd_totalirrad_clean/1000, config, seed)
    temperature = fit_tree(wx, d.lmd_temperature_clean-d.nwp_temperature, config, seed)
    power = fit_tree(local_power_features(d, d.lmd_totalirrad_clean, d.lmd_temperature_clean),
 d.power_clean/CAPACITY, config, seed)
    bias = {'ghi': float((d.lmd_totalirrad_clean-d.nwp_globalirrad).mean()),
            'temperature': float((d.lmd_temperature_clean-d.nwp_temperature).mean())}
    return (ghi, temperature, power, bias), {'local_training_points': len(d), 'max_local_train
ing_time': d.index.max().isoformat()}


def predict_downscaling(models, data):
    gm, tm, pm, bias = models
    wx = weather_features(data)
    ghi = np.maximum(gm.predict(wx)*1000, 0)
    temp = tm.predict(wx)+data.nwp_temperature.to_numpy()
    candidates = {'q4_raw_control': (data.nwp_globalirrad, data.nwp_temperature),
                  'q4_bias': (np.maximum(data.nwp_globalirrad+bias['ghi'], 0), data.nwp_temper
ature+bias['temperature']),
                  'q4_local': (ghi, temp)}
```

```
    output = {name: project(pm.predict(local_power_features(data, g, t))*CAPACITY, data.daylig
ht)
              for name, (g, t) in candidates.items()}
    return output, ghi, temp


def validation_and_test(data, pool, start):
    issue = start-pd.Timedelta(days=1)
    val_issue = issue-pd.Timedelta(days=8)
    val = episode(data, val_issue)
    valid_target = ~val[3].test & val[3].power_clean.notna()
    val = tuple(item.loc[valid_target] if isinstance(item, pd.DataFrame) else
                {k: v[np.asarray(valid_target)] for k,v in item.items()} for item in val)
    configs = {}
    validation_rows = []
    for config in CONFIGS:
        models, _ = fit_power_models(pool, val_issue, {name: config for name in ('hist_tree',
'nwp_tree', 'nwp_radiation')})
        pred = predict_power(models, val)
        validation_rows.extend({'week': start.strftime('%Y-%m'), 'config': config, 'model': na
me,
            **metrics(val[3].power_clean, pred[name], val[3].daylight)} for name in ('hist_tre
e','nwp_tree','nwp_radiation'))
    for name in ('hist_tree','nwp_tree','nwp_radiation'):
        candidates = [r for r in validation_rows if r['model']==name]
        configs[name] = min(candidates, key=lambda r:r['nRMSE'])['config']
    models, _ = fit_power_models(pool, val_issue, configs)
    vpred = predict_power(models, val)
    codes = scene_codes(data.loc[val[3].index])
    selected, weights = choose_gate(val[3].power_clean, vpred, val[3].daylight, codes)
    # 拟合最终模型前验证集归回可用历史，保持测试标签永不参与。
    models, audit = fit_power_models(pool, issue, configs)
    ep = episode(data, issue)
    pred = apply_gate(predict_power(models, ep), selected, weights, scene_codes(data.loc[ep[3]
.index]))
    downs, audit_local = fit_downscaling(data, issue)
    q4, ghi, temp = predict_downscaling(downs, data.loc[ep[3].index])
    pred.update(q4)
    audit.update(audit_local)
    audit.update({'week': start.strftime('%Y-%m'), 'issue': issue.isoformat(), 'validation_iss
ue': val_issue.isoformat(),
        'validation_points': len(val[3]), 'validation_last_target': val[3].index.max().isoform
at(),
        'history_last_timestamp': ep[3].last_history.iloc[0].isoformat(), 'q2_selected': selec
ted,
        'gate_weights': weights, 'tree_configs': configs})
    assert pd.Timestamp(audit['max_train_target']) < issue
    assert pd.Timestamp(audit['max_local_training_time']) < issue
    assert pd.Timestamp(audit['history_last_timestamp']) < issue
    assert ep[3].test.all() and len(ep[3])==672 and ep[3].power_clean.notna().all()
    return ep, pred, ghi, temp, audit, validation_rows


def result_rows(predictions, info, protocol):
    rows = []
    codes = info.scene.to_numpy()
    for model, values in predictions.items():
        groups = [('overall', 'all', np.ones(len(info), bool))]
        groups += [('month', month, (info.week==month).to_numpy()) for month in info.week.uniq
ue()]
        groups += [('horizon', str(h), (info.horizon_day==h).to_numpy()) for h in range(1,8)]
        groups += [('scene', code, codes==code) for code in ('low','medium','high')]
        for kind, group, mask in groups:
            rows.append({'protocol': protocol, 'model': model, 'group_type': kind, 'group': gr
oup,
                         **metrics(info.power_clean, values, info.daylight.to_numpy() & mask)}
)
    return rows
```

```
def bootstrap_improvement(info, first, second, name):
    """28 个日期为独立单元，在每个 7 天测试周内重抽日期；不给 15 分钟点伪造独立性。"""
    d = pd.DataFrame({'date': info.index.strftime('%Y-%m-%d'), 'week': info.week,
        'e1': ((np.asarray(first)-info.power_clean)/CAPACITY)**2,
        'e2': ((np.asarray(second)-info.power_clean)/CAPACITY)**2,
        'daylight': info.daylight}).loc[info.daylight]
    daily = d.groupby(['week','date']).agg(e1=('e1','sum'), e2=('e2','sum'), n=('e1','size'))
    rng = np.random.default_rng(SEED)
    weeks = [g.to_numpy() for _,g in daily.groupby(level=0)]
    delta = []
    for _ in range(2000):
        sample = np.concatenate([w[rng.integers(0,len(w),len(w))] for w in weeks])
        mse1, mse2, n = sample.sum(axis=0)
        delta.append(np.sqrt(mse1/n)-np.sqrt(mse2/n))
    delta = np.array(delta)
    point = metrics(info.power_clean, first, info.daylight)['nRMSE']-metrics(info.power_clean,
 second, info.daylight)['nRMSE']
    return {'comparison': name, 'n_dates': len(daily), 'delta_nRMSE_first_minus_second': point
,
            'ci95_low': float(np.quantile(delta,.025)), 'ci95_high': float(np.quantile(delta,.
975)),
            'bootstrap_samples': 2000, 'scope': '仅四个固定测试周，按周内日期分层重抽，不代表
全年或跨站推断'}


def draw_summary(info, predictions, out):
    fonts = {f.name for f in font_manager.fontManager.ttflist}
    plt.rcParams['font.family'] = next((f for f in ('PingFang HK','Songti SC','Heiti TC','SimH
ei') if f in fonts),'DejaVu Sans')
    plt.rcParams['axes.unicode_minus'] = False
    fig, axes = plt.subplots(3,2,figsize=(15,12))
    for ax, week in zip(axes[:2].flat, info.week.unique()):
        mask = info.week.eq(week)
        # 显示固定周的第一日；全周逐点结果在证据包，避免挤成无法读的图。
        mask &= info.index.normalize()==info.index[mask][0].normalize()
        x = info.index[mask].hour+info.index[mask].minute/60
        ax.plot(x, info.loc[mask,'power_clean'], color='#222222',label='实测',linewidth=1.8)
        for name,color in [('q2_selected','#ac7a34'),('nwp_tree','#28689e'),('q4_local','#4c82
62')]:
            ax.plot(x,np.asarray(predictions[name])[mask],label=MODEL_NAMES[name],color=color,
linewidth=1.3)
        ax.set_title(f'{week}测试周首日（北京时间）')
        ax.set(xlabel='本地时刻（h）',ylabel='功率（MW）',xlim=(0,24),ylim=(0,20))
        ax.grid(alpha=.2)
    axes[0,0].legend(fontsize=9)
    selected = ('persistence','climatology','physical','hist_tree','q2_selected','nwp_radiatio
n','nwp_tree','q3_gate','q4_raw_control','q4_bias','q4_local')
    scores = [metrics(info.power_clean,predictions[n],info.daylight)['nRMSE'] for n in selecte
d]
    axes[2,0].barh([MODEL_NAMES[n] for n in selected],scores,color='#28689e')
    axes[2,0].invert_yaxis();axes[2,0].set_xlabel('白昼容量归一化 RMSE（越小越好）')
    for j,score in enumerate(scores): axes[2,0].text(score+.002,j,f'{score:.3f}',va='center',f
ontsize=8)
    axes[2,0].set_xlim(0,max(scores)*1.25)
    for name,color in [('q2_selected','#ac7a34'),('nwp_tree','#28689e'),('q4_local','#4c8262')
]:
        scores = [metrics(info.power_clean,predictions[name],info.daylight & info.horizon_day.
eq(h))['nRMSE'] for h in range(1,8)]
        axes[2,1].plot(range(1,8),scores,'o-',label=MODEL_NAMES[name],color=color)
    axes[2,1].set(xlabel='目标日序号（起报次日起 1–7）',ylabel='白昼容量归一化 RMSE')
    axes[2,1].legend(fontsize=9);axes[2,1].grid(alpha=.2)
    fig.suptitle('station08：问题 2–4 向前回测与局地气象订正',fontsize=17)
    fig.text(.03,.015,'固定起报七天；历史输入与训练标签早于起报。NWP 版本可用性未核实；单点订正
不证明细网格重建。',fontsize=10)
    fig.tight_layout(rect=(0,.04,1,.97));fig.savefig(out,dpi=300);plt.close(fig)


def md_table(records, columns, rename=None):
    frame = pd.DataFrame(records)[columns].rename(columns=rename or {})
    def fmt(value):
```

```
        if value is None or (isinstance(value,float) and not np.isfinite(value)): return '—'
        if isinstance(value,(float,np.floating)): return f'{value:.4f}'
        return str(value)
    return '\n'.join(['| '+' | '.join(frame.columns)+' |','| '+' | '.join(['---']*len(frame.co
lumns))+' |']+
        ['| '+' | '.join(fmt(v) for v in r)+' |' for r in frame.itertuples(index=False,name=No
ne)])


def report(info, predictions, summary, path):
    rows = summary['metrics']
    overall = [dict(r,model=MODEL_NAMES[r['model']]) for r in rows if r['protocol']=='forward'
 and r['group_type']=='overall']
    retrospective = [dict(r,model=MODEL_NAMES[r['model']]) for r in rows if r['protocol']=='re
trospective' and r['group_type']=='overall']
    score = {r['model']:r for r in rows if r['protocol']=='forward' and r['group_type']=='over
all'}
    q2, q3, q4 = [score[n]['nRMSE'] for n in ('q2_selected','nwp_tree','q4_local')]
    delta3, delta4 = (q2-q3)/q2*100, (score['q4_raw_control']['nRMSE']-q4)/score['q4_raw_contr
ol']['nRMSE']*100
    monthly = [r for r in rows if r['protocol']=='forward' and r['group_type']=='month' and r[
'model'] in ('q2_selected','nwp_tree','q3_gate','q4_raw_control','q4_local')]
    scenes = [dict(r, model=MODEL_NAMES[r['model']]) for r in rows if r['protocol']=='forward'
 and r['group_type']=='scene' and r['model'] in ('q2_selected','nwp_tree','q3_gate','q4_local'
)]
    horizons = [dict(r, model=MODEL_NAMES[r['model']]) for r in rows if r['protocol']=='forwar
d' and r['group_type']=='horizon' and r['model'] in ('q2_selected','nwp_tree','q4_local')]
    rank = {'model':'模型','n':'白昼样本','nRMSE':'归一RMSE','nMAE':'归一MAE','nME':'归一ME','
CR_pct':'CR(%)','QR_pct':'QR(%)'}
    table = md_table(overall,['model','n','nRMSE','nMAE','nME','r','CR_pct','QR_pct'],rank)
    checks = summary['checks']
    result3 = '降低' if delta3>=0 else '增加'
    result4 = '降低' if delta4>=0 else '增加'
    winner = min(score,key=lambda k:score[k]['nRMSE'])
    scene_lookup = {(r['group'], r['model']): r for r in rows if r['protocol']=='forward' and
r['group_type']=='scene'}
    scene_lines = []
    for code in ('low', 'medium', 'high'):
        first, second = scene_lookup[(code,'q2_selected')], scene_lookup[(code,'nwp_tree')]
        if first['n']:
            gain = (first['nRMSE']-second['nRMSE'])/first['nRMSE']*100
            scene_lines.append(f"{code}档（{first['n']}点）：{first['nRMSE']:.4f}→{second['nR
MSE']:.4f}，相对变化{-gain:+.2f}%")
    scene_analysis = '；'.join(scene_lines)+'。负百分数表示误差降低。场景融合总体RMSE为'+f"{sc
ore['q3_gate']['nRMSE']:.4f}"+'，没有超过直接全字段NWP模型，不能据此宣称场景门控必然更优。低档
样本较少，场景结论限于本次测试。'
    month_lookup = {(r['group'], r['model']): r for r in monthly}
    month_lines = []
    for month in info.week.unique():
        first, second = month_lookup[(month,'q2_selected')], month_lookup[(month,'nwp_tree')]
        gain = (first['nRMSE']-second['nRMSE'])/first['nRMSE']*100
        month_lines.append(f"{month}相对变化{-gain:+.2f}%")
    month_analysis = '；'.join(month_lines)+'。该现象说明不能把总体改善写成各季节均改善。'
    q4_analysis = f"向前回测非线性订正RMSE={q4:.4f}，原始气象对照={score['q4_raw_control']['nR
MSE']:.4f}，总体未改善；均值订正为{score['q4_bias']['nRMSE']:.4f}，同样没有改善。GHI的MAE变小
但RMSE变大，说明平均绝对偏差改善不足以保证大误差尾部改善；这与早期样本少、非线性外推及两阶段输
入分布变化相容，但尚未证明唯一原因。气象订正后总体功率也未超过Q3，不能称为空间降尺度成功。"
    retro_lookup = {r['model']:r for r in rows if r['protocol']=='retrospective' and r['group_
type']=='overall'}
    retro_analysis = f"离线对照中，Q4原始气象RMSE={retro_lookup['q4_raw_control']['nRMSE']:.4f
}，非线性订正={retro_lookup['q4_local']['nRMSE']:.4f}。离线结果与向前结果不同，更多日期用于拟
合可能影响分布覆盖；它无法证明过去起报时已经可以获得这些未来训练样本，故不作为运行收益证据。"
    ci = summary['bootstrap'][0]
    significance = f"NWP相对第二问的RMSE差为{ci['delta_nRMSE_first_minus_second']:.4f}，区间[{
ci['ci95_low']:.4f}, {ci['ci95_high']:.4f}]跨0。因此可以报告本次测试点估计改善，但不能声称已经
在日期层面证明稳定显著收益。"
    text = rf'''# station08问题2、3、4：C交接总结与论文初稿

整理日期：2026-10-14。用途：建模练习、数字复核与正文整理；尚不能宣称完成真实NWP版本可用性及细
网格空间验证。
```

给 C 的交接说明

本次核心交付只有`src/q234_forecast.py`和本稿。逐点预测、指标表、配置、输入与源代码哈希集中在`results/q234/station08/证据包.npz`，图集中为一张`汇总图.png`。不新增清洗表，不生成散落的模型文件和重复报告。问题 1 稿件保留，需由 C 合并已有第一问与本稿；旧报告已清理；本次建模的数字以证据包为准，C 的待办统一见根目录`C 剩余任务.md`。

```bash
OMP_NUM_THREADS=2 .venv/bin/python src/q234_forecast.py
.venv/bin/python src/q234_forecast.py --inspect
.venv/bin/python src/q234_forecast.py --export-table2 /指定位置/表2.csv
.venv/bin/python -m unittest discover -s tests -v
```

第三条只在需要比赛附件时生成 CSV：包含四周全部 2688 条记录、每周固定起报时间、目标时间、实测及各方法预测。起报时间为目标周开始前一日 00:00，目标范围为起报 24 至 191.75 小时，七天内不更新功率输入。表 2 样例存在每日不同起报，本稿采用题目文字允许的七天固定起报实验；如果团队决定每日滚动 24–48 小时，应单独重做该口径，不把两种实验混写。

摘要可用素材（与正式摘要分开）

在 station08 四个规定测试周、{int(info.daylight.sum())}个白昼样本上，历史功率验证选优模型归一化 RMSE 为{q2:.4f}；加入 NWP 后为{q3:.4f}，相对前者{result3}{abs(delta3):.2f}%。单站非线性局地气象订正为{q4:.4f}，相对同功率映射的原始气象对照{result4}{abs(delta4):.2f}%。所有模型按规定测试周评分；向前实验训练和历史输入早于起报，测试标签不参与训练或选参。这些结果只能用于给定数据与信息假设的回测，NWP 是否在起报时覆盖全部七天仍待核验。

1 题目、数据与统一评价

题目第二问要求仅根据历史功率预测；第三问加入 NWP 并验证有效场景；第四问检验空间降尺度。数据为 PVOD station08（36.70761°N、113.89999°E，朝南 33°，容量 20 MW）、沿用已清洗的`data/processed/station08_q1_clean.csv`和第一问参考假设。原始 UTC 转北京时间；2018-07-01 至 2019-06-13，共 33120 个实测点、345 个完整日，缺 2018-11-11 至 11-13 整 3 日，不补零。全部日期网格用于太阳几何计算，缺测标签始终不训练、不评分。

按本地自然月末 7 日，测试为 2018-08-25–08-31、2018-11-24–11-30、2019-02-22–02-28、2019-05-25–05-31，各 672 点，共 2688 点；白昼采用几何太阳高度角>0，不用实际功率决定白昼。其余日期为候选训练集。第一问 25°C 晴空参考只提供确定性的太阳几何基线，实际温度不被冒充为未来组件温度。

两种评价协议必须区分。 向前协议在每个起报时刻只用此前的非测试标签训练；它少于“其余日期全部训练”，用于核对时间因果关系。补充离线协议使用整个数据范围的非测试日期训练，符合题目静态划分，但训练中可以存在晚于早期测试周的日期，所以不能当成真实向前预测能力。两者测试范围相同，均永久排除四周功率作为模型标签或历史输入。

NWP 只有有效时间行，没有发行时刻、预报版本、提前量或邻域格点坐标；计算假设对应 NWP 列已在该次起报时可获得。因此第 3/4 问应称**NWP 可用性假设下的回测**，不能宣称已证明完整七天运行预报。作者仓库可核实 NWP 与现场测量字段分类，但未提供本次输入中的版本索引[1]。

容量归一化误差定义$e_i=(\widehat P_i-P_i)/C$，$C=20$ MW：

$$E_{{rmse}}= \sqrt{{n^{{-1}}\sum_i e_i^2}},\qquad E_{{mae}}=n^{{-1}}\sum_i|e_i|,\qquad E_{{me}}=n^{{-1}}\sum_i e_i.$$

$$CR=(1-E_{{rmse}})\times100\%,\qquad QR=n^{{-1}}\sum_i\mathbf1(|e_i|<0.25)\times100\%.$$

另报告 Pearson 相关系数。附件 1 的容量符号是开机容量；当前没有开机容量记录，20 MW 是装机容量代替开机容量的明确假设。原附件第一段符号解释与相关公式段对实际/预测符号有冲突，本稿明确 ME 正值表示预测偏高，避免符号混淆。CR 和 QR 都按百分数报告，QR 严格小于 0.25。指标不能与第一问“实际相对晴空参考的偏差”混用。

2 第二问：仅历史功率预测

2.1 模型与输入

考虑三类方法：持续性与近 14 日同刻均值易解释，作为朴素基线；晴空比例提供确定日周期；梯度提升树学习历史状态与季节残差，复杂度可控。深度神经网络因有效观测不足一年、最早测试周训练样本少，本次不采用。

持续性重复起报前最后一个完整非测试日的 96 点；气候态为起报前 14 天非测试日同刻功率均值，缺日不加入分母；晴空比例系数由起报前 28 天有效白昼功率和参考功率作过原点最小二乘：

$$a(o)=\frac{{\sum_{{t<o}}P_tP_{{cs,t}}}}{{\sum_{{t<o}}P_{{cs,t}}^2}},\qquad \widehat P_{{phys,t}}=a(o)P_{{cs,t}}.$$

太阳高度角、方位角、时刻和年内周期编码、参考功率、目标日序号，加上最后完整日同刻功率、14 日同刻均值/中位数/标准差、最近 7 日平均功率与历史晴空系数，形成特征向量$x_{{o,t}}$。历史功率全部早于o，七天目标日共用起报时历史，不进行真值递推。第二问没有 NWP 和现场气象输入。

以物理基线残差为回归目标：

$$\widehat P_{{2,t}}=\Pi_{{[0,C]}}\left[a(o)P_{{cs,t}}+C f_2(x_{{o,t}})\right].$$

其中f_2为直方图梯度提升树。所有模型夜间输出置 0，白昼输出投影至[0,20] MW。这里只裁剪预测，没有改变真实评分功率。

2.2 训练与选参

候选历史起报从首日后 14 天开始、每 3 天一次，每次生成七天目标。首段历史不足的日期不作为残差树训练目标，仍可用作历史输入；“其余日期”是候选标签集，并非每个日期都能构造足够历史的七天训练样本。训练标签排除缺失、夜间及四个测试周。一个目标可能对应多个起报提前量，属于直接多步训练而非独立样本；显著性检验不把这些重复目标当独立证据。验证起报为测试起报前 8 天，验证目标为其后第 1–7 天，验证模型只用验证起报之前标签训练。

树参数：学习率 0.05、180 轮、叶节点候选 7/15、叶子最少 30 样本、L2=2、随机种子{SEED}、关闭随机内部早停。各方法按独立验证周白昼 RMSE 选叶节点，第二问主方案由四个历史候选模型按验证 RMSE 选优。测试真值完全不参与选参。最终向前拟合纳入测试起报前的验证标签，仍不纳入任何测试周标签。

2.3 数值结果

{table}

上表同时包含第三、四问以方便交接统一复核。不能因为测试表中某个候选较好就回头替换验证选优规则；本次全表最低为{MODEL_NAMES[winner]}，这只是固定测试样本的事后描述。验证选优方案没有在总体测试中超过全部简单基线；晴空比例为{score['physical']['nRMSE']:.4f}，历史树模型为{score['hist_tree']['nRMSE']:.4f}，说明仅靠历史信息时增加非线性复杂度并不必然提高精度。

3 第三问：NWP 融合及场景验证

3.1 融合与消融

在第二问相同历史特征与残差目标上增加 NWP 全球辐照、直射辐照、气温、湿度、风速、压力、风向正余弦和预报晴空指数，得到f_3。NWP 原始列不是未来现场测量。辐照消融只加全球/直射辐照和预报晴空指数，其余结构一致；比较它与全字段模型可以检查非辐照信息是否带来额外收益。

3.2 场景划分及融合

对目标自然日太阳高度角>10°的预报点，计算$k_d=mean(G_{{NWP}}/G_{{cs}})$，分母至少 20 W/m²。预先固定低辐照($k_d\le0.35$)、中辐照($0.35<k_d\le0.65$)、高辐照($k_d>0.65$)三类。场景只依赖该日预报与太阳几何，不读取当日功率或现场天气。这些名称是预报辐照档，不应直接写成已确认阴雨/云/雾类别。

场景融合为$\widehat P_s=(1-w_s)\widehat P_2+w_s\widehat P_3$，$w_s$从 0、0.25、0.5、0.75、1 中按验证误差选择；若某类验证白昼点不足 20，则沿用总体最优权重。允许不启用 NWP，避免预设融合必然有效。

{md_table(scenes,['model','group','n','nRMSE','nMAE'],dict(rank,group='预报辐照档'))}

{scene_analysis}

证据包可查看各场景权重、验证样本数与分月结果。若某类无测试样本，显示空值，不虚构验证结论；场景改善与总体改善均应按数值及下面日块重抽区间解释。

3.3 分月结果

{md_table([dict(r,model=MODEL_NAMES[r['model']]) for r in monthly],['group','model','n','nRMSE','QR_pct'],dict(rank,group='测试月'))}

{month_analysis}

4 第四问：从NWP到电站局地气象的统计映射

4.1 可验证范围

当前只有单站NWP序列与对应现场测量，没有NWP网格间距、原始格点坐标及邻域场，不能做双线性/IDW插值，也不能展示真实1km→100m的空间分辨率提升。这里实现单点统计降尺度的有限形式，即“模式尺度预报量→电站测点气象”的回归订正，验证其对站点功率回测的作用。严格细网格空间降尺度仍未完成，正文和摘要必须保留此边界。

4.2 双阶段方法和对照

只用训练时段现场辐照与气温作为标签，学习$D_G(NWP,solar)$和$D_T(NWP,solar)$；辐照目标单位按1000 W/m²缩放，温度学习现场气温减NWP气温的残差。两者采用15叶、180轮的固定提升树。随后只用训练时段真实局地辐照、气温与太阳几何学习相同的功率映射F：

$$\widehat P_{{raw}}=F(G_{{NWP}},T_{{NWP}},solar),\quad
\widehat P_{{bias}}=F(G_{{NWP}}+\overline{{G_{{obs}}-G_{{NWP}}}},T_{{NWP}}+\overline{{T_{{obs}}-T_{{NWP}}}},solar),$$

$$\widehat P_{{local}}=F(D_G(NWP,solar),T_{{NWP}}+D_T(NWP,solar),solar).$$

同一F用于三组，原始气象对照隔离映射输入订正带来的增益；均值订正隔离简单偏差修正与非线性效应。测试时没有真实局地辐照或气温作为预测输入，现场测量只在生成预测后用于独立气象误差评价。同时与第三问强基线比较，避免用弱对照掩盖两阶段模型的整体效果。

{md_table(summary['weather_metrics'],['variable','input','n','RMSE','MAE'],{'variable':'现场标签','input':'气象输入','n':'白昼点','RMSE':'RMSE(原单位)','MAE':'MAE(原单位)'})}

{q4_analysis}

气象映射可用于拟合给定样本中的条件偏差，但不产生未观测的云场空间结构；即使功率误差降低，也不能归因为恢复次网格云影位置或识别具体故障。

5 检验与适用边界

5.1 配对日期块检验

比较采用四个测试周内按日期分层重抽，共2000次，统计第一模型RMSE减第二模型RMSE的95%分位区间；正值表示第二模型较好，跨0表示证据不足。独立单元是28个日期，而非大量相互相关的15分钟点。区间条件于四个固定测试周，不能外推为全年显著性。相邻日期仍可能存在持续天气相关，且每次仅有一个验证周，区间和模型选择稳定性均需更多独立日期复核。

{md_table(summary['bootstrap'],['comparison','n_dates','delta_nRMSE_first_minus_second','ci95_low','ci95_high'],{'comparison':'第一→第二','n_dates':'日期数','delta_nRMSE_first_minus_second':'RMSE差','ci95_low':'区间下界','ci95_high':'区间上界'})}

{significance}

5.2 提前量与白昼口径

{md_table(horizons,['model','group','n','nRMSE'],dict(rank,group='目标日序号'))}

每个序号在四个测试周只有4个目标日，不足以证明误差必须随提前量单调增长；NWP行又未按提前量存档，因此该表也是固定目标周的样本比较。

{md_table(summary['daylight_sensitivity'],['model','solar_threshold_deg','n','nRMSE'],dict(rank,solar_threshold_deg='太阳高度阈值(°)'))}

0°为正式评分口径，5°只作敏感性。降低RMSE可能只是删除低太阳高度角样本，不能叫模型改进。

5.3 离线题目划分对照

{md_table(retrospective,['model','n','nRMSE','nMAE','CR_pct','QR_pct'],rank)}

离线对照训练使用整段非测试日期（包括部分早期测试之后的日期），树模型固定采用向前验证选得的末次配置，第二问方案及融合权重保留对应起报之前的验证选择。它有全时段日历信息，但没有测试功率标签/现场测量作为测试预测特征，不与向前结果混成一张准确率结论。

{retro_analysis}

5.4 样本与泄漏核对

{md_table(summary['audits'],['week','issue','max_train_target','unique_training_times','history_last_timestamp','q2_selected'],{'week':'测试月','issue':'起报','max_train_target':'最晚训练标签','unique_training_times':'唯一训练点','history_last_timestamp':'最晚历史输入','q2_selected':'第二问验证选择'})}

检查结果：{json.dumps(checks,ensure_ascii=False)}。

6 结论、未决项与 C 整理清单

第二问已经完成历史基线、残差树模型及验证选优；第三问已经完成 NWP 字段消融、预报场景划分与验证融合；第四问已经完成单站气象统计映射的有限实证，但未获得多格点空间场。是否改善只能依据上述表格和区间，不把“模型更复杂”当成结论。

向前总体结果：Q2={q2:.4f}，Q3={q3:.4f}，Q4={q4:.4f}；第 3 问相对 Q2{result3}{abs(delta3):.2f}%，第 4 问相对同功率映射的原始气象对照{result4}{abs(delta4):.2f}%。第 4 问若未超过第三问，则应明确两阶段误差传播及映射分布变化的可能影响，而不能只报对照增益。

C 整理时应保留：装机容量替代开机容量、ME 符号约定、白昼定义、固定起报实验、跨年不完整观测、NWP 发布时间与七天版本待核实、单点订正与细网格空间降尺度的区别。数据不足一年与 3 个缺日不因完成预测而消失。预报场景不能当真实天气分类；提升树重要性也不能当因果归因。本稿由 AI 辅助实现及起草，团队应复核方法与数字并按适用赛事要求披露。

建议正文结构：把本稿第 2 节整合为第二问，第 3 节为第三问，第 4 节为第四问，第 1/5 节作为共享评价与模型检验。不要复制旧报告中的不同样本数、旧实验或未经核实的故障解释。若要完成严格第四问，下一步需要同期邻近格点 NWP 坐标、空间分辨率及发行版本；不能把其他电站的站点 NWP 冒充规则网格。

![汇总图](../results/q234/station08/汇总图.png)

参考与运行信息

[1] Yao T, Wang J, Wu H, et al. A photovoltaic power output dataset: Multi-source photovoltaic power output dataset with Python toolkit. Solar Energy, 2021, 230:122–130. DOI:10.1016/j.solener.2021.09.050；[作者仓库](https://github.com/yaotc/PVODataset)。本次核实字段说明，访问日期 2026-10-14。

[2] scikit-learn 开发者。[HistGradientBoostingRegressor 官方接口](https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html)。访问日期 2026-10-14。

[3] 本地原题`docs/B 题 光伏电站发电功率日前预测问题.pdf`与`docs/附件 1 误差分析方法.pdf`；附件公式已视觉复核。

运行版本：{json.dumps(summary['versions'],ensure_ascii=False)}；固定随机种子{SEED}。源代码、清洗表、参数及题目哈希见证据包 metadata，全部数字与逐点预测同包保存，可用`--inspect`复核。
'''
    path.write_text(text,encoding='utf-8')

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--inspect',action='store_true')
    parser.add_argument('--export-table2',type=Path)
    args = parser.parse_args()
    out = ROOT/'results/q234/station08'
    bundle_path = out/'证据包.npz'
    if args.inspect or args.export_table2:
        with np.load(bundle_path,allow_pickle=False) as bundle:
            summary = json.loads(str(bundle['metadata_json']))
            if args.inspect:
                print(json.dumps({k:summary[k] for k in ('versions','audits','checks','bootstrap')},ensure_ascii=False,indent=2))
            if args.export_table2:
                table = pd.DataFrame({'起报时间':bundle['issue_time'], '预报时间':bundle['target_time'],
                    '实际功率(MW)':bundle['actual_mw']})
                for name in bundle['model_names']:
                    table[MODEL_NAMES[str(name)]+'(MW)'] = bundle['forward_'+str(name)]
                args.export_table2.parent.mkdir(parents=True,exist_ok=True)

```
                table.to_csv(args.export_table2,index=False,encoding='utf-8-sig')
                print(f'已导出{len(table)}条预测：{args.export_table2}')
        return
    began = time.perf_counter()
    data = load_data()
    print('输入加载完成；建立严格起报历史特征。',flush=True)
    pool = train_pool(data)
    starts = data.loc[data.test].groupby(data.loc[data.test].index.strftime('%Y-%m')).apply(la
mbda g:g.index.min()).tolist()
    records, predictions, ghis, temps, audits, validations = [], [], [], [], [], []
    with threadpool_limits(limits=2):
        for start in starts:
            print(f'向前回测{start:%Y-%m}：验证、选参和冻结七天预测。',flush=True)
            ep, pred, ghi, temp, audit, validation = validation_and_test(data,pool,start)
            info = ep[3].copy()
            info['week'] = start.strftime('%Y-%m')
            info['scene'] = scene_codes(data.loc[info.index])
            records.append(info);predictions.append(pred);ghis.append(ghi);temps.append(temp)
            audits.append(audit);validations.extend(validation)
        info = pd.concat(records)
        forward = {n:np.concatenate([p[n] for p in predictions]) for n in predictions[0]}
        print('向前回测完成：计算题目静态划分离线对照。',flush=True)
        configs = audits[-1]['tree_configs']
        retro_models, retro_audit = fit_power_models(pool,None,configs)
        retro_downs, retro_local_audit = fit_downscaling(data,None)
        retrospective = []
        for start, audit in zip(starts, audits):
            ep = episode(data,start-pd.Timedelta(days=1))
            pred = apply_gate(predict_power(retro_models,ep),audit['q2_selected'],audit['gate_
weights'],scene_codes(data.loc[ep[3].index]))
            pred.update(predict_downscaling(retro_downs,data.loc[ep[3].index])[0])
            retrospective.append(pred)
        retro = {n:np.concatenate([p[n] for p in retrospective]) for n in retrospective[0]}
    rows = result_rows(forward,info,'forward')+result_rows(retro,info,'retrospective')
    weather_rows = []
    target = data.loc[info.index]
    for variable, raw, mapped, actual in [
        ('GHI(W/m²)',target.nwp_globalirrad,np.concatenate(ghis),target.lmd_totalirrad_clean),
        ('气温(°C)',target.nwp_temperature,np.concatenate(temps),target.lmd_temperature_clean)
]:
        for name,pred in [('原始NWP',raw),('非线性局地订正',mapped)]:
            mask = target.daylight & np.isfinite(actual) & np.isfinite(pred)
            delta = np.asarray(pred)[mask]-actual[mask].to_numpy()
            weather_rows.append({'variable':variable,'input':name,'n':int(mask.sum()),'RMSE':f
loat(np.sqrt(np.mean(delta**2))),'MAE':float(np.mean(np.abs(delta)))})
    boot = [bootstrap_improvement(info,forward[a],forward[b],f'{MODEL_NAMES[a]}→{MODEL_NAMES[
b]}') for a,b in
        [('q2_selected','nwp_tree'),('q2_selected','q3_gate'),('q4_raw_control','q4_local'),('
nwp_tree','q4_local')]]
    sensitivity = [{'model':MODEL_NAMES[name],'solar_threshold_deg':threshold,
        **metrics(info.power_clean,forward[name],info.elevation>threshold)}
        for name in ('q2_selected','nwp_tree','q4_local') for threshold in (0,5)]
    checks = {'test_points':len(info),'each_week_672':bool(info.groupby('week').size().eq(672)
.all()),
        'actual_no_missing':bool(info.power_clean.notna().all()),
        'all_forward_training_before_issue':all(pd.Timestamp(a['max_train_target'])<pd.Timesta
mp(a['issue']) for a in audits),
        'test_training_labels':sum(a['test_labels_in_training'] for a in audits)+retro_audit['
test_labels_in_training'],
        'predictions_finite_and_in_capacity':all(np.isfinite(v).all() and (v>=0).all() and (v<
=CAPACITY).all() for v in (*forward.values(),*retro.values())),
        'all_night_predictions_zero':all((v[~info.daylight.to_numpy()]==0).all() for v in (*fo
rward.values(),*retro.values())),
        'nwp_issue_availability_verified':False,'multigrid_spatial_downscaling_verified':False
}
    assert checks['test_points']==2688 and checks['each_week_672'] and checks['actual_no_missi
ng']
    assert checks['all_forward_training_before_issue'] and checks['test_training_labels']==0
    assert checks['predictions_finite_and_in_capacity'] and checks['all_night_predictions_zero
']
```

```
    inputs = [Path(__file__),ROOT/'data/processed/station08_q1_clean.csv',ROOT/'data/PVOD/metadata.csv',
        ROOT/'docs/q1_reference_assumptions_station08.json',ROOT/'docs/B题 光伏电站发电功率日前预测问题.pdf',ROOT/'docs/附件1 误差分析方法.pdf',
        ROOT/'requirements.txt',ROOT/'requirements-lock.txt',ROOT/'src/q1_check_data.py',ROOT/'src/q1_reference_power.py',
        ROOT/'results/q1/station08/cleaning_summary.json',ROOT/'results/q1/station08/reference_summary.json',
        ROOT/'docs/q1_cleaning_rules_station08.json',ROOT/'tests/test_q234_forecast.py']
    summary = {'version':'q234-station08-v1','date':'2026-10-14','station':'station08','capacity_mw':CAPACITY,
        'seed':SEED,'versions':{'python':platform.python_version(),'numpy':np.__version__,'pandas':pd.__version__,'sklearn':sklearn.__version__,'pvlib':pvlib.__version__},
        'metrics':rows,'audits':audits,'validation':validations,'retrospective_audit':dict(retro_audit,**retro_local_audit),
        'weather_metrics':weather_rows,'bootstrap':boot,'daylight_sensitivity':sensitivity,'checks':checks,
        'input_sha256':{str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in inputs},
        'wall_seconds':time.perf_counter()-began}
    out.mkdir(parents=True,exist_ok=True)
    arrays = {'metadata_json':np.array(json.dumps(summary,ensure_ascii=False)),
        'model_names':np.array(list(forward)),'issue_time':info.issue.astype(str).to_numpy(dtype=str),
        'target_time':info.index.astype(str).to_numpy(dtype=str),'actual_mw':info.power_clean.to_numpy(),
        'daylight':info.daylight.to_numpy(),'scene':info.scene.to_numpy(dtype=str),'horizon_day':info.horizon_day.to_numpy(),
        'elevation_deg':info.elevation.to_numpy(), 'week':info.week.to_numpy(dtype=str),
        'actual_ghi_wm2':target.lmd_totalirrad_clean.to_numpy(), 'actual_temperature_c':target.lmd_temperature_clean.to_numpy(),
        'raw_ghi_wm2':target.nwp_globalirrad.to_numpy(), 'raw_temperature_c':target.nwp_temperature.to_numpy(),
        'mapped_ghi_wm2':np.concatenate(ghis),'mapped_temperature_c':np.concatenate(temps)}
    arrays.update({'forward_'+n:v for n,v in forward.items()})
    arrays.update({'retrospective_'+n:v for n,v in retro.items()})
    np.savez_compressed(bundle_path,**arrays)
    draw_summary(info,forward,out/'汇总图.png')
    draft = ROOT/'paper/第2至4问_station08_C交接初稿.md'
    report(info,forward,summary,draft)
    print(json.dumps({'draft':str(draft),'wall_seconds':summary['wall_seconds'],
        'overall':[r for r in rows if r['protocol']=='forward' and r['group_type']=='overall'], 'checks':checks},ensure_ascii=False,indent=2))


if __name__=='__main__':
    main()
```

tests/test_q1_characteristics.py

```
"""防止缺测日被当完整日，以及跨缺口制造 15 分钟变化。"""

from pathlib import Path
import sys
import unittest

import numpy as np
import pandas as pd

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))
from q1_characteristics import complete_daily_integrals, continuous_changes


class CharacteristicsTests(unittest.TestCase):
    def setUp(self):
        self.day = pd.DataFrame({
            "time_local": pd.date_range("2019-05-01", periods=96, freq="15min", tz="Asia/Shang
hai"),
            "power_clean": np.ones(96), "power": np.ones(96),
            "reference_ac_loss0_mw": np.ones(96) * 2, "needs_review": np.zeros(96, dtype=bool)
,
        })

    def test_constant_one_mw_integrates_to_twenty_four_mwh(self):
        daily = complete_daily_integrals(self.day)
        self.assertTrue(daily.loc[0, "complete_power_day"])
        self.assertEqual(daily.loc[0, "actual_daily_integral_estimate_mwh"], 24.0)
        self.assertEqual(daily.loc[0, "reference_daily_integral_same_day_mwh"], 48.0)

    def test_one_missing_power_does_not_become_zero_or_a_complete_day(self):
        self.day.loc[40, "power_clean"] = np.nan
        daily = complete_daily_integrals(self.day)
        self.assertFalse(daily.loc[0, "complete_power_day"])
        self.assertTrue(pd.isna(daily.loc[0, "actual_daily_integral_estimate_mwh"]))
        self.assertTrue(pd.isna(daily.loc[0, "reference_daily_integral_same_day_mwh"]))

    def test_ninety_six_rows_with_duplicate_time_are_not_a_complete_grid(self):
        self.day.loc[40, "time_local"] = self.day.loc[39, "time_local"]
        daily = complete_daily_integrals(self.day)
        self.assertFalse(daily.loc[0, "complete_grid"])
        self.assertTrue(pd.isna(daily.loc[0, "actual_daily_integral_estimate_mwh"]))

    def test_ramp_does_not_bridge_time_gap_or_missing_power(self):
        frame = pd.DataFrame({
            "source_csv_row": [2, 3, 4, 5],
            "time_local": pd.to_datetime(["2019-05-01 10:00", "2019-05-01 10:15",
                                          "2019-05-01 10:45", "2019-05-01 11:00"]).tz_localiz
e("Asia/Shanghai"),
            "power_clean": [0, 1, 3, np.nan], "lmd_totalirrad_clean": [0, 100, 300, 400],
            "is_daylight": [True] * 4,
        })
        ramps = continuous_changes(frame)
        self.assertEqual(ramps.loc[1, "power_change_15min_mw"], 1.0)
        self.assertTrue(ramps.loc[[0, 2, 3], "power_change_15min_mw"].isna().all())
        self.assertEqual(int(ramps["valid_daylight_weather_power_pair"].sum()), 1)


if __name__ == "__main__":
    unittest.main()
```

tests/test_q1_clean_data.py

```
"""验证会影响统计的清洗边界：连续性、昼夜证据、首点和真实零值。"""

import json
from pathlib import Path
import sys
import unittest

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
from q1_clean_data import clean_local_values, find_frozen_runs


class CleaningBoundaryTests(unittest.TestCase):
    def setUp(self):
        self.rules = json.loads((ROOT / "docs/q1_cleaning_rules.json").read_text())
        self.data = pd.DataFrame({
            "time_local": pd.date_range("2018-12-08 17:00", periods=6, freq="15min", tz="Asia/
Shanghai"),
            "source_csv_row": range(2, 8), "power": [10.0] * 6,
            "lmd_totalirrad": [400.0] * 6, "lmd_diffuseirrad": [80.0] * 6,
            "lmd_temperature": [10.0] * 6, "lmd_pressure": [1000.0] * 6,
            "lmd_winddirection": [180.0] * 6, "lmd_windspeed": [2.0] * 6,
            "nwp_globalirrad": [50.0, 20.0, 0.0, 0.0, 0.0, 0.0],
            "solar_elevation_deg": [-2.0, -5.0, -8.0, -11.0, -14.0, -17.0],
            "is_daylight": [False] * 6, "capacity_mw": [20.0] * 6,
        })

    def test_frozen_repeat_is_masked_but_anchor_and_raw_are_preserved(self):
        cleaned, log, runs = clean_local_values(self.data, self.rules)
        pd.testing.assert_frame_equal(cleaned[self.data.columns], self.data)
        self.assertEqual(len(cleaned), 6)
        self.assertEqual(cleaned.loc[0, "power_clean"], 10.0)
        self.assertTrue(cleaned.loc[1:, "power_clean"].isna().all())
        self.assertTrue(cleaned.loc[0, "flag_frozen_anchor_review"])
        self.assertEqual(len(log), 5 * 7)
        self.assertEqual(len(runs), 1)
        pd.testing.assert_series_equal(cleaned["nwp_globalirrad"], self.data["nwp_globalirrad"
])

    def test_gap_does_not_join_short_runs_into_a_frozen_block(self):
        frame = self.data.iloc[:4].copy()
        frame.loc[2:, "time_local"] += pd.Timedelta(hours=1)
        candidate, repeats, runs = find_frozen_runs(frame, self.rules)
        self.assertFalse(candidate.any())
        self.assertFalse(repeats.any())
        self.assertTrue(runs.empty)

    def test_positive_flat_daytime_is_not_removed_without_night_evidence(self):
        frame = self.data.copy()
        frame["solar_elevation_deg"] = 30.0
        frame["is_daylight"] = True
        cleaned, log, runs = clean_local_values(frame, self.rules)
        self.assertTrue(runs.empty)
        self.assertTrue(log.empty)
        self.assertTrue(cleaned["power_clean"].eq(10).all())

    def test_ordinary_zero_night_is_not_a_frozen_failure(self):
        frame = self.data.copy()
        frame["power"] = 0.0
        frame["lmd_totalirrad"] = 0.0
        cleaned, log, runs = clean_local_values(frame, self.rules)
        self.assertTrue(runs.empty)
        self.assertTrue(log.empty)
        self.assertTrue(cleaned["power_clean"].eq(0).all())
```

```
    def test_daytime_zero_with_changing_weather_remains_observed_zero(self):
        frame = self.data.copy()
        frame["power"] = 0.0
        frame["is_daylight"] = True
        frame["solar_elevation_deg"] = 30.0
        frame["lmd_temperature"] = np.arange(6) + 10.0
        cleaned, log, runs = clean_local_values(frame, self.rules)
        self.assertTrue(runs.empty)
        self.assertTrue(log.empty)
        self.assertTrue(cleaned["flag_day_low_power_raw"].all())
        self.assertTrue(cleaned["valid_power_for_q1"].all())
        self.assertTrue(cleaned["power_clean"].eq(0).all())


if __name__ == "__main__":
    unittest.main()
```

tests/test_q1_reference_power.py

```
"""检验参考模型的单位、DC/AC 容量口径和损耗情景。"""

from pathlib import Path
import sys
import unittest

import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))
from q1_reference_power import reference_from_poa


class ReferencePowerTests(unittest.TestCase):
    def test_zero_irradiance_produces_zero_dc_and_ac(self):
        dc, ac = reference_from_poa([0], 19.98e6, 20e6, 0.96)
        self.assertEqual(dc[0], 0)
        self.assertEqual(ac[0], 0)

    def test_dc_capacity_at_reference_irradiance_and_separate_ac_limit(self):
        dc, ac = reference_from_poa([1000, 2000], 19.98e6, 20e6, 0.96)
        np.testing.assert_allclose(dc, [19.98, 39.96])
        self.assertLess(ac[0], 19.98)
        self.assertAlmostEqual(ac[1], 20.0)

    def test_increasing_loss_does_not_increase_ac_power(self):
        outputs = [reference_from_poa([100, 500, 1000, 2000], 19.98e6, 20e6, 0.96, loss)[1]
                   for loss in [0, 0.1, 0.2]]
        self.assertTrue((outputs[0] >= outputs[1]).all())
        self.assertTrue((outputs[1] >= outputs[2]).all())


if __name__ == "__main__":
    unittest.main()
```

tests/test_q234_forecast.py

```
"""问题2–4的预测信息边界与附件指标检验。"""
from pathlib import Path
import sys
import unittest
import numpy as np
import pandas as pd

sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'src'))
from q234_forecast import history_features, metrics, project, test_mask


class ForecastBoundaryTests(unittest.TestCase):
    def data(self):
        index = pd.date_range('2018-08-01',periods=45*96,freq='15min',tz='Asia/Shanghai')
        return pd.DataFrame({'power_clean':np.sin(np.arange(len(index))/96)**2*10,
            'reference_mw':10.,'clear_ghi':800.,'elevation':45.,'azimuth':180.,
            'daylight':True,'test':False,'nwp_globalirrad':500.,'lmd_totalirrad_clean':500.},i
ndex=index)

    def test_future_power_and_weather_cannot_change_history_model_input(self):
        data = self.data()
        issue = pd.Timestamp('2018-08-25',tz='Asia/Shanghai')
        target = pd.date_range(issue+pd.Timedelta(days=1),periods=672,freq='15min')
        x,baseline,last = history_features(data,issue,target)
        modified = data.copy()
        modified.loc[modified.index>=issue,'power_clean'] = 999.
        modified['nwp_globalirrad'],modified['lmd_totalirrad_clean'] = -999.,-999.
        other,b2,last2 = history_features(modified,issue,target)
        pd.testing.assert_frame_equal(x,other)
        for name in baseline: np.testing.assert_array_equal(baseline[name],b2[name])
        self.assertLess(last,issue);self.assertEqual(last,last2)

    def test_preceding_test_power_is_not_used_as_history(self):
        data = self.data()
        data.loc[data.index.day>=25,'test'] = True
        issue = pd.Timestamp('2018-09-03',tz='Asia/Shanghai')
        target = pd.date_range(issue+pd.Timedelta(days=1),periods=672,freq='15min')
        x,baseline,_ = history_features(data,issue,target)
        data.loc[data.test,'power_clean'] = 9999.
        other,b2,_ = history_features(data,issue,target)
        pd.testing.assert_frame_equal(x,other)
        for name in baseline: np.testing.assert_array_equal(baseline[name],b2[name])

    def test_qr_uses_strict_threshold_cr_is_percent_and_me_is_pred_minus_actual(self):
        score = metrics([0,0,999],[4,5,999],[True,True,False],20)
        self.assertEqual(score['n'],2)
        self.assertEqual(score['QR_pct'],50.)
        self.assertAlmostEqual(score['nME'],.225)
        self.assertAlmostEqual(score['nRMSE'],np.sqrt((.2**2+.25**2)/2))
        self.assertAlmostEqual(score['CR_pct'],(1-score['nRMSE'])*100)

    def test_last_seven_natural_days_and_no_night_truth_mask(self):
        dates = pd.date_range('2019-02-01',periods=28,freq='D',tz='Asia/Shanghai')
        mask = test_mask(dates)
        self.assertEqual(mask.sum(),7)
        self.assertEqual(dates[mask][0].day,22)
        values = project([-3,30,5],[True,True,False])
        np.testing.assert_array_equal(values,[0,20,0])


if __name__=='__main__': unittest.main()
```